

CHAPTER 6

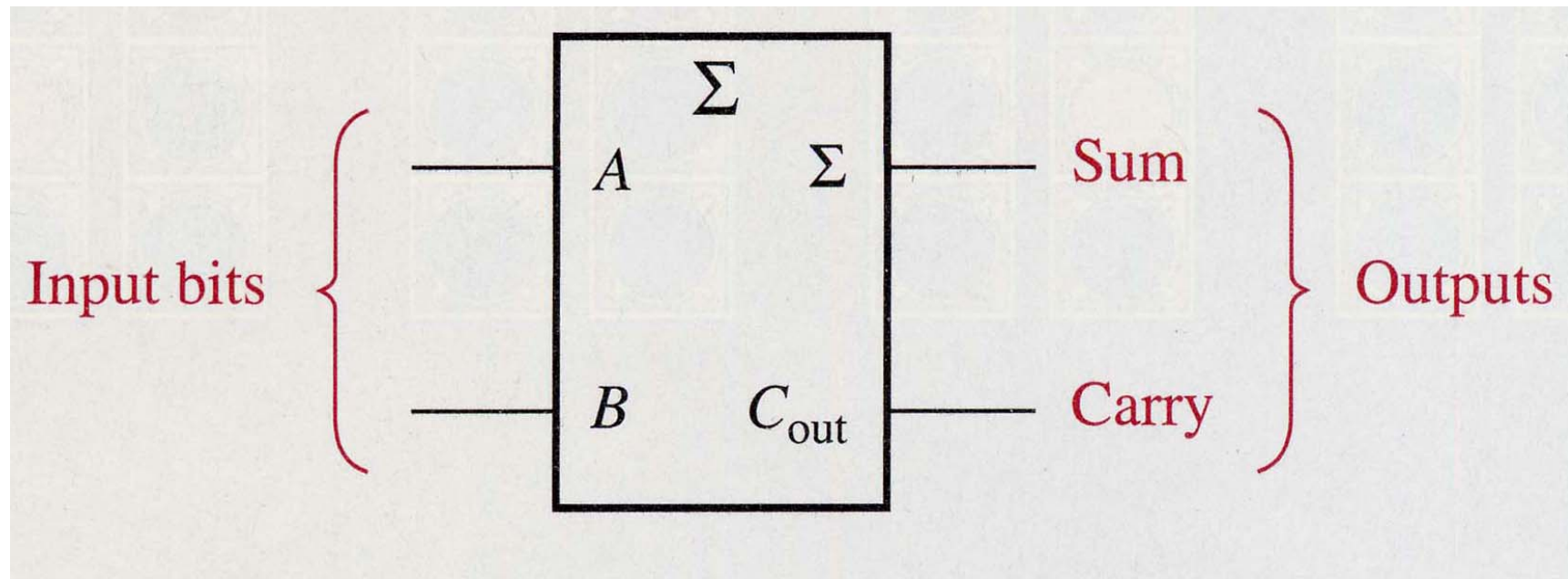
Functions of Combinational Logic

组合逻辑电路函数

1 Adders

- Half-Adder (半加器)
- Full-Adder (全加器)
- Parallel Binary Adder 并行二进制加法器

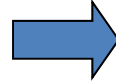
Half-Adder (I)



Logic symbol

Half-Adder (II)

0	+	0	=	0
0	+	1	=	1
1	+	0	=	1
1	+	1	=	10



A	B	C_{out}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

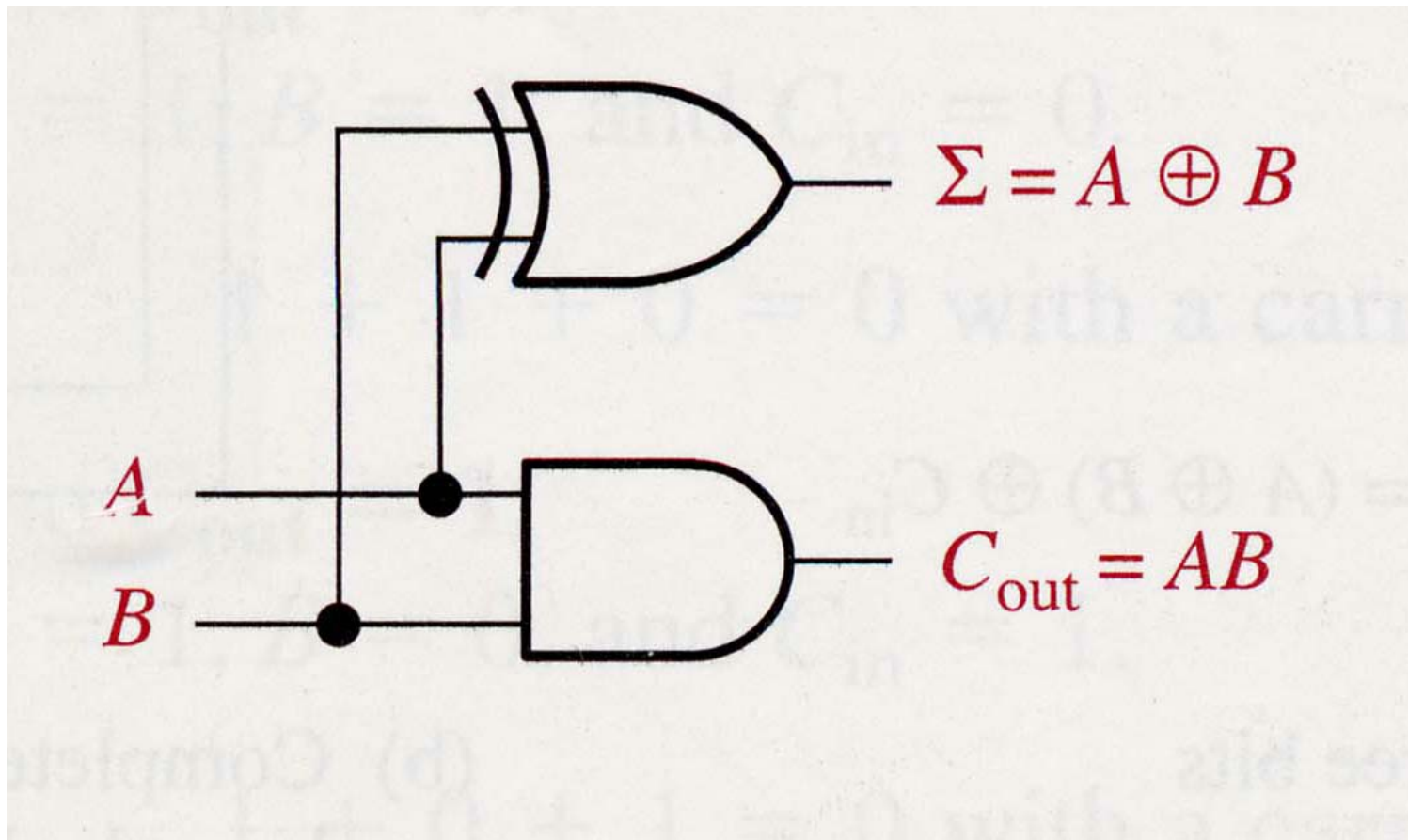


$$\Sigma = A \oplus B$$

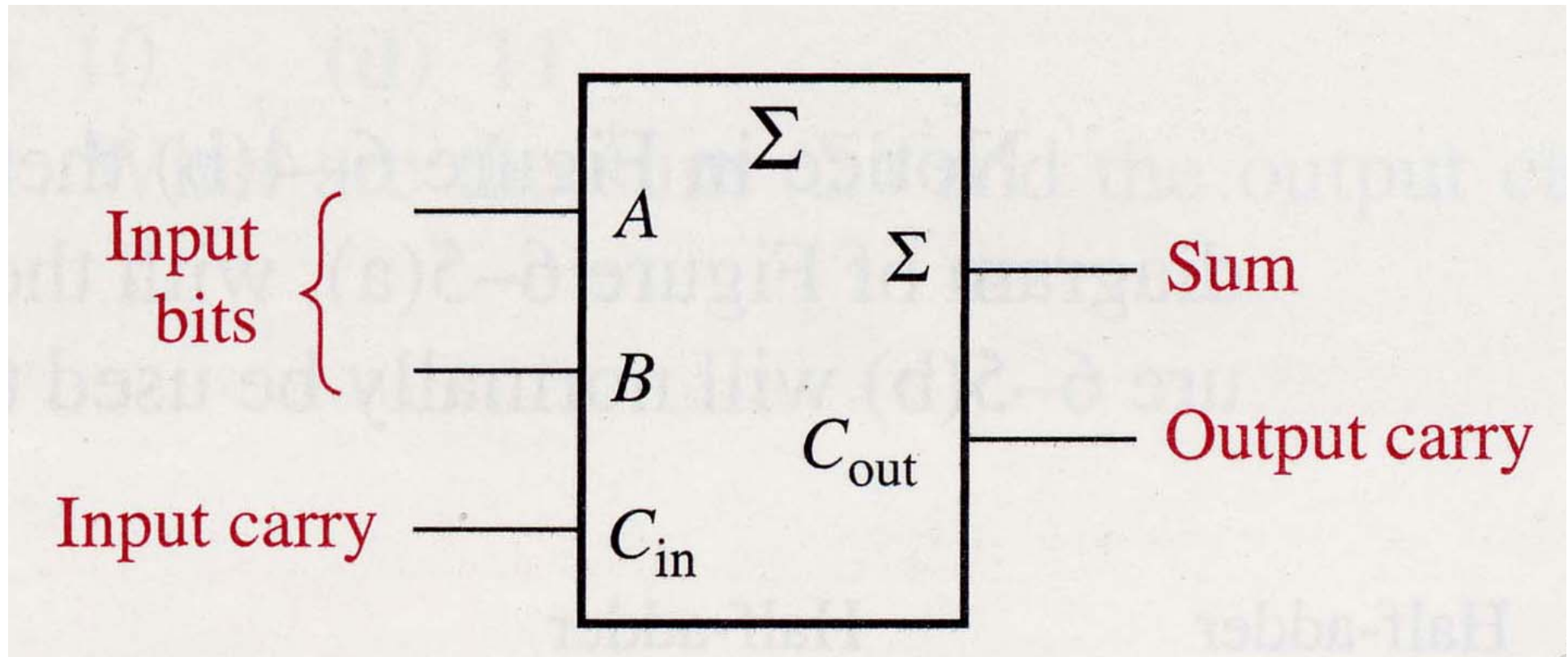
$$C_{out} = AB$$

Half-Adder (III)

Logic diagram



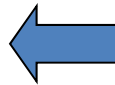
Full-Adder (I)



Full-Adder(II)

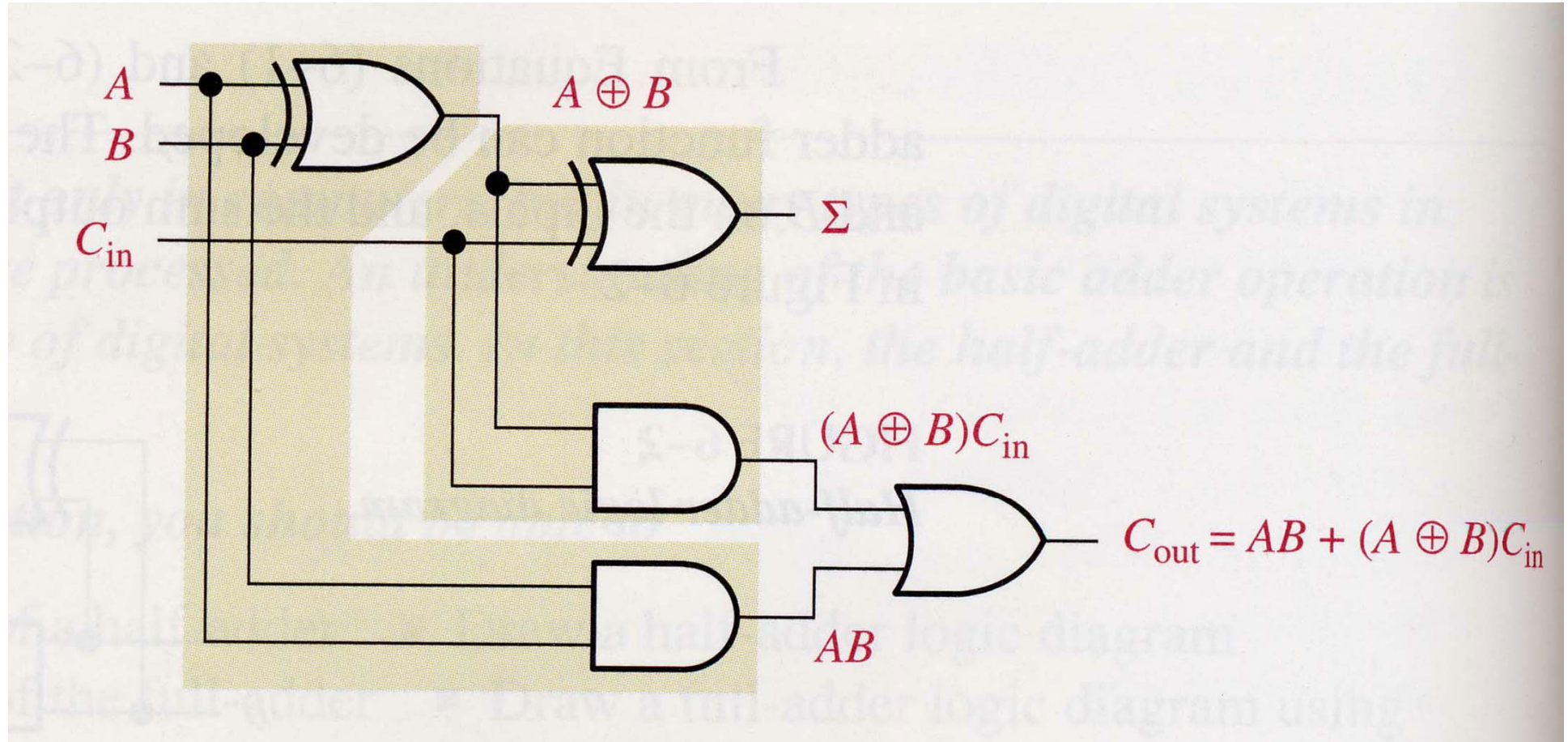
$$\Sigma = (A \oplus B) \oplus C_{in}$$

$$C_{out} = AB + (A \oplus B)C_{in}$$

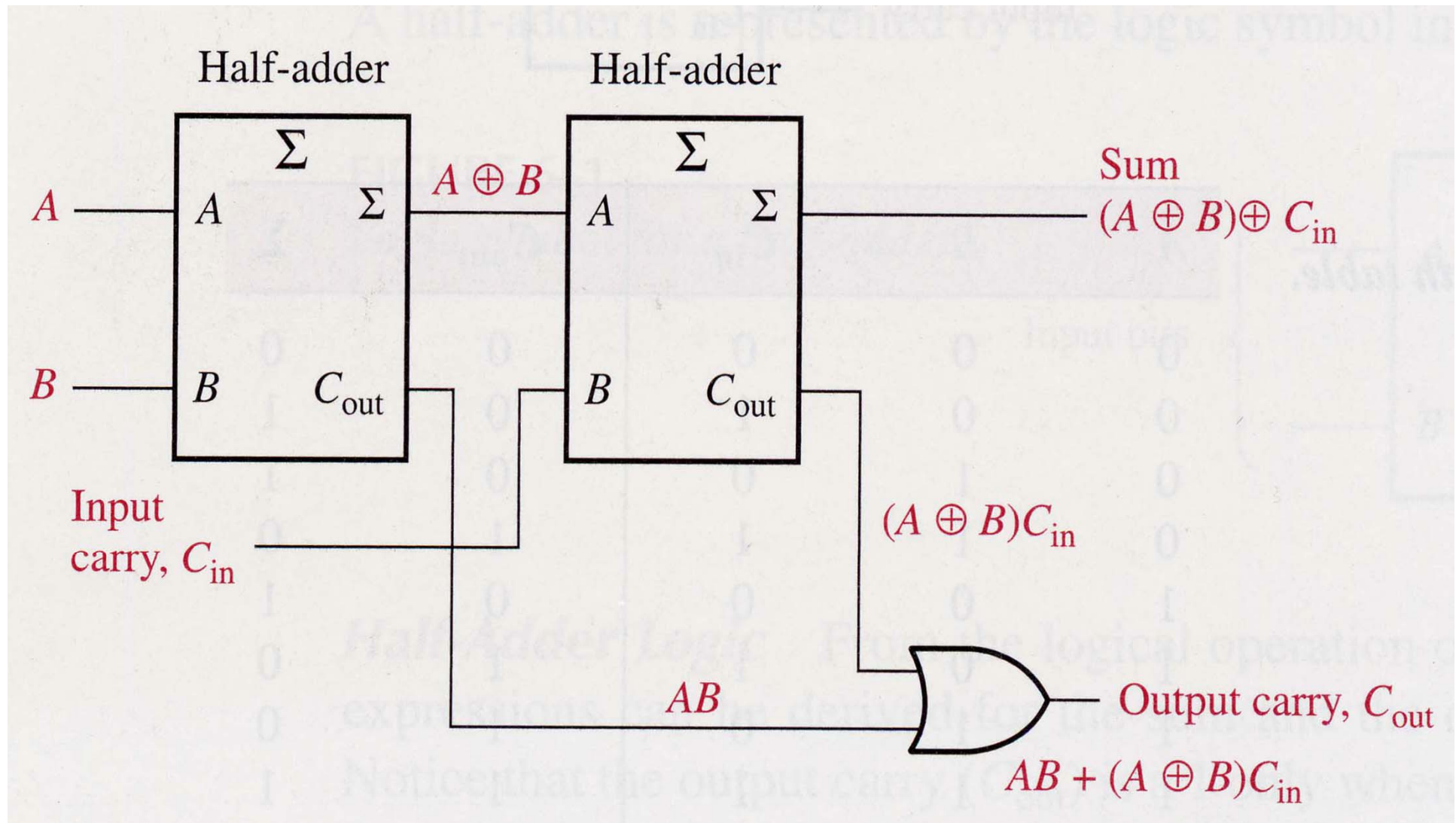


A	B	C _{in}	C _{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Logic Diagram



Full-adder implemented with two half-adders.



Parallel Binary Adders

- Two or more full-adders are connected to form parallel binary adders.
- A single full-adders is capable of adding two 1-bit binary numbers and an input carry. To add binary numbers with more than one bit, additional full-adders must be used.

Addition of Two-bit Binary Number

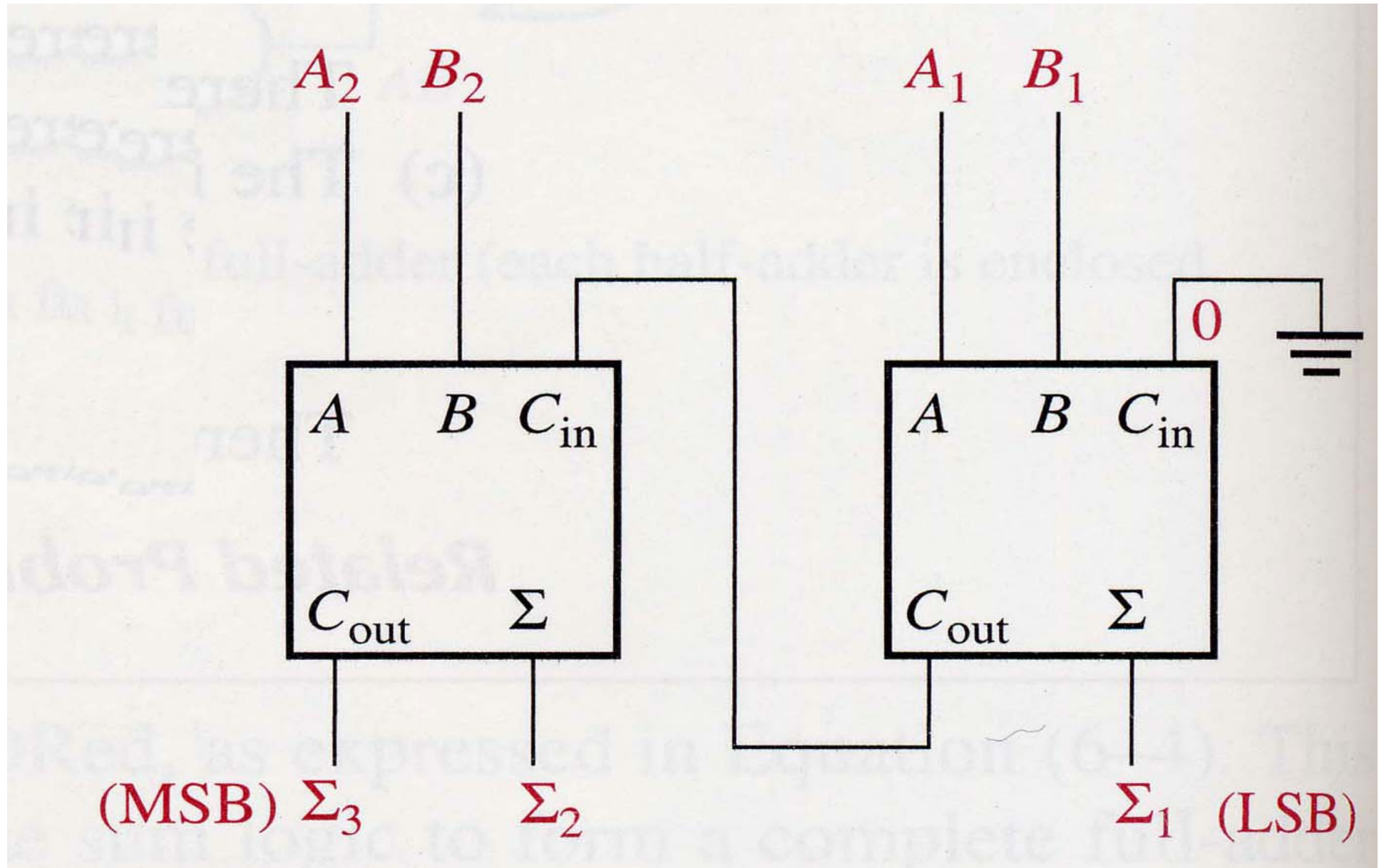
Carry bit
from right
column

$$\begin{array}{r} 0 \\ 11 \\ +01 \\ \hline 100 \end{array}$$

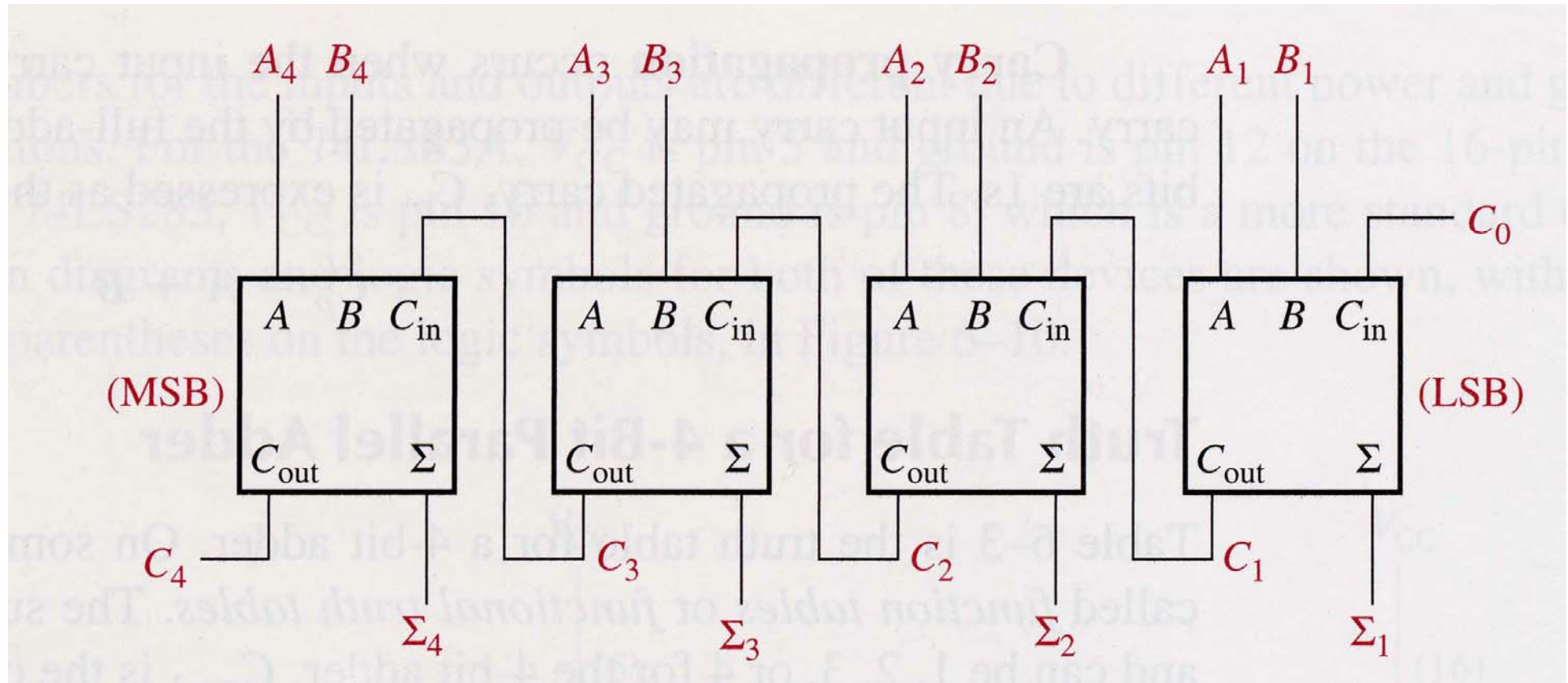
Carry bit to
the next
column

The diagram illustrates the addition of two 2-bit binary numbers, 11 and 01. A horizontal line separates the addends from the sum. The sum is 100. A blue arrow points from the right column (the 1s place) to the left column (the 2s place), indicating a carry bit. The carry bit is labeled '0' in blue. The sum '100' is also in blue, with the '1' in the 2s place being the carry bit.

2-bit parallel adder using two full-adders

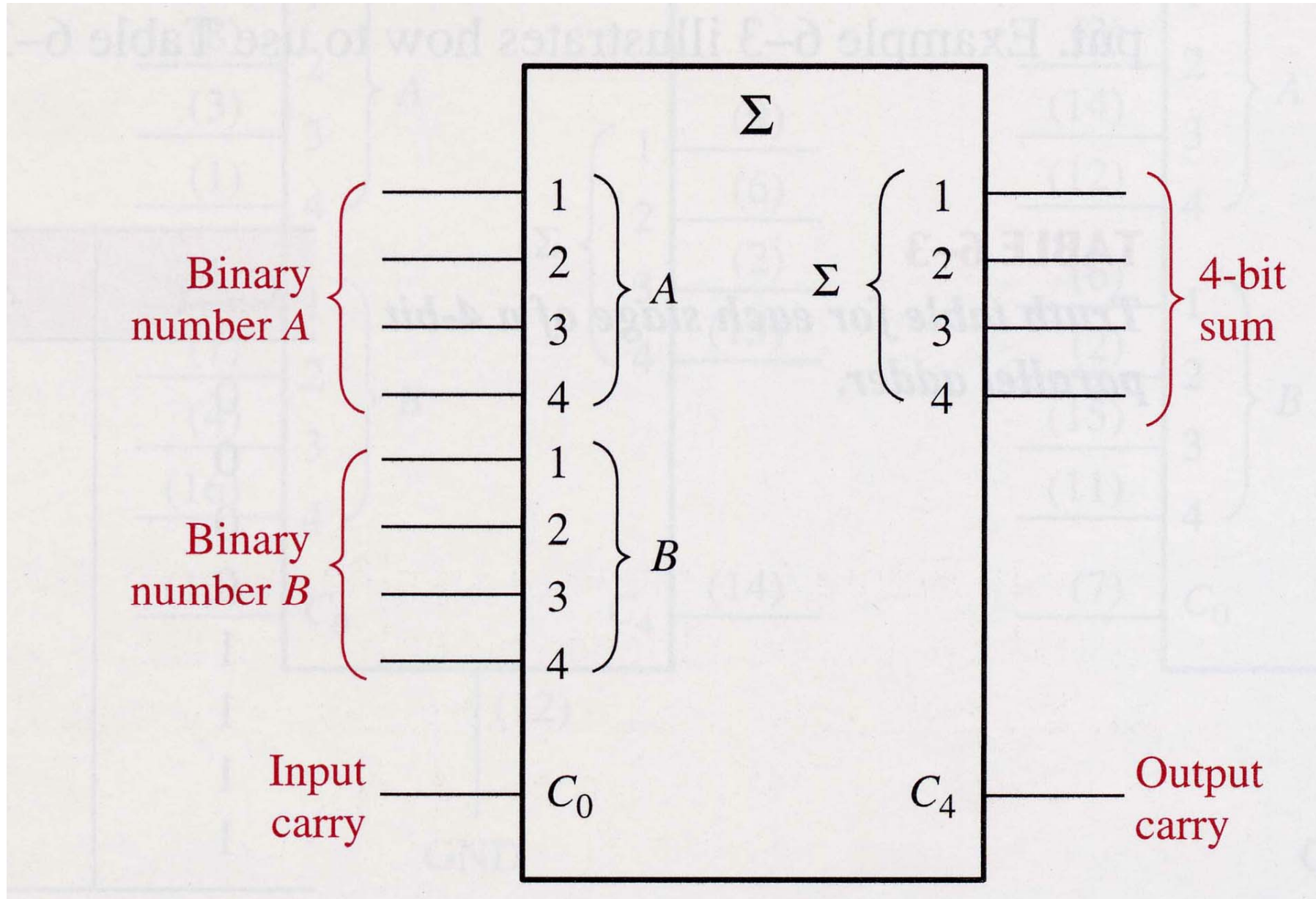


4-bit parallel adder using four full-adders



Block diagram

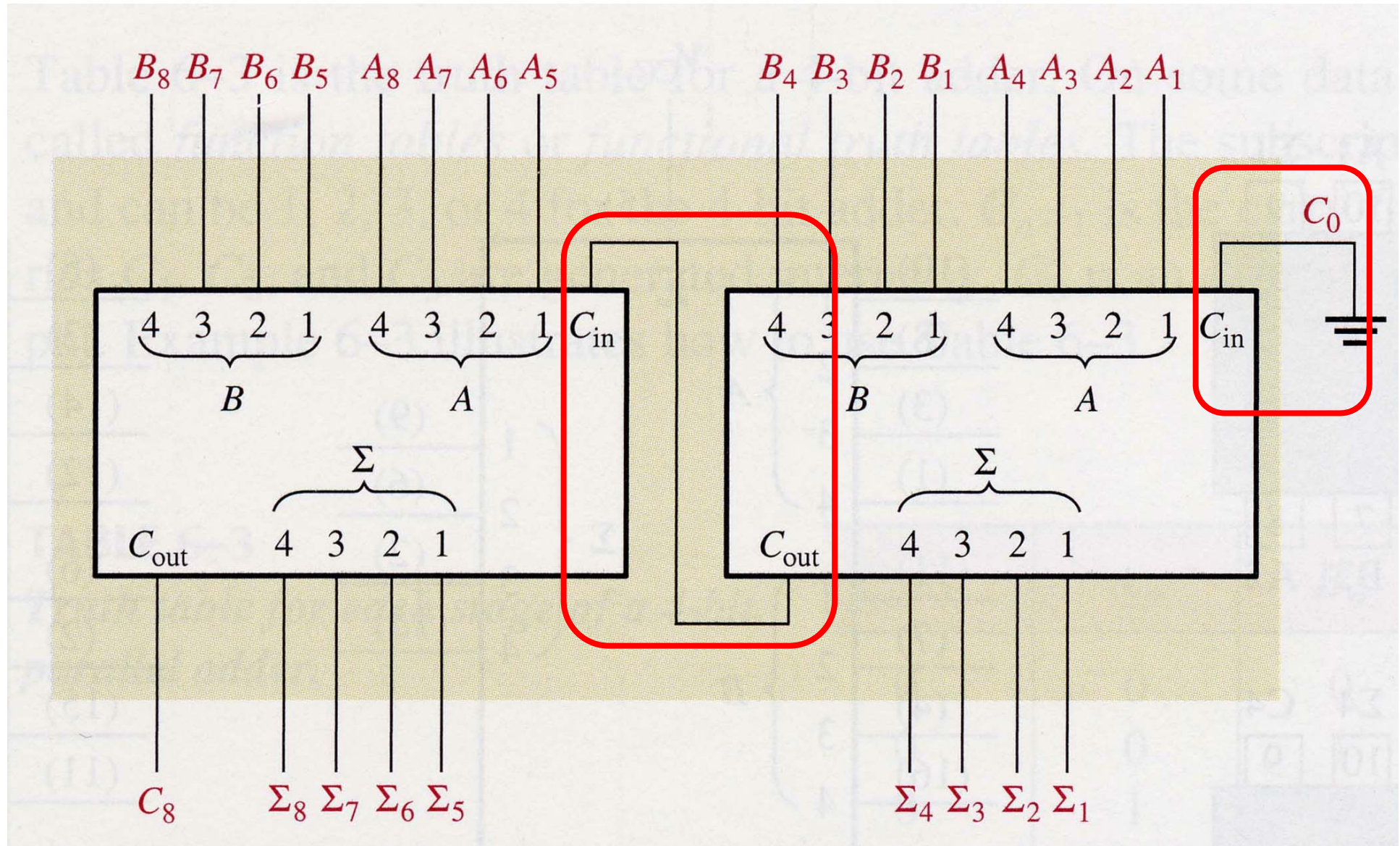
Logic symbol



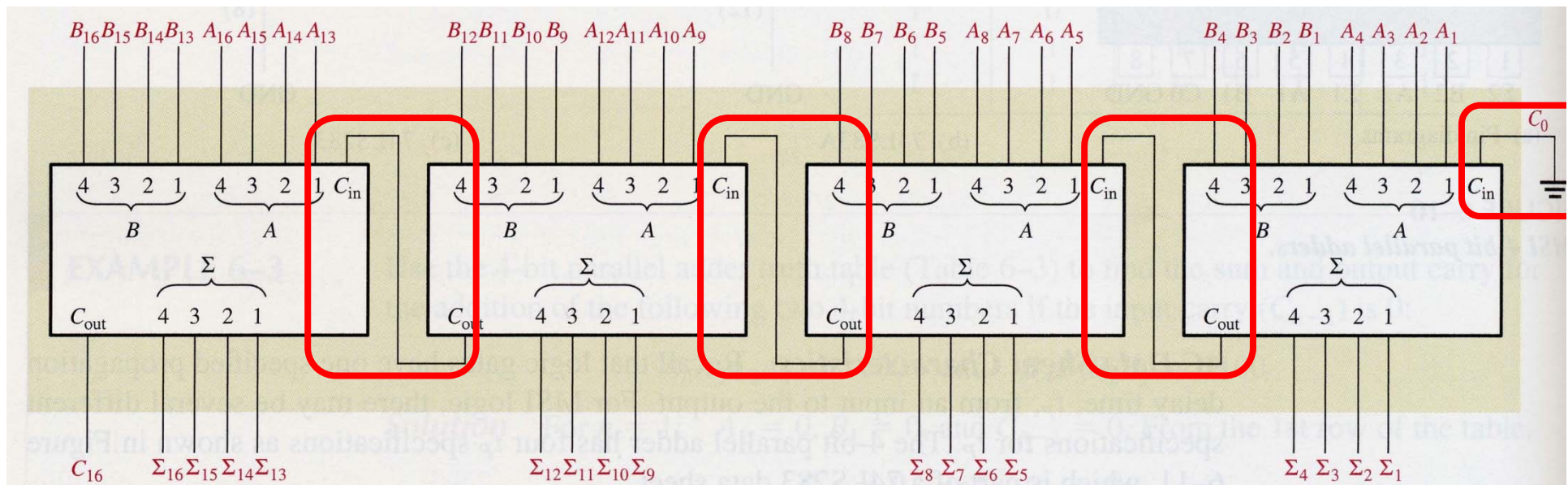
Adder Expansion

- 8-bit adder
- 16-bit adder

8-bit adder



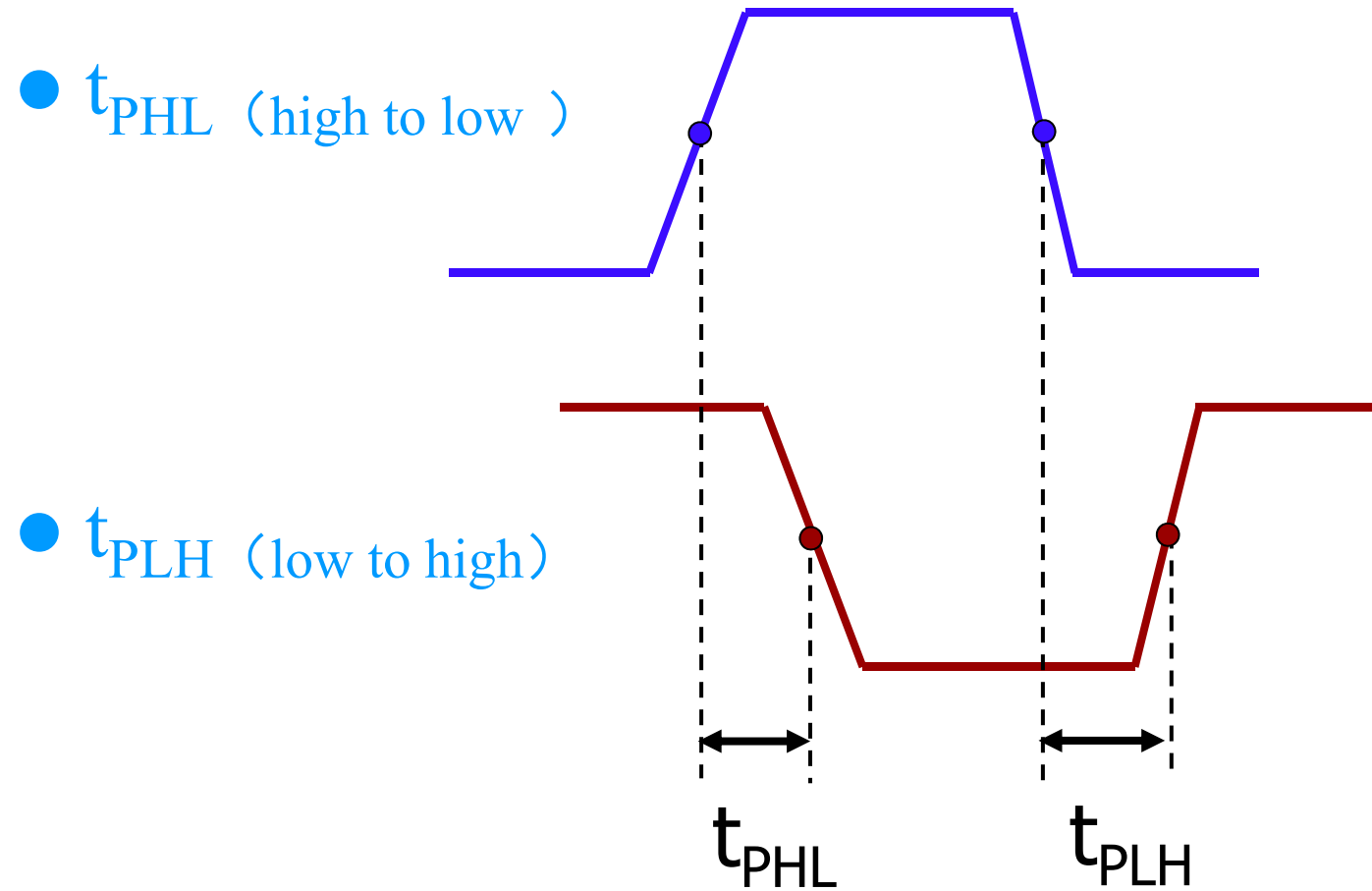
16-bit adder



Performance Characteristics and Parameters (性能特性与参数)

- *Definition of Propagation Delay Time (传播时延) t_p :* the time interval between a specified reference on the input pulse and a corresponding reference point on the resulting output pulse. (输入脉冲的某一参考电平到输出脉冲的对应参考电平之间的时间间隔)
- The term low speed and high speed (低速与高速), applied to logic circuits refer to the propagation delay time. The shorter the propagation delay and the higher the frequency at which it can operate. (一个电路的高速与低速的区别通常是指该电路的传播时延参数, 传播时延越小, 电路速度越快)

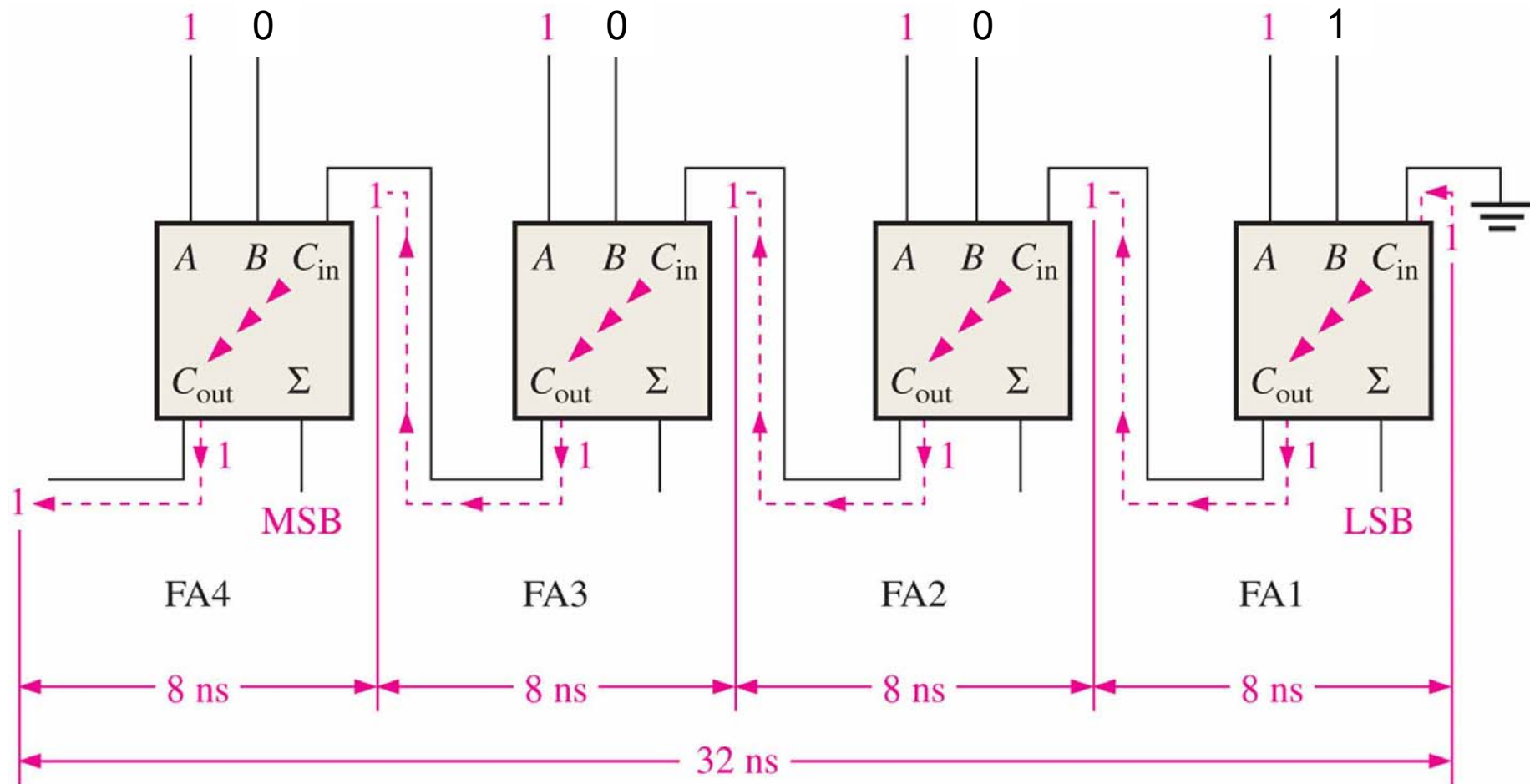
Measurement of t_p



Comparison of t_p

	Series	Typical delay time
1	TTL standard-series (TTL标准系列)	11ns
2	TTL F-series (TTL高速系列)	3.3ns
3	CMOS HCT-series (CMOS高速系列)	7ns
4	CMOS AC-series (先进CMOS电路系列)	5ns
5	CMOS ALVC-series	3ns
6	ECL (Emitter Coupled Logic) 射极耦合逻辑 (工作在非饱和态)	0.22ns

The disadvantage of Ripple Carry Adder (级联进位加法器)



Each of the six full-adders in a 6-bit parallel ripple carry adder exhibits the following propagation delay:

$$A \text{ to } \sum \text{ and } C_{out}: 30ns$$

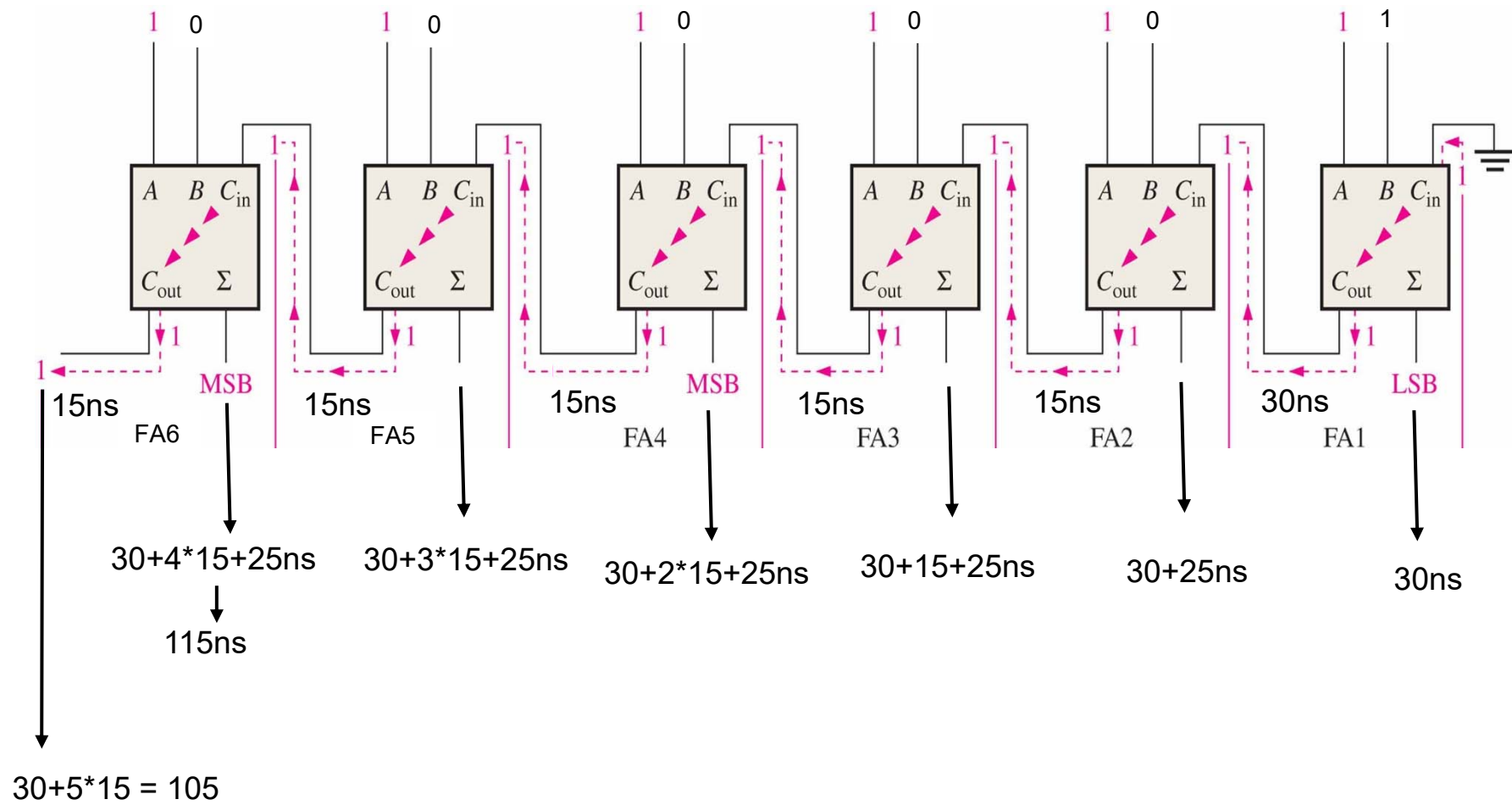
$$B \text{ to } \sum \text{ and } C_{out}: 30ns$$

$$C_{in} \text{ to } \sum : 25ns$$

$$C_{in} \text{ to } C_{out}: 15ns$$

The maximum total time for the addition of two 6-bit numbers is ().

- (A) 105ns (B) 115ns (C) 130ns (D) 180ns



Look-ahead carry addition

(超前进位加法器)

- The *look-ahead carry adder* anticipates the output carry based on the input bits, and produces the output carry by either *carry generation* or *carry propagation*.

(能够提前预测进位的加法器)

The Look-Ahead Carry Adder (I)

(进位加法器)

- **Carry generation** (进位产生项) : occurs when an output carry is produced internally by the full adder. That is both input bits are 1s.
- 进位项是由该全加器自身的输入产生，也就是全加器的两个输入都是1的情况

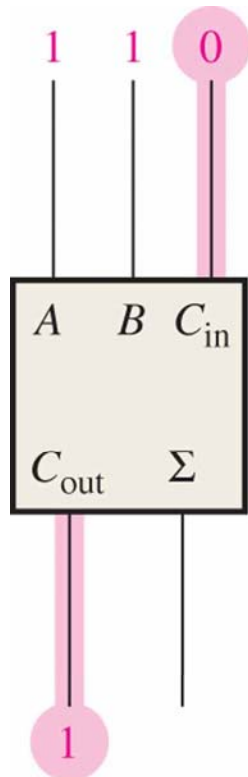
$$C_g \text{ (or } g \text{)} = AB$$

The Look-Ahead Carry Adder (I)

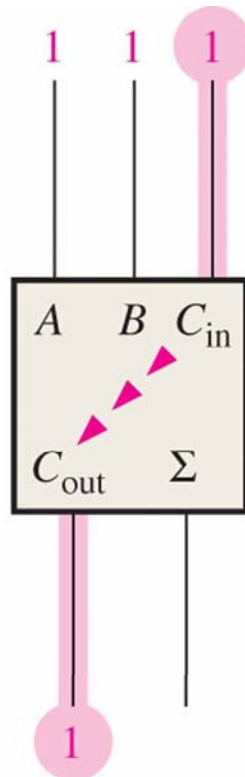
(进位加法器)

- **Carry propagation** (进位传递项) : occurs when the input carry is rippled to become the output carry. An input carry may be propagated by the full-adder when either or both of the input bits are 1s.
- 进位传递项表示该全加器的进位输出是其进位输入传递产生。该情况发生是当输入的两个相加项至少一个为1。

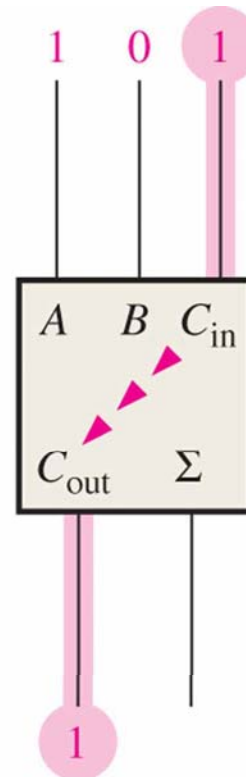
$$C_p \text{ (or } p \text{)} = A + B$$



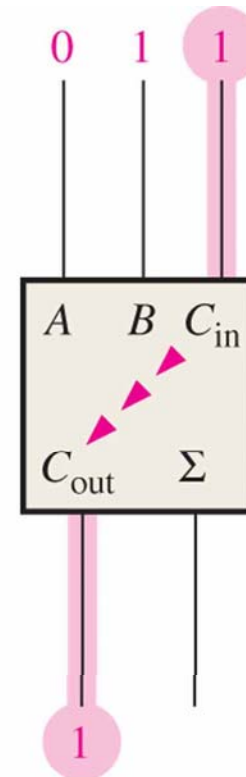
Generated
carry



Propagated carry/
Generated carry



Propagated
carry

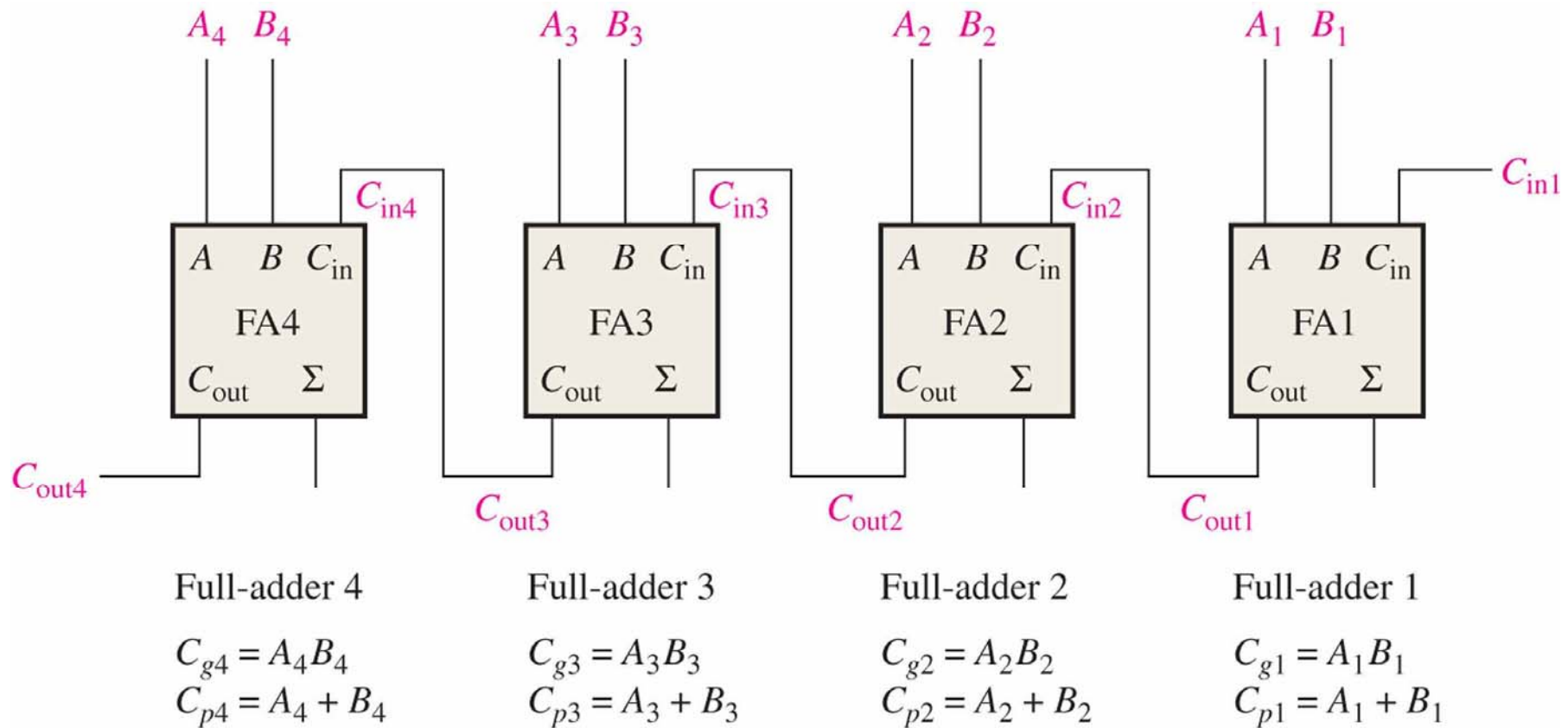


Propagated
carry

$$C_{out} = C_g + C_p C_{in}$$

$$(C_{out} = g + pC_{in})$$

The Look-Ahead Carry Adder (II)



The Look-Ahead Carry Adder (II)

- **Full-adder 1:** $C_{out} = C_g + C_p C_{in}$

- **Full-adder 2:** $C_{in2} = C_{out1}$

$$\begin{aligned}C_{out2} &= C_{g2} + C_{p2} C_{in2} = C_{g2} + C_{p2} C_{out1} \\&= C_{g2} + C_{p2} (C_{g1} + C_{p1} C_{in1}) \\&= C_{g2} + C_{p2} C_{g1} + C_{p2} C_{p1} C_{in1}\end{aligned}$$

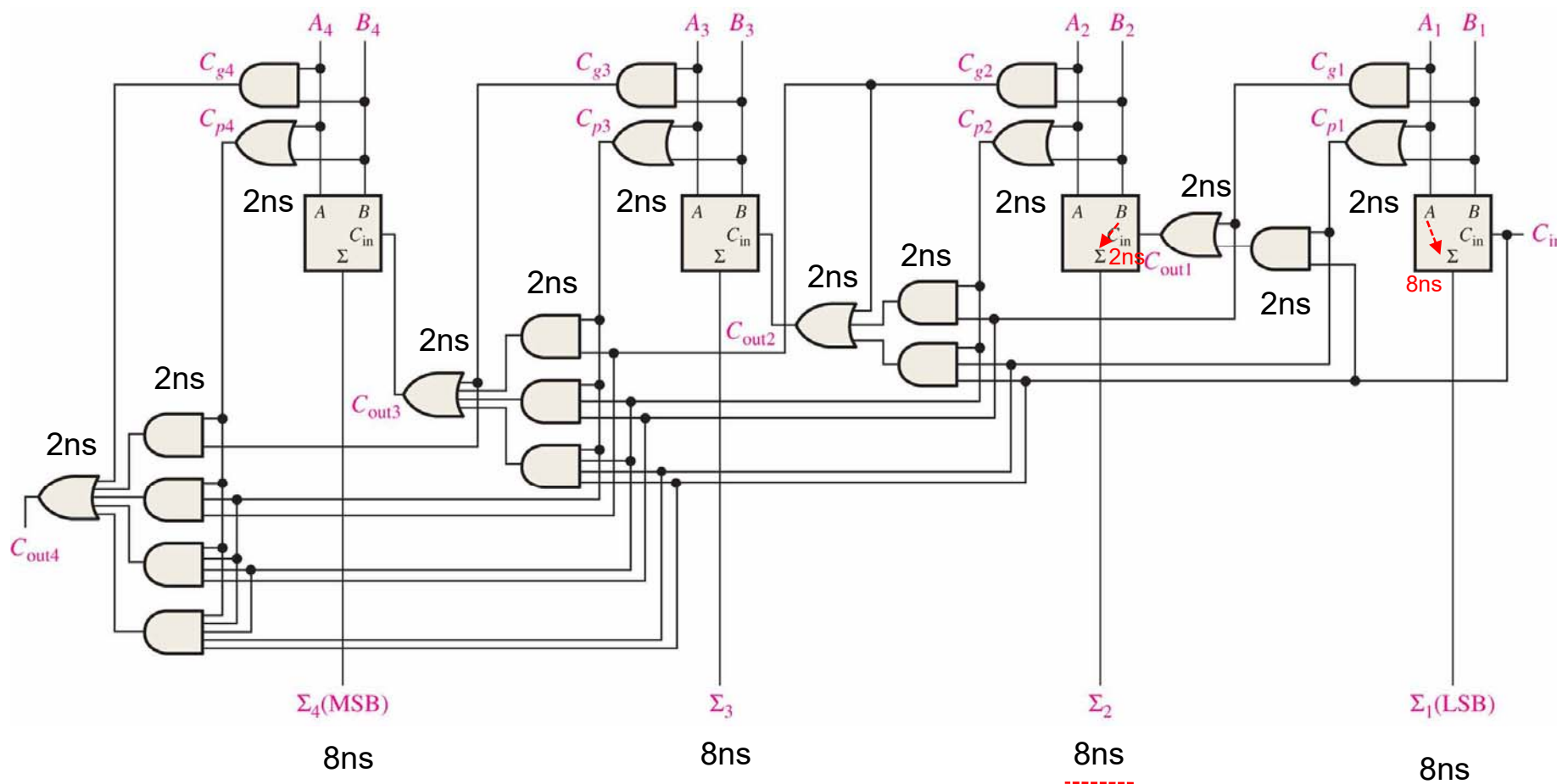
- **Full-adder 3:** $C_{in3} = C_{out2}$

$$\begin{aligned}C_{out3} &= C_{g3} + C_{p3} C_{in3} = C_{g3} + C_{p3} C_{out2} \\&= C_{g3} + C_{p3} C_{g2} + C_{p3} C_{p2} C_{g1} + C_{p3} C_{p2} C_{p1} C_{in1}\end{aligned}$$

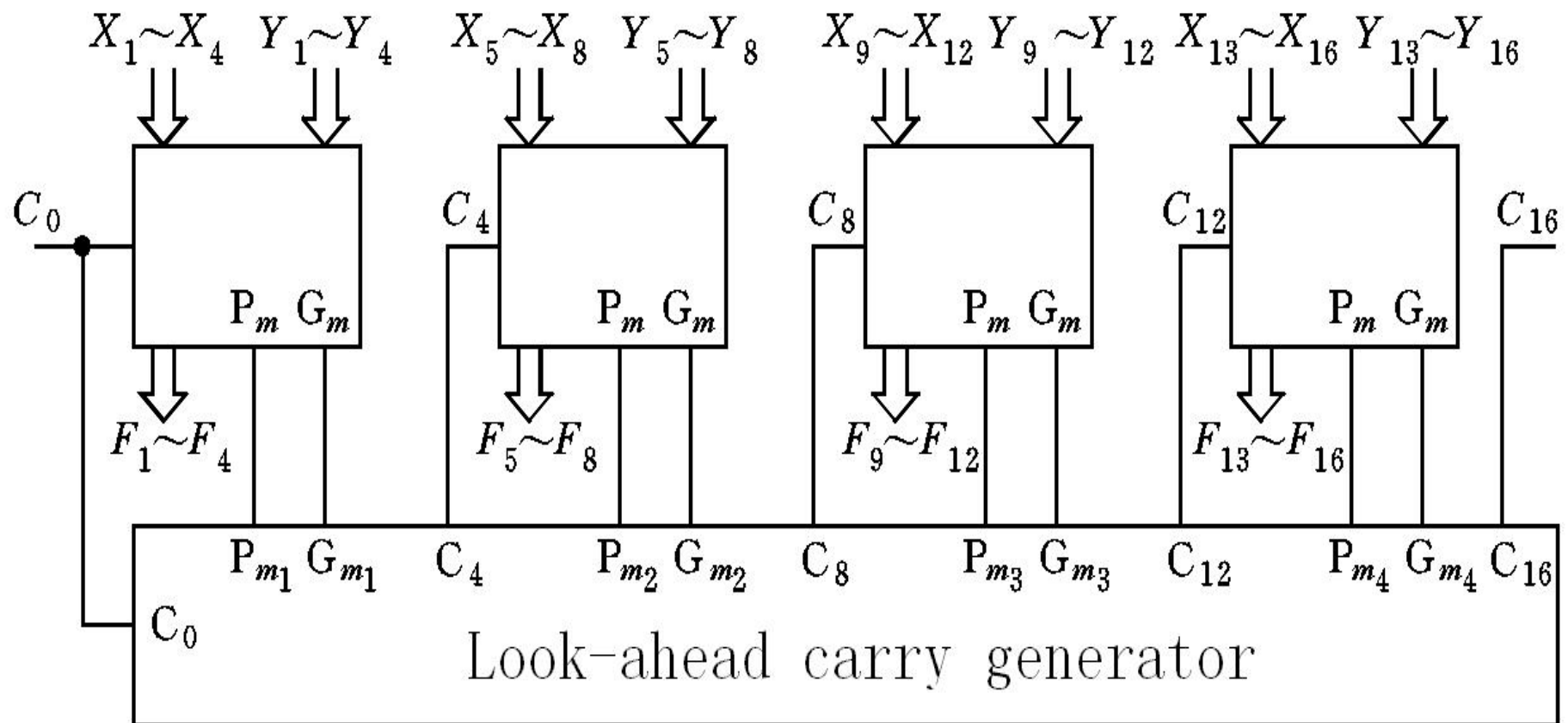
The Look-Ahead Carry Adder (II)

- **Full-adder 4:** $C_{in4} = C_{out3}$

$$\begin{aligned} C_{out4} &= C_{g4} + C_{p4}C_{in4} = C_{g4} + C_{p4}C_{out3} \\ &= C_{g4} + C_{p4}C_{g3} + C_{p4}C_{p3}C_{g2} + C_{p4}C_{p3}C_{p2}C_{g1} + C_{p4}C_{p3}C_{p2}C_{p1}C_{in1} \end{aligned}$$



16-bit parallel adder



16-bit Look-Ahead Carry Adder (III)

- **Full-adder 4:** $C_{in4} = C_{out3}$

$$\begin{aligned} C_{out4} &= C_{g4} + C_{p4}C_{in4} = C_{g4} + C_{p4}C_{out3} \\ &= \boxed{C_{g4} + C_{p4}C_{g3} + C_{p4}C_{p3}C_{g2} + C_{p4}C_{p3}C_{p2}C_{g1}} + \boxed{C_{p4}C_{p3}C_{p2}C_{p1}}C_{in1} \end{aligned}$$



G



P



$$C_{out4} = G + PC_{in1}$$

16-bit Look-Ahead Carry Adder (III)

- Full-adder m1:

$$C_{m1} = G_{m1} + P_{m1}C_{in0}$$

- Full-adder m2:

$$C_{m2} = G_{m2} + P_{m2}C_{m2in} = G_{m2} + P_{m2}G_{m1} + P_{m2}P_{m1}C_{in0}$$

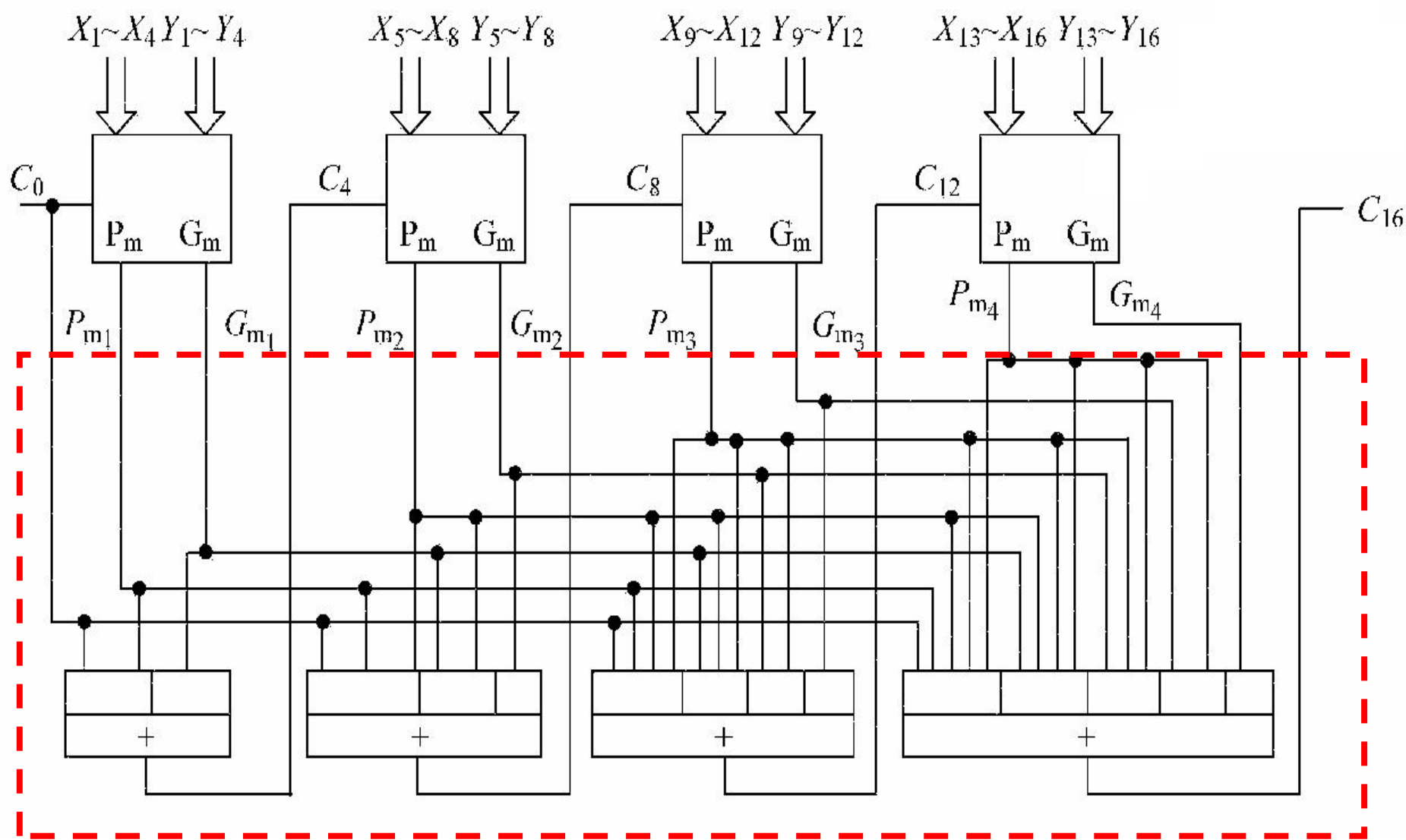
- Full-adder m3:

$$\begin{aligned} C_{m3} &= G_{m3} + P_{m3}C_{m3in} \\ &= G_{m3} + P_{m3}G_{m2} + P_{m3}P_{m2}G_{m1} + P_{m3}P_{m2}P_{m1}C_{in0} \end{aligned}$$

16-bit Look-Ahead Carry Adder (III)

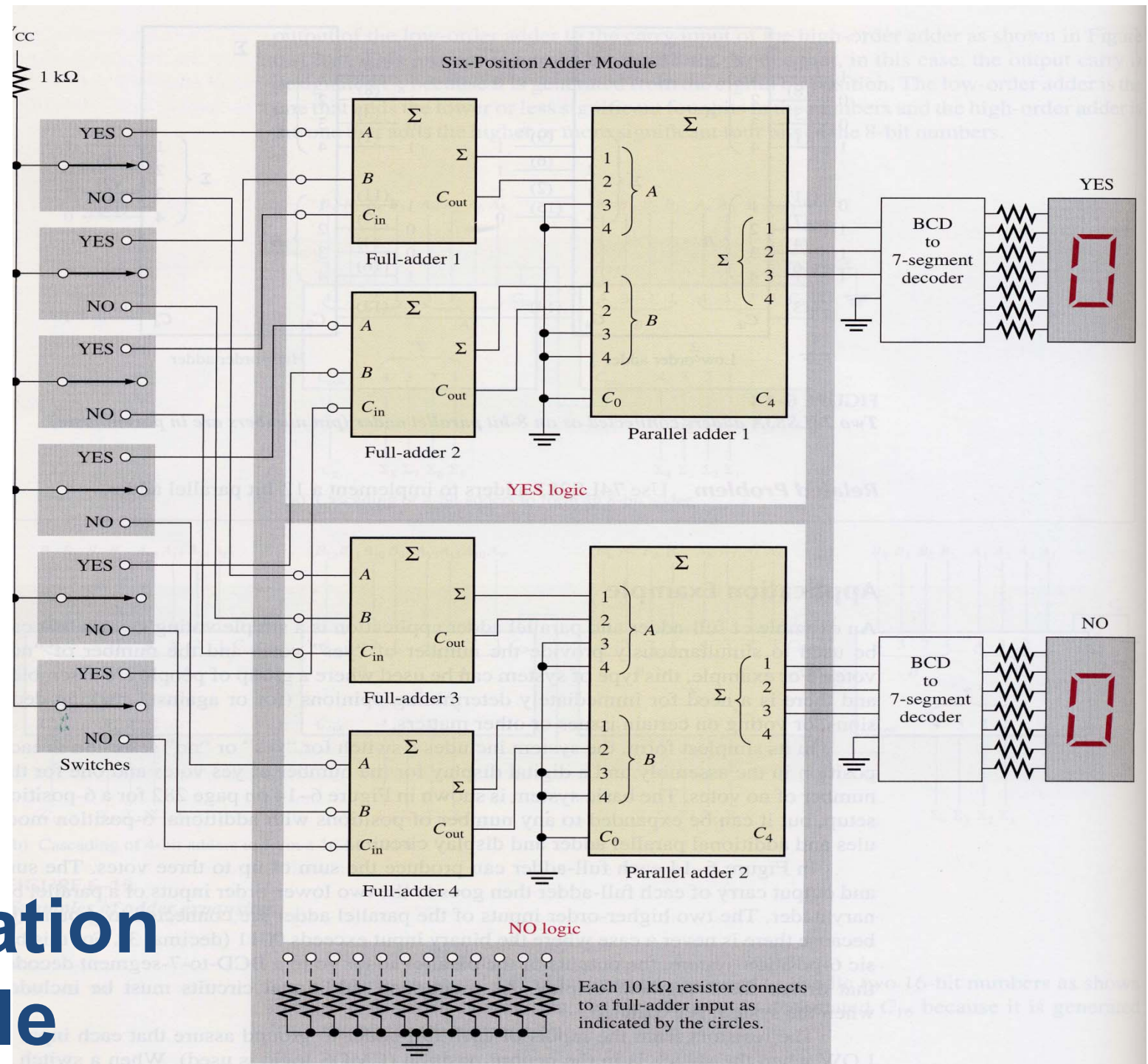
- Full-adder m4:

$$\begin{aligned}C_{m4} &= G_{m4} + P_{m4}C_{m4in} \\&= G_{m4} + P_{m4}G_{m3} + P_{m4}P_{m3}G_{m2} + P_{m4}P_{m3}P_{m2}G_{m1} + P_{m4}P_{m3}P_{m2}P_{m1}C_{in0}\end{aligned}$$



A voting system

Application Example

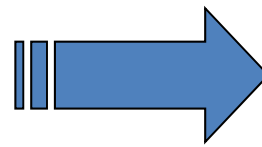
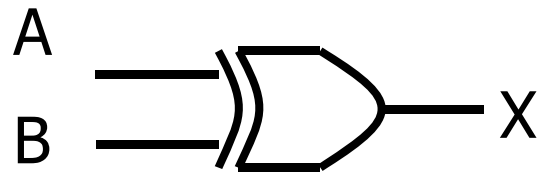


2 Comparator（比较器）

- Basic comparator—Exclusive-OR（基本比较器）
- Magnitude comparator（幅值比较器）
- Expansion of comparators

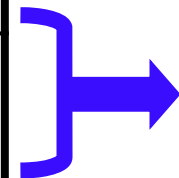
Basic comparator

XOR gate

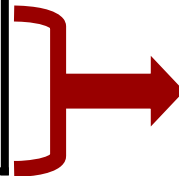


basic comparator
for 1-bit number

Inputs		Output
A	B	X
0	0	0
1	1	0
0	1	1
1	0	1



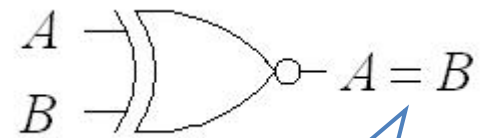
equal



Not equal

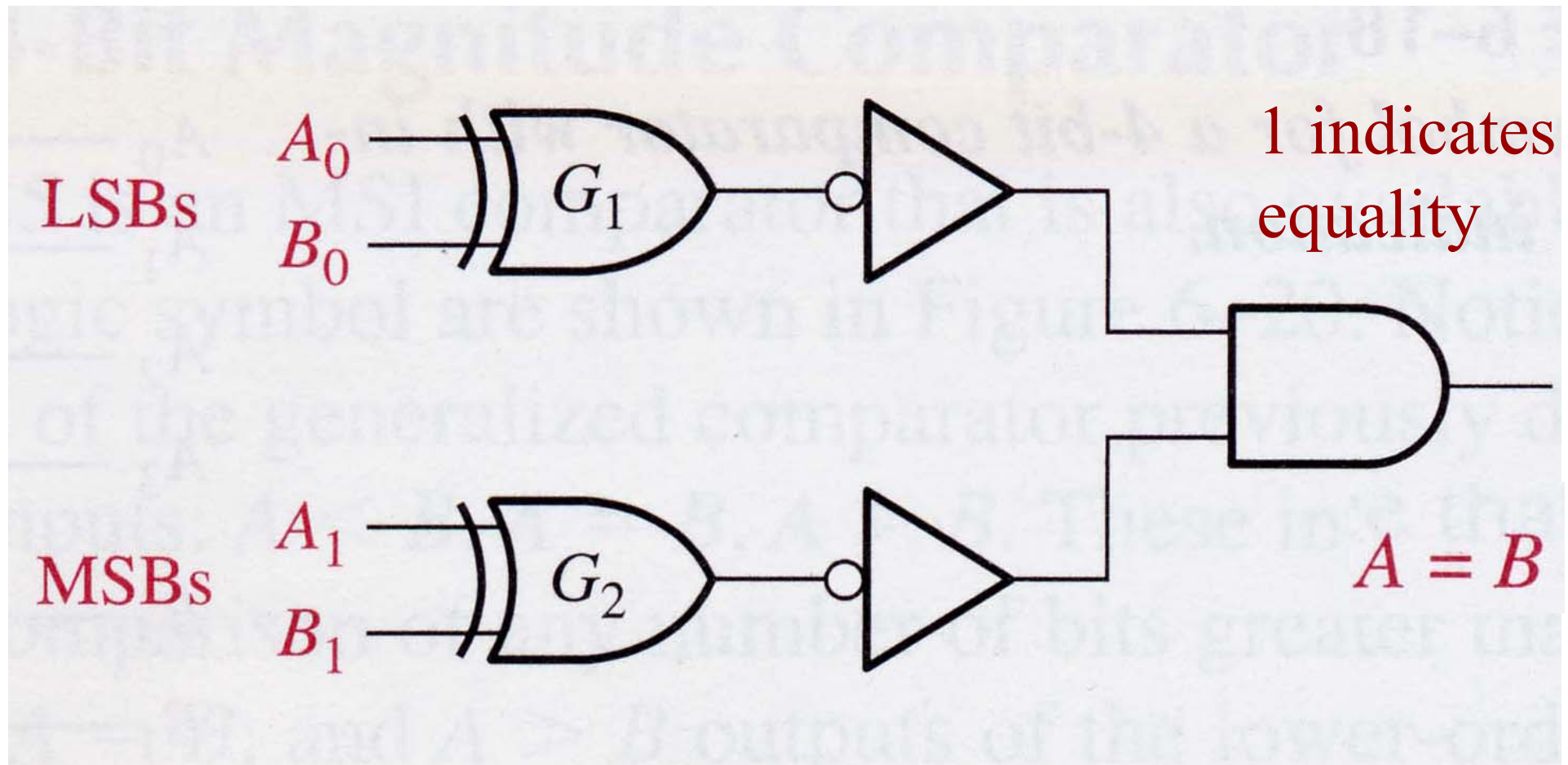


$$(A = B) = \overline{A \oplus B}$$



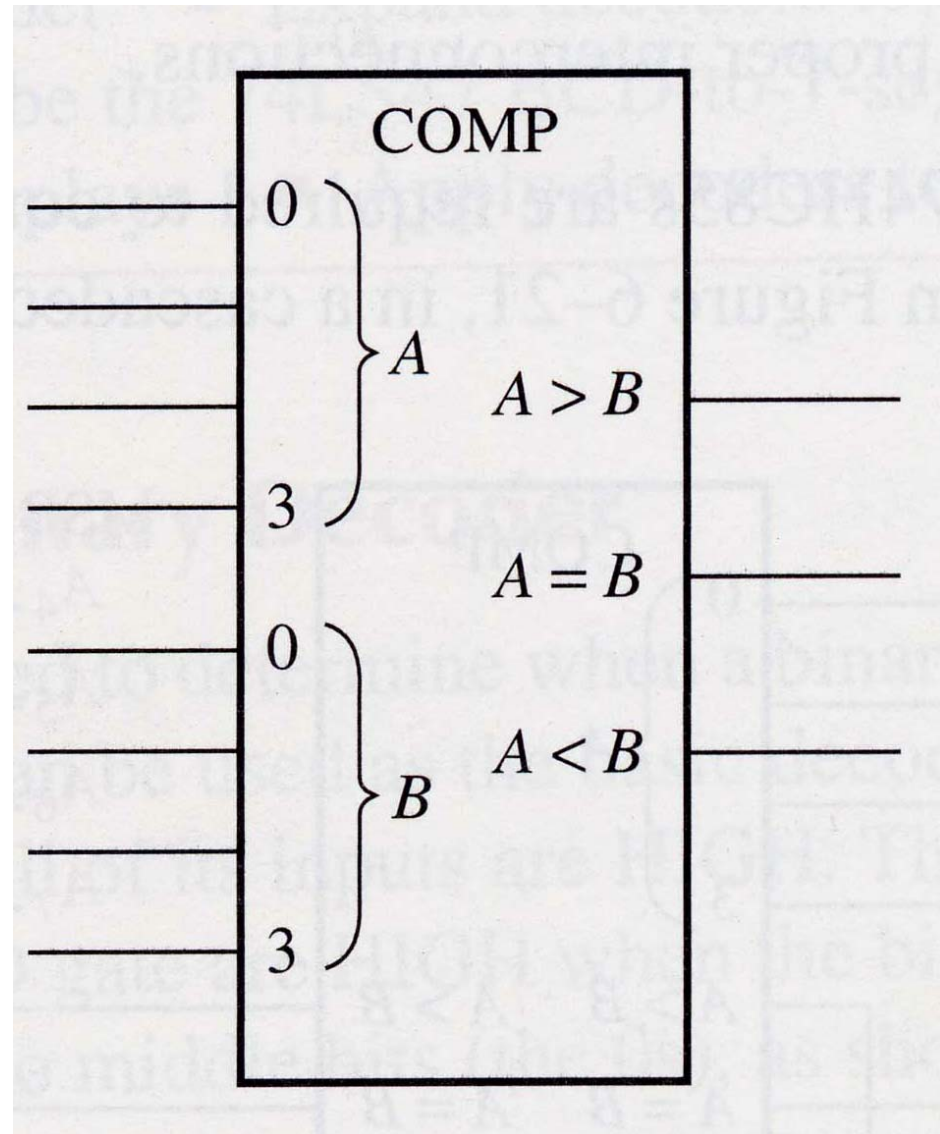
HIGH indicates equality

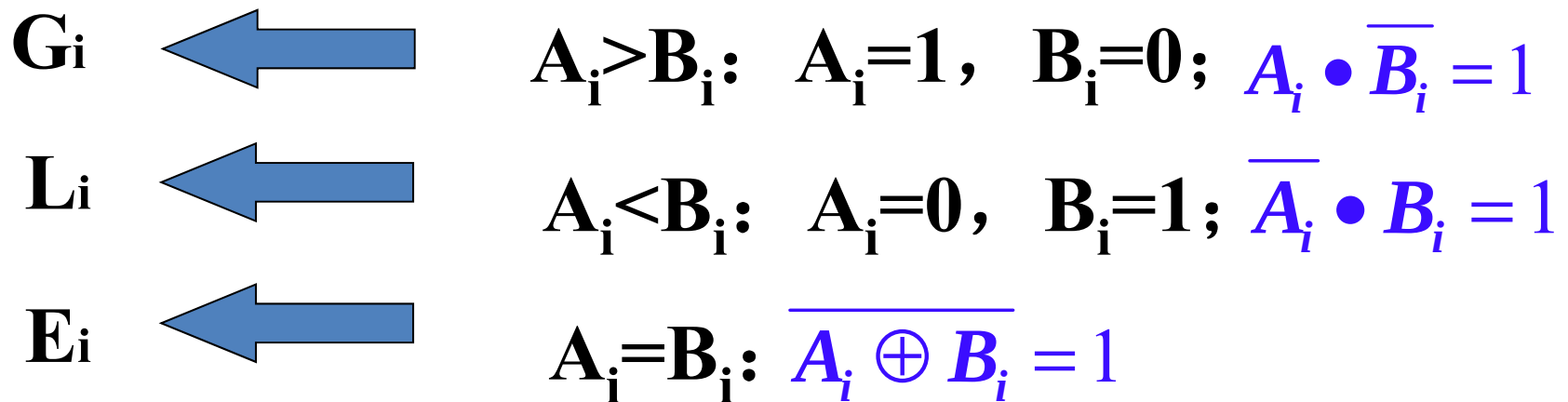
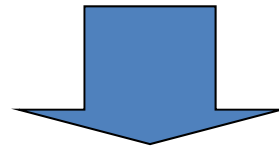
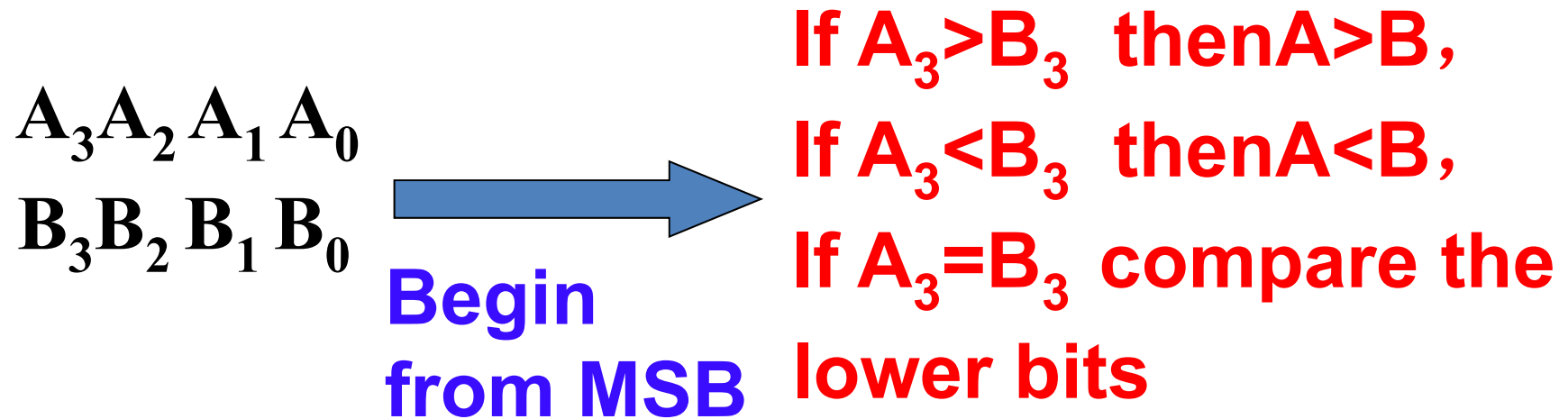
Two 2-bit Comparator



4-bit Magnitude Comparator

input	output
$A > B$	1 0 0
$A = B$	0 1 0
$A < B$	0 0 1





A_3, B_3	A_2, B_2	A_1, B_1	A_0, B_0	$A > B$	$A < B$	$A = B$
$A_3 > B_3$	$\times$	$\times$	$\times$	1	0	0
$A_3 < B_3$	$\times$	$\times$	$\times$	0	1	0
$A_3 = B_3$	$A_2 > B_2$	$\times$	$\times$	1	0	0
$A_3 = B_3$	$A_2 < B_2$	$\times$	$\times$	0	1	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 > B_1$	$\times$	1	0	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 < B_1$	$\times$	0	1	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 = B_1$	$A_0 > B_0$	1	0	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 = B_1$	$A_0 < B_0$	0	1	0
$A_3 = B_3$	$A_2 = B_2$	$A_1 = B_1$	$A_0 = B_0$	0	0	1

Example: $A < B$

G_3	L_3	E_3	G_2	L_2	E_2	G_1	L_1	E_1	G_0	L_0	E_0	$A < B$
1	0	0	X	x	x	X	x	x	x	x	X	0
0	1	0	x	x	x	X	x	x	X	x	x	1
0	0	1	1	0	0	X	x	x	x	x	x	0
0	0	1	0	1	0	x	x	x	x	x	x	1
0	0	1	0	0	1	1	0	0	x	x	x	0
0	0	1	0	0	1	0	1	0	x	x	x	1
0	0	1	0	0	1	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0	1	0	1	0	1
0	0	1	0	0	1	0	0	1	0	0	1	0

Example: $A < B$

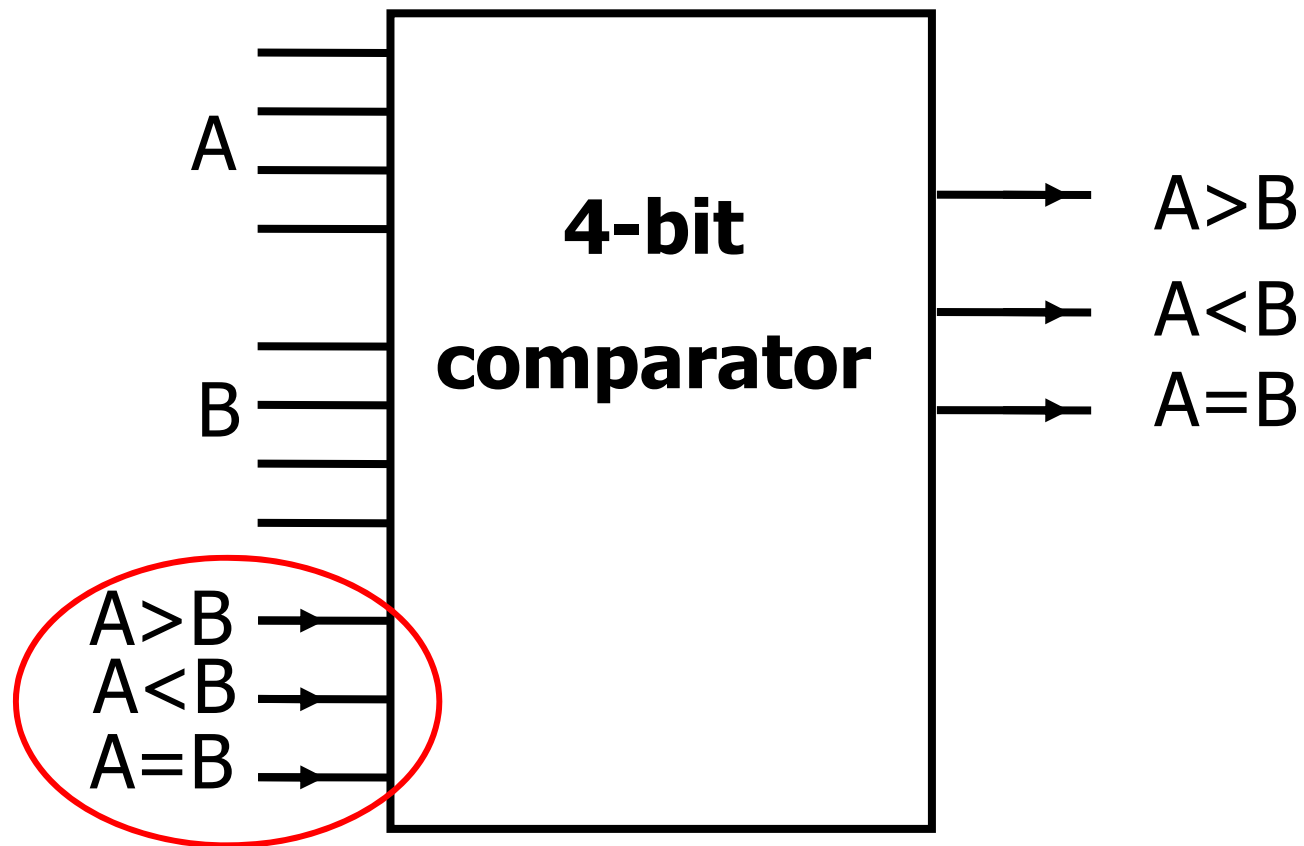
$$A < B = L_3 + E_3 L_2 + E_3 E_2 L_1 + E_3 E_2 E_1 L_0$$

where:

$$E_i = \overline{A_i \oplus B_i} = A_i \odot B_i$$

$$L_i = \overline{A_i} \bullet B_i$$

4-bit Magnitude Comparator



级联进位信号

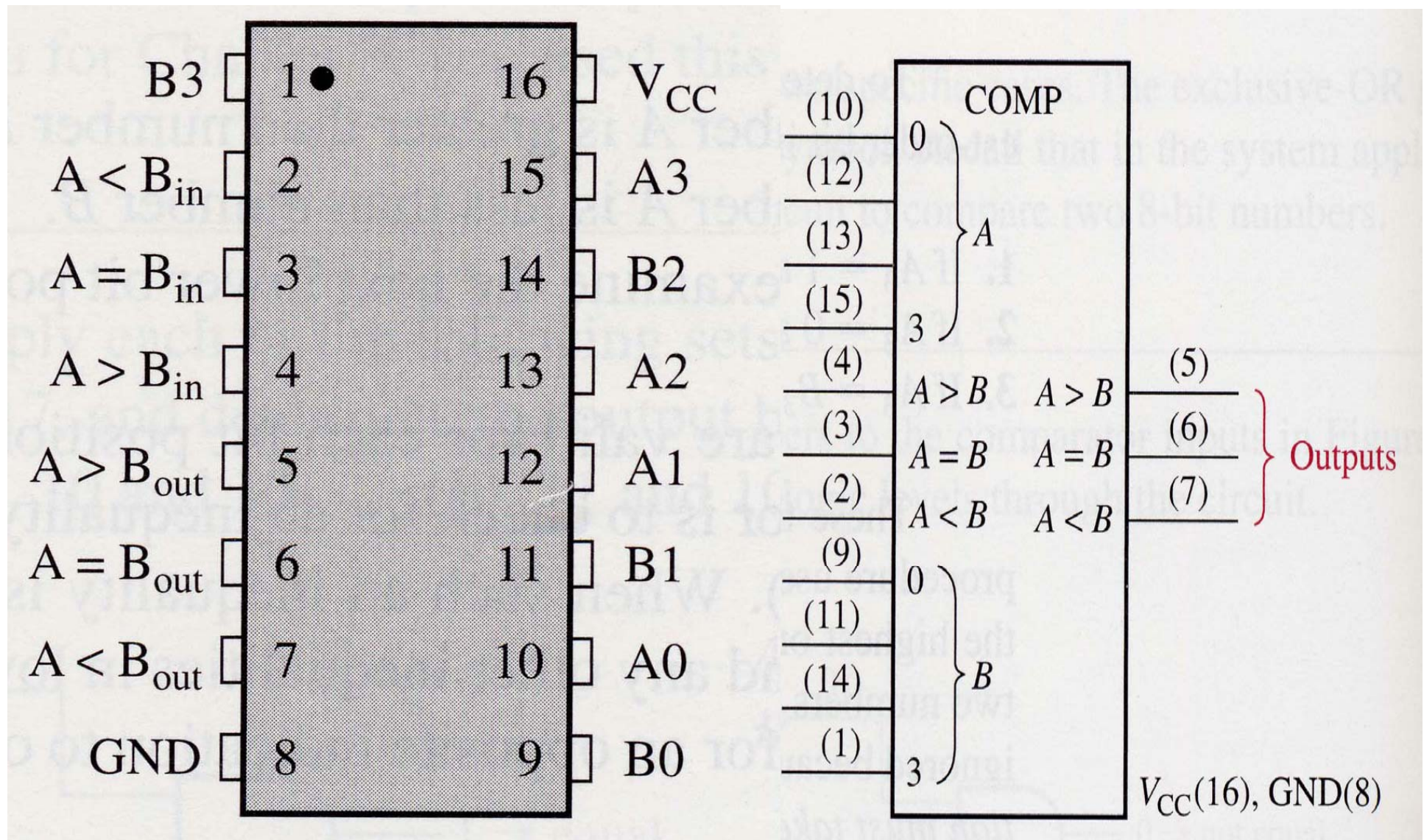
Logic functions of outputs

$$\begin{aligned} Y_{(A < B)} = & \bar{A}_3 B_3 + (A_3 \odot B_3) \bar{A}_2 B_2 + (A_3 \odot B_3) (A_2 \odot B_2) \bar{A}_1 B_1 \\ & + (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) \bar{A}_0 B_0 \\ & + (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) (A_0 \odot B_0) I_{(A < B)} \end{aligned}$$

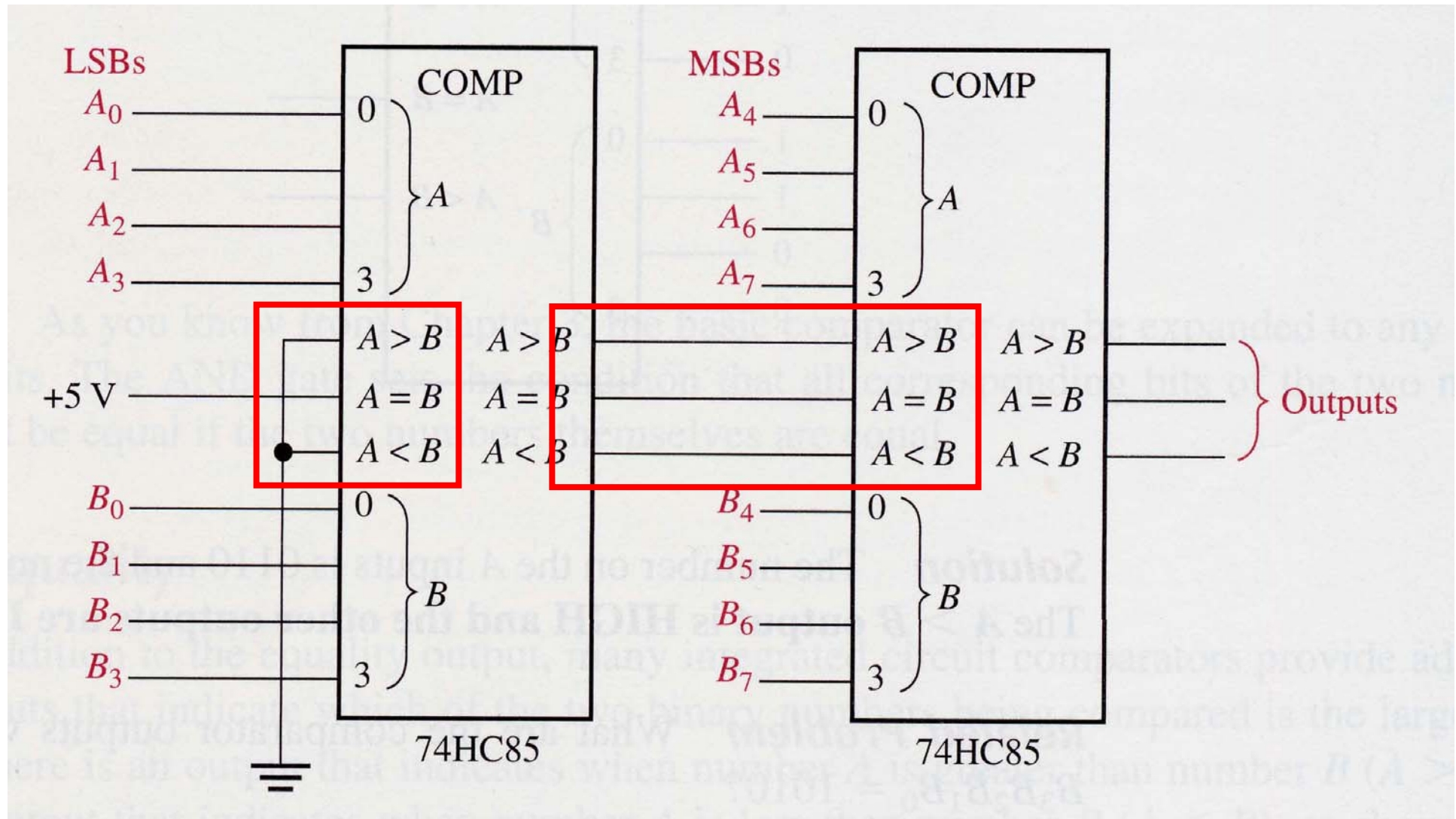
$$Y_{(A = B)} = (A_3 \odot B_3) (A_2 \odot B_2) (A_1 \odot B_1) (A_0 \odot B_0) I_{(A = B)}$$

$$Y_{(A > B)} = \overline{Y_{(A < B)} + Y_{(A = B)}}$$

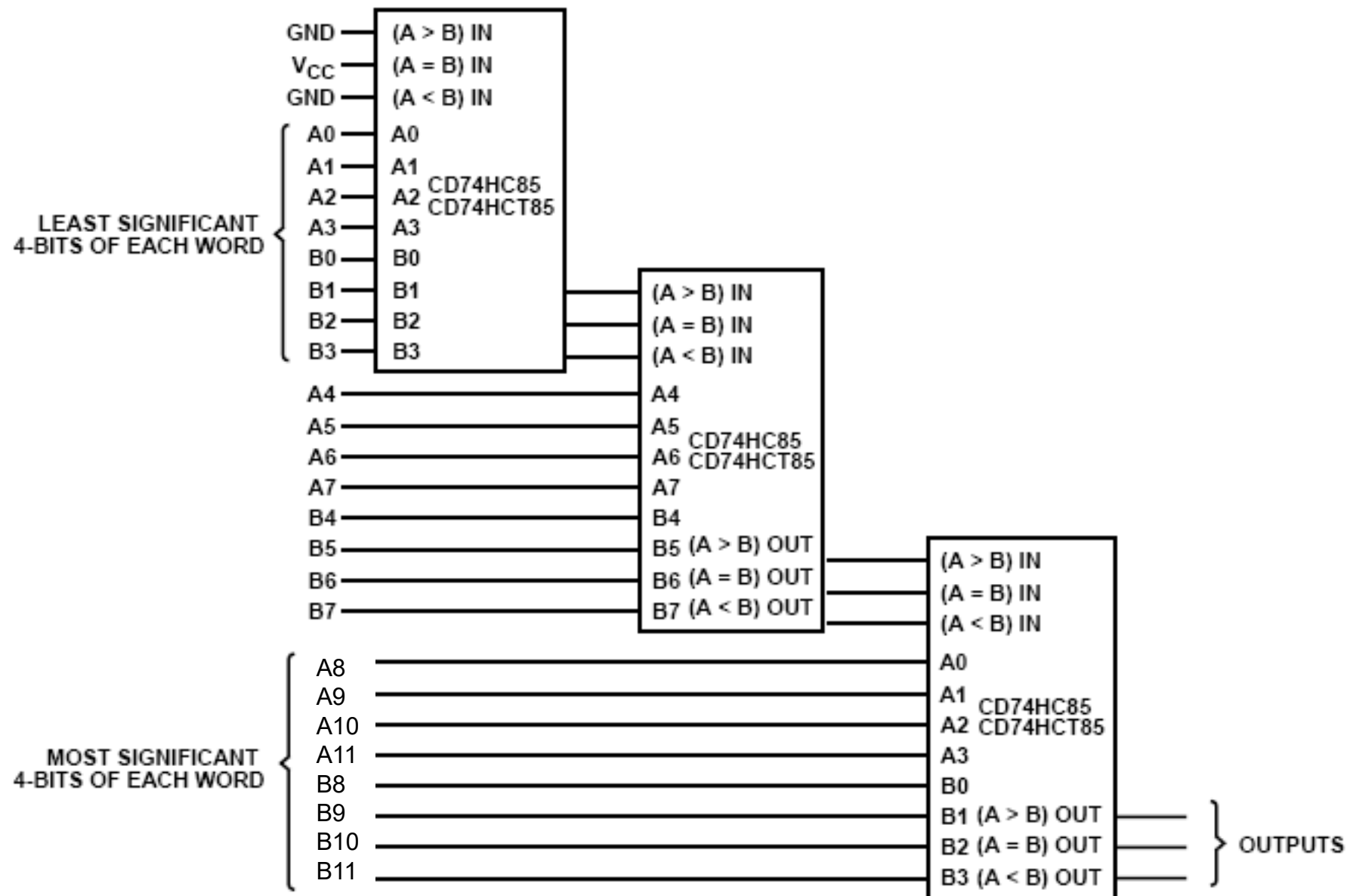
MSI 4-bit Magnitude Comparator– 74HC85/74LS85



Expansion: 8-bit Comparator



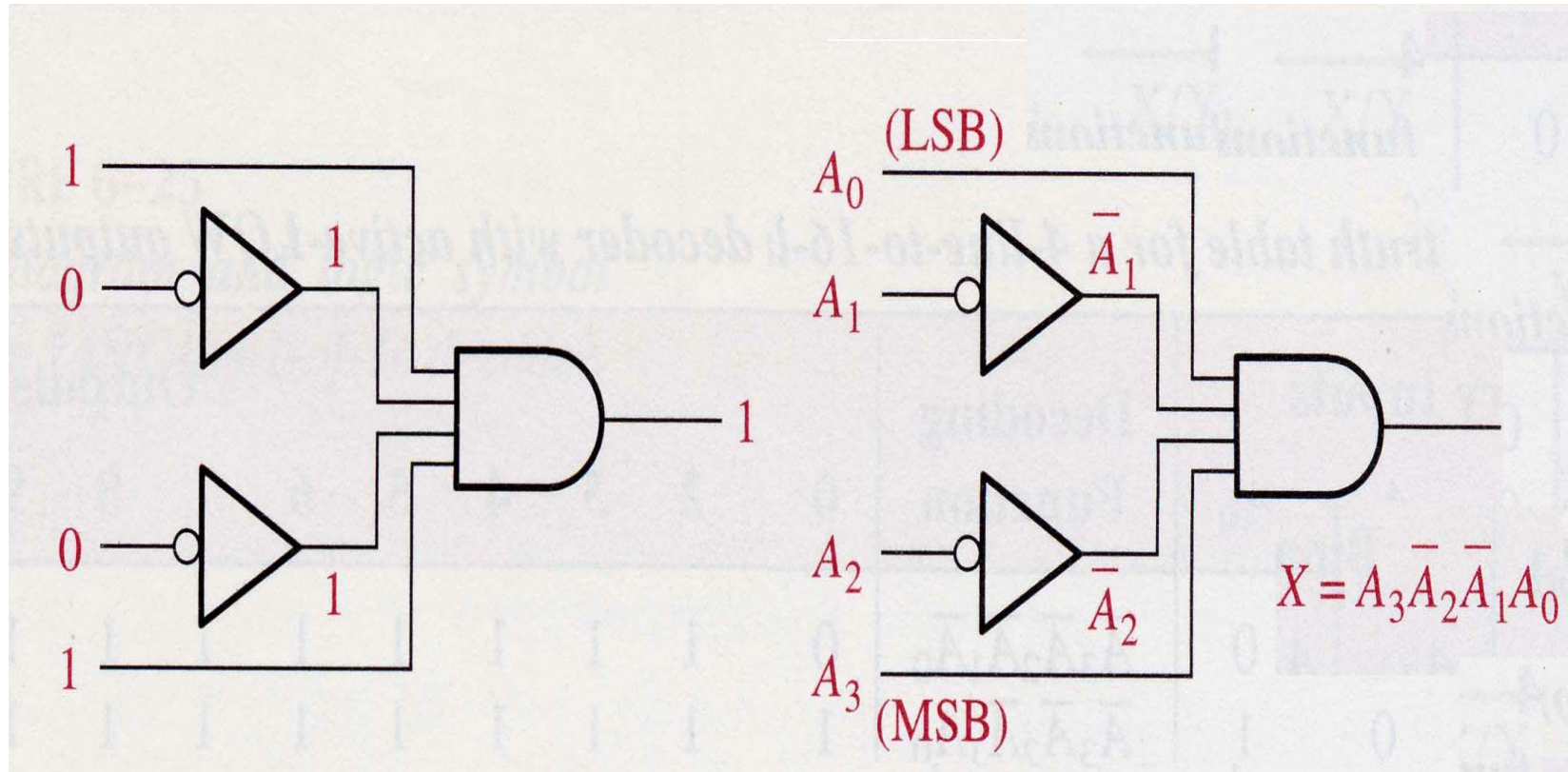
Series Cascading - Comparing 12-Bit Words



3 Decoders (解码器)

- The basic function of a decoder is to detect the presence of a specified code on its input and to indicate the presence of that code by a specified output level.

Basic Decoder

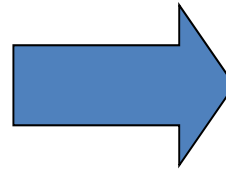


Decoding logic for the binary code 1001 with an active-HIGH output

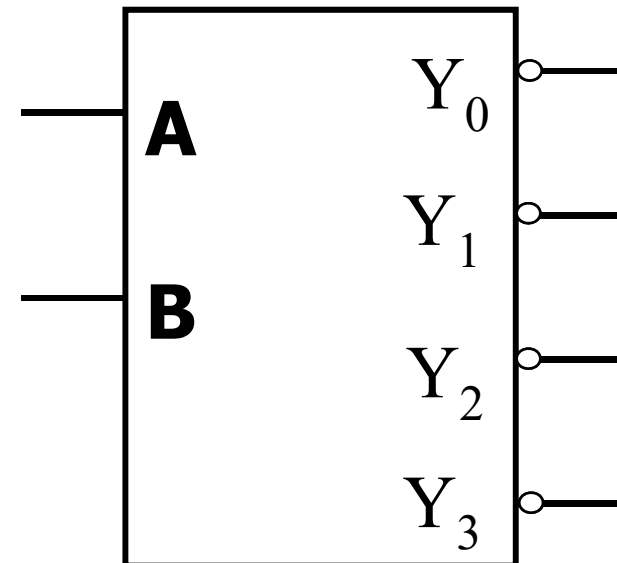
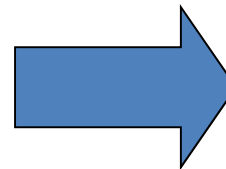
2-line-to-4-line (1-of-4) decoder

Function table

A	B	Y_0	Y_1	Y_2	Y_3
0	0	0	1	1	1
1	0	1	0	1	1
0	1	1	1	0	1
1	1	1	1	1	0



$$\begin{cases} Y_0 = \overline{\overline{A}}\overline{\overline{B}} \\ Y_1 = \overline{A}\overline{\overline{B}} \\ Y_2 = \overline{\overline{A}}\overline{B} \\ Y_3 = \overline{A}B \end{cases}$$

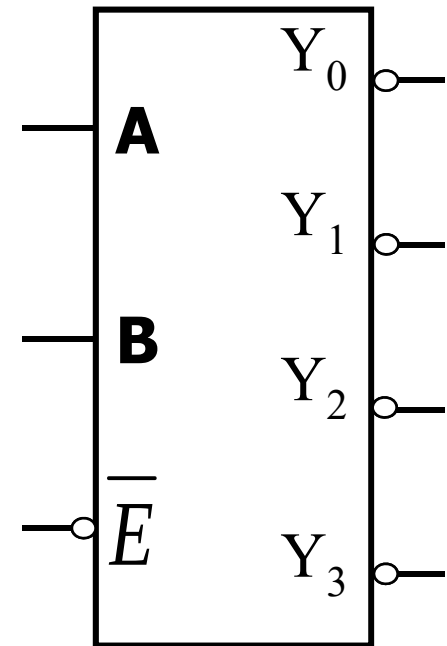


Active-Low. NAND gate is needed!

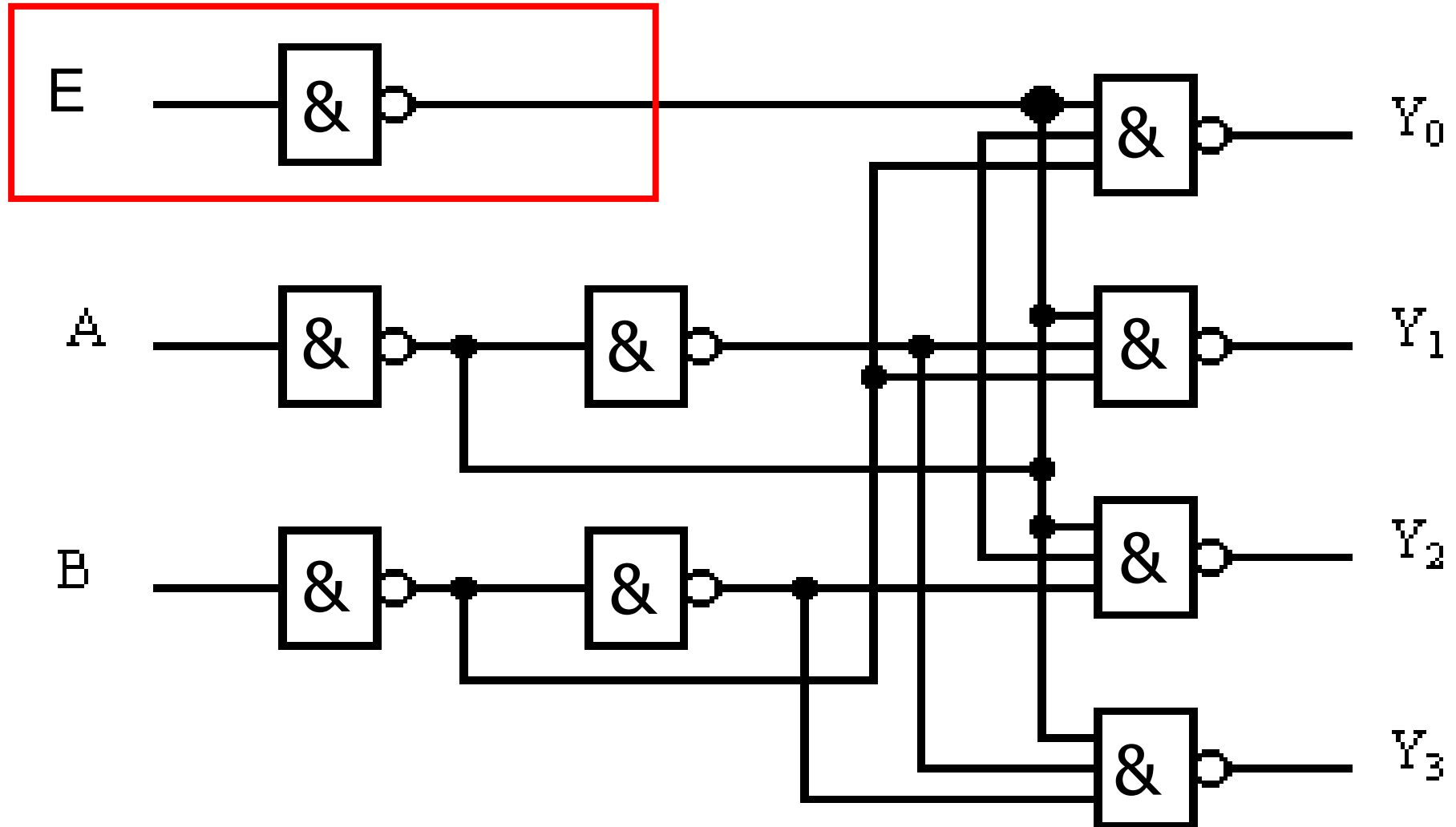
2-4 decoder with EN (使能)

E	A	B	Y_0	Y_1	Y_2	Y_3
1	×	×	1	1	1	1
0	0	0	0	1	1	1
0	1	0	1	0	1	1
0	0	1	1	1	0	1
0	1	1	1	1	1	0

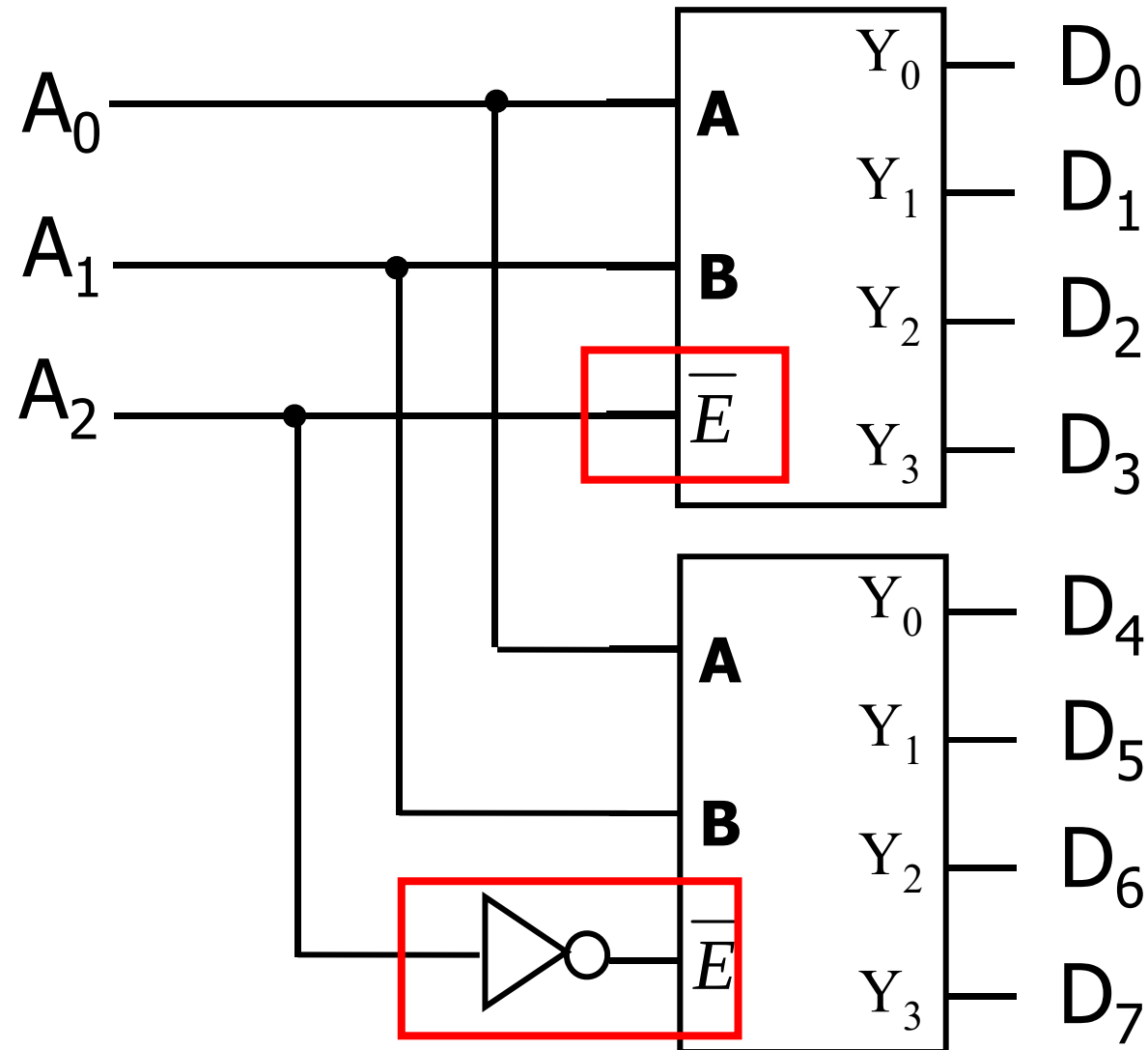
$$\begin{cases} Y_0 = \overline{\overline{E}} \overline{\overline{A}} \overline{\overline{B}} \\ Y_1 = \overline{\overline{E}} \overline{A} \overline{\overline{B}} \\ Y_2 = \overline{\overline{E}} \overline{\overline{A}} \overline{B} \\ Y_3 = \overline{\overline{E}} \overline{A} B \end{cases}$$



Logic diagram

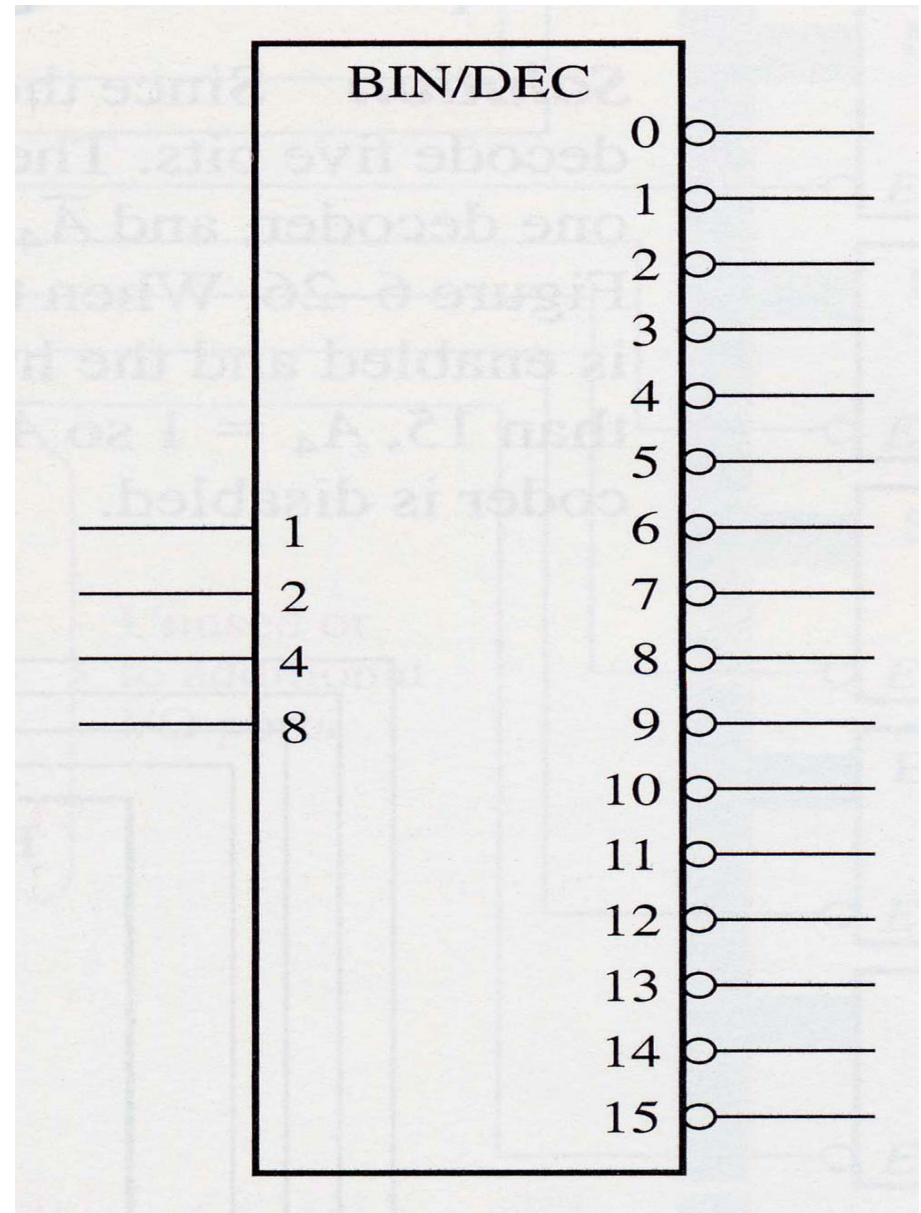


Expansion: 3-8 decoder

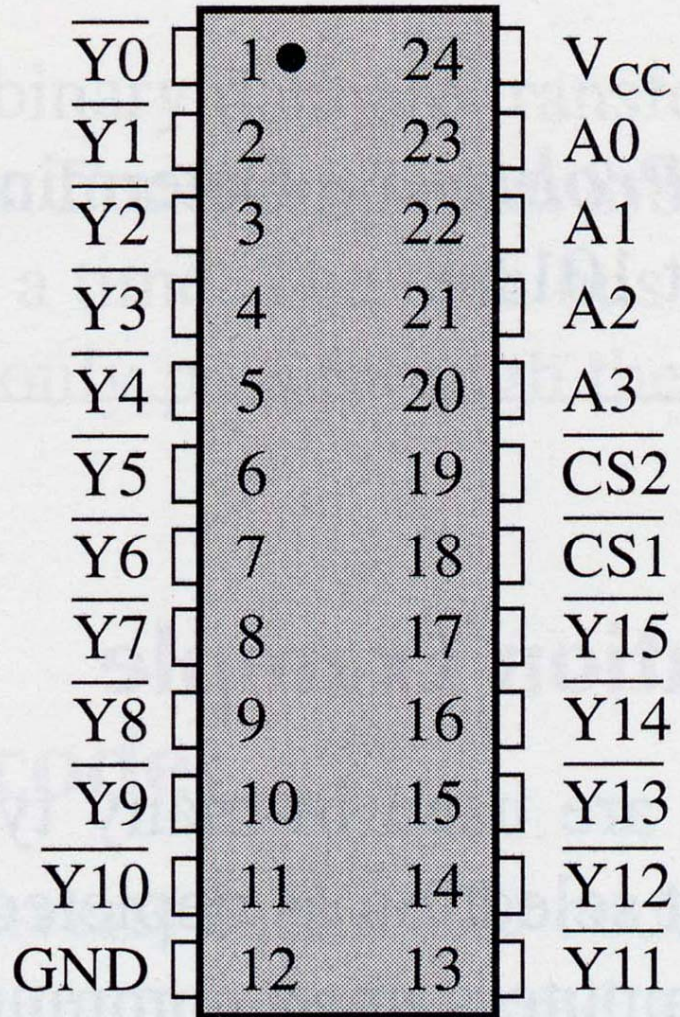


4-line-to-16-line Decoder

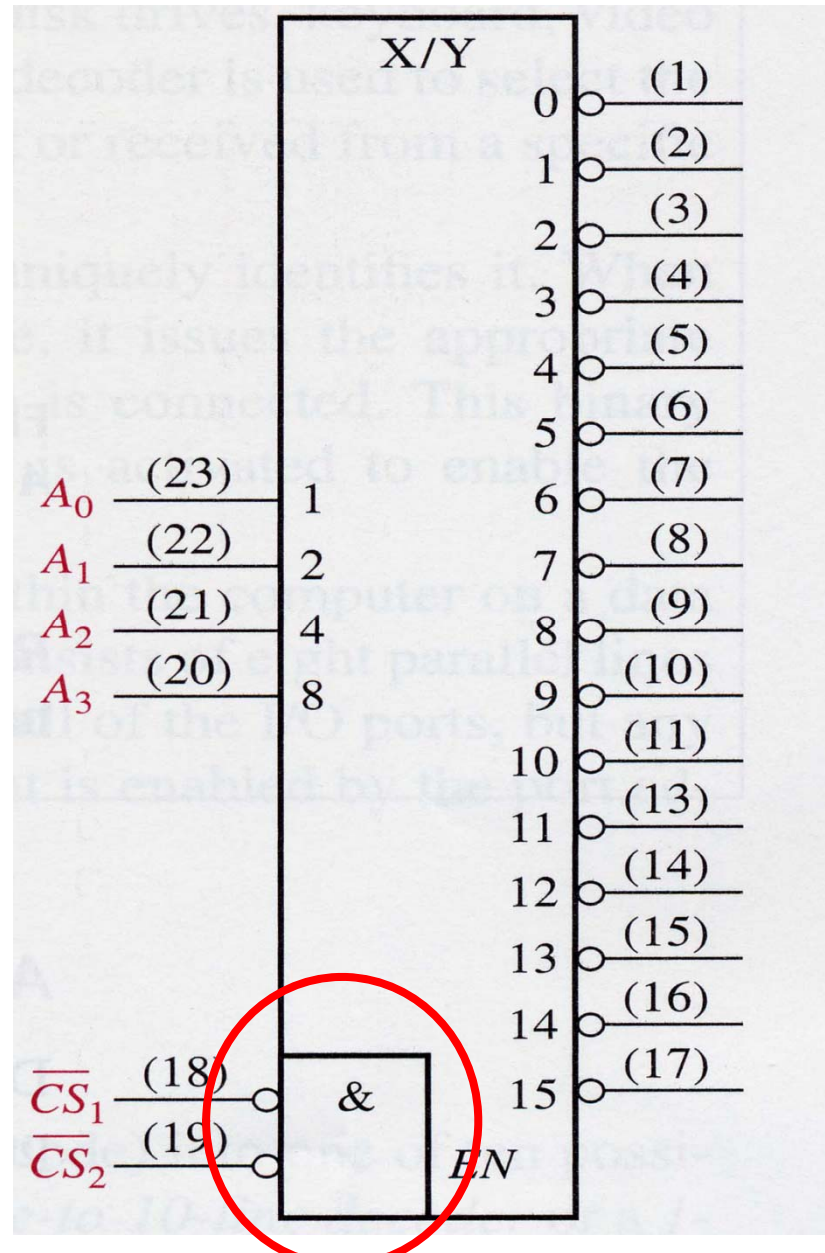
4-line-to-16-line Decoder
with active-LOW output



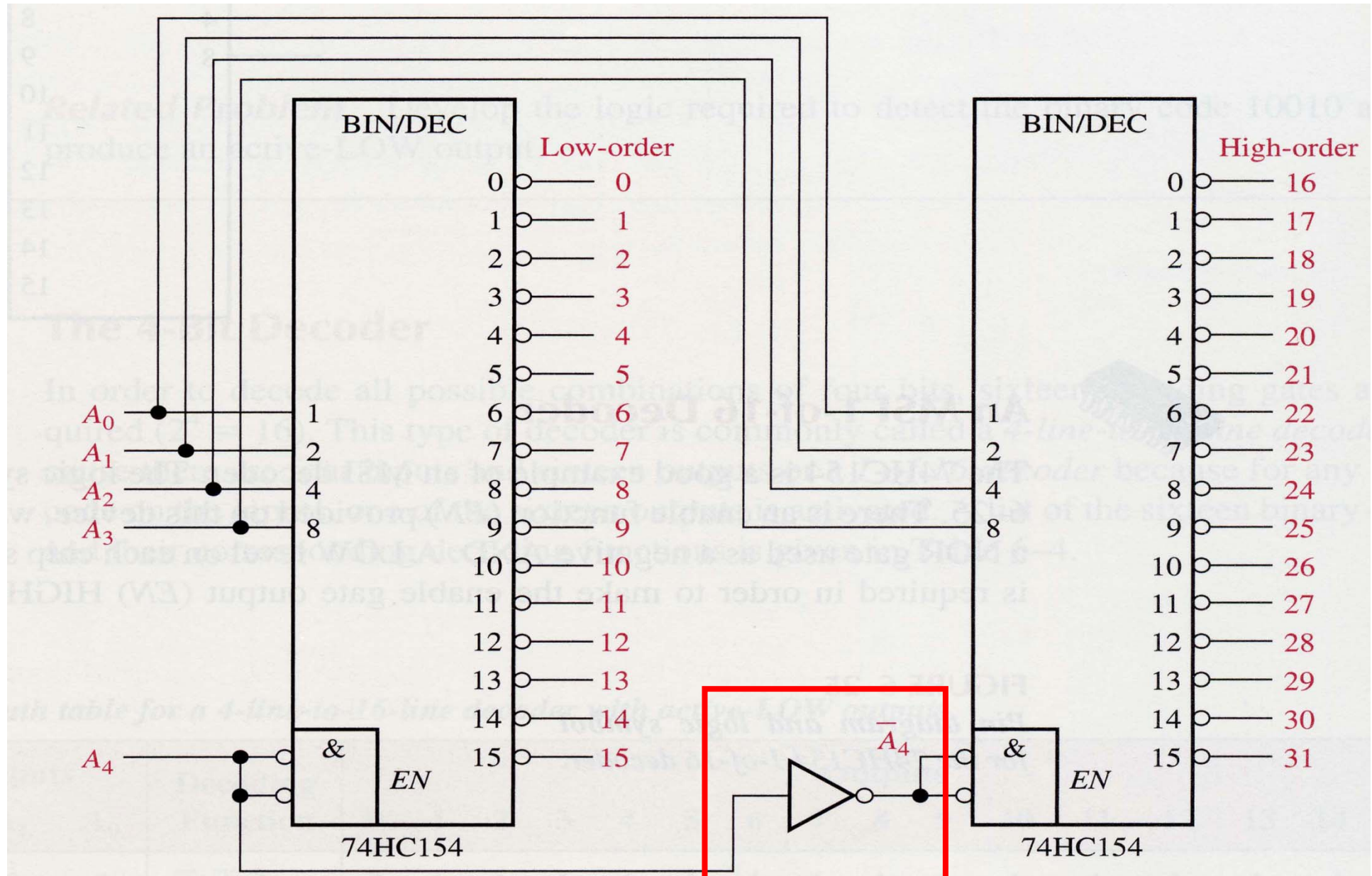
74HC154



(a) Pin diagram



Expansion: 5-bit decoder



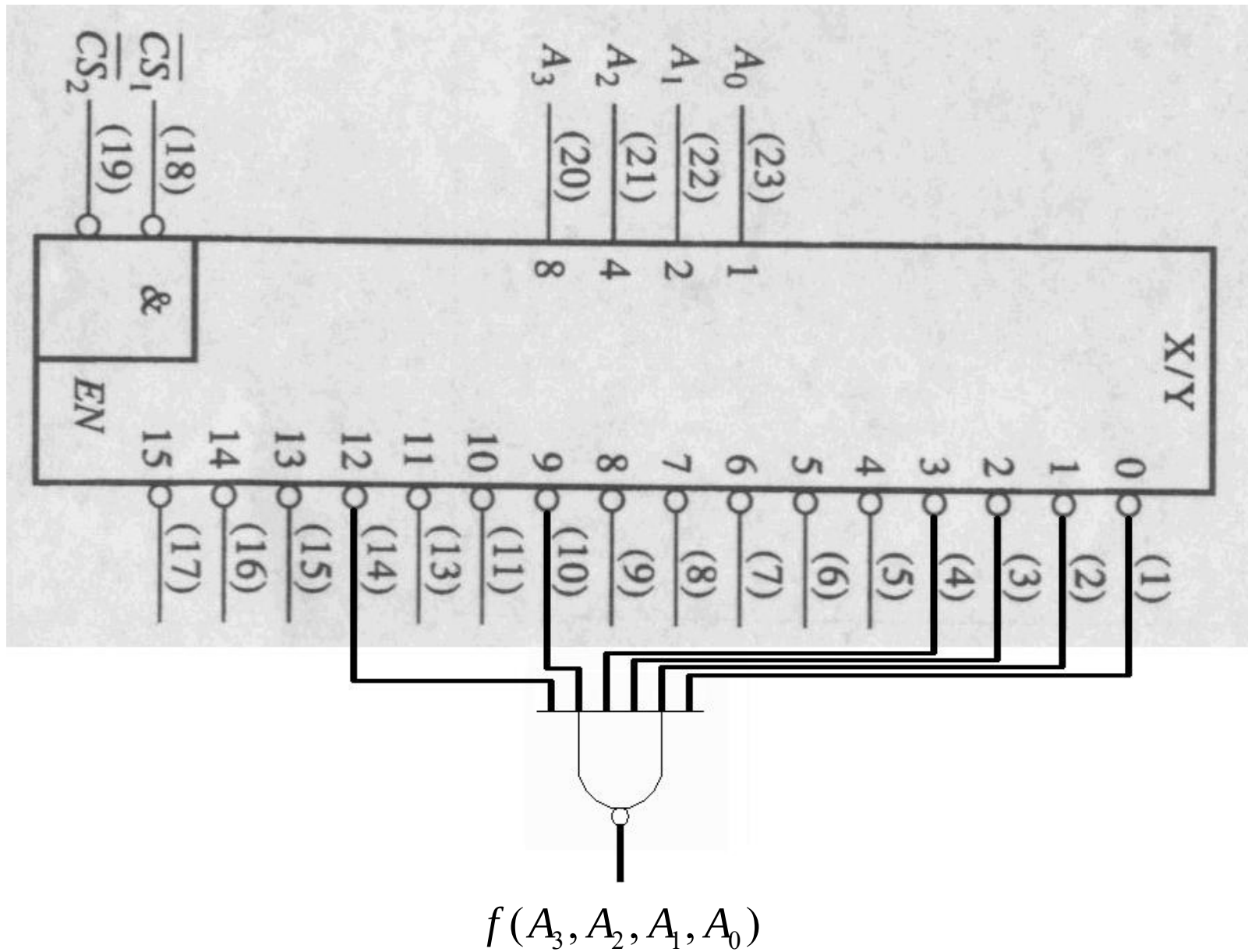
Exercise (homework) :

**用4片74HC154级联，设计一个6bit（6
line – 64 line）解码器**

Application Example : A Function Implemented by Decoder

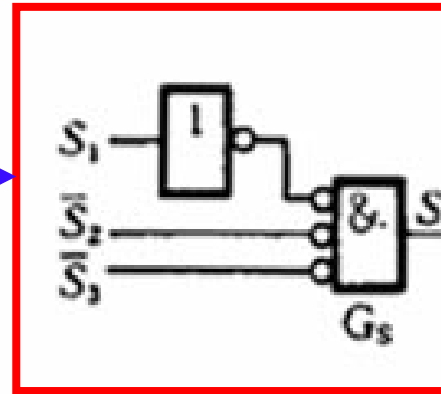
$$f(A, B, C, D) \\ = \overline{A}\overline{B} + AC\overline{D}$$

Example A Function Implemented by Decoder

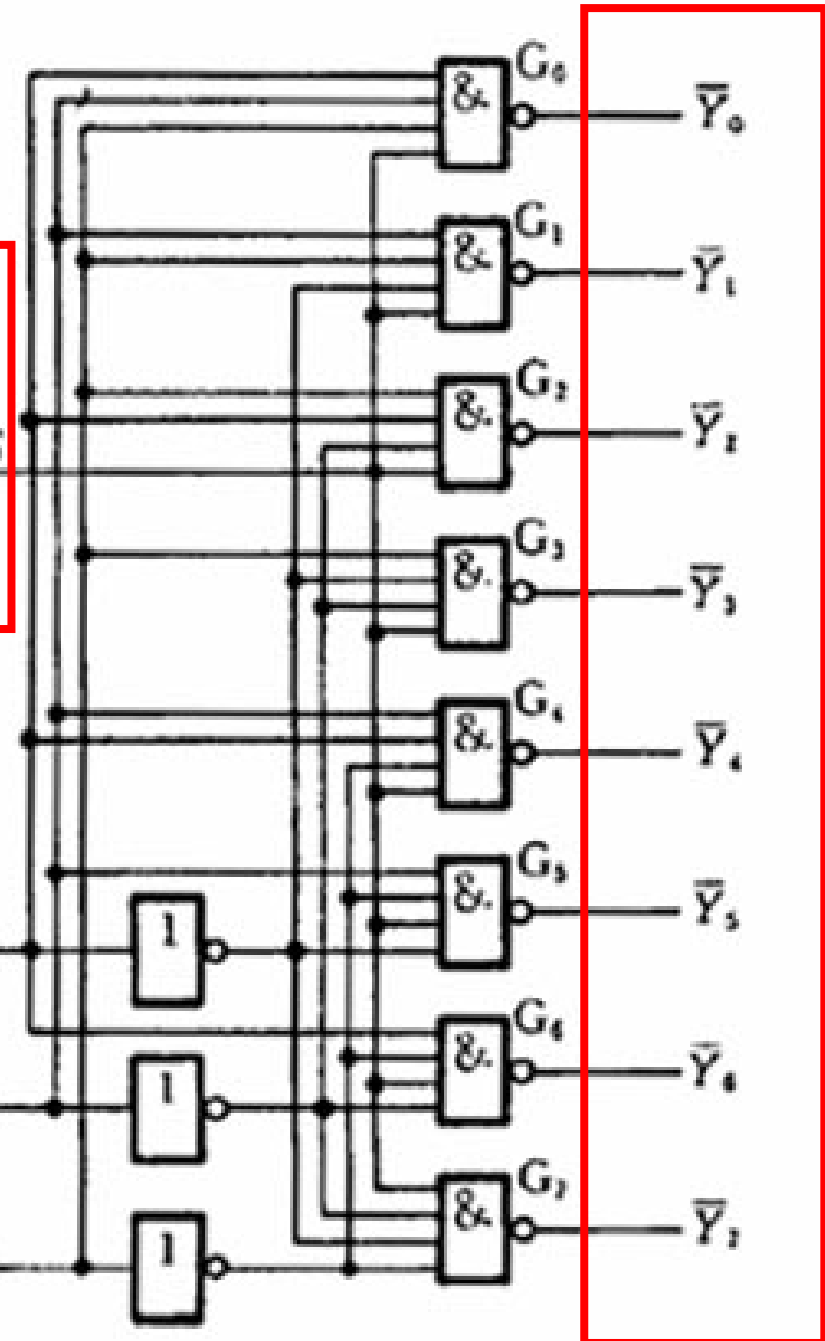
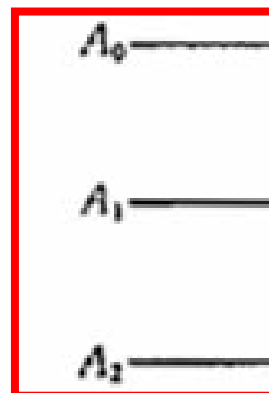


74LS138

EN →



Data →



Function table

[illegible]

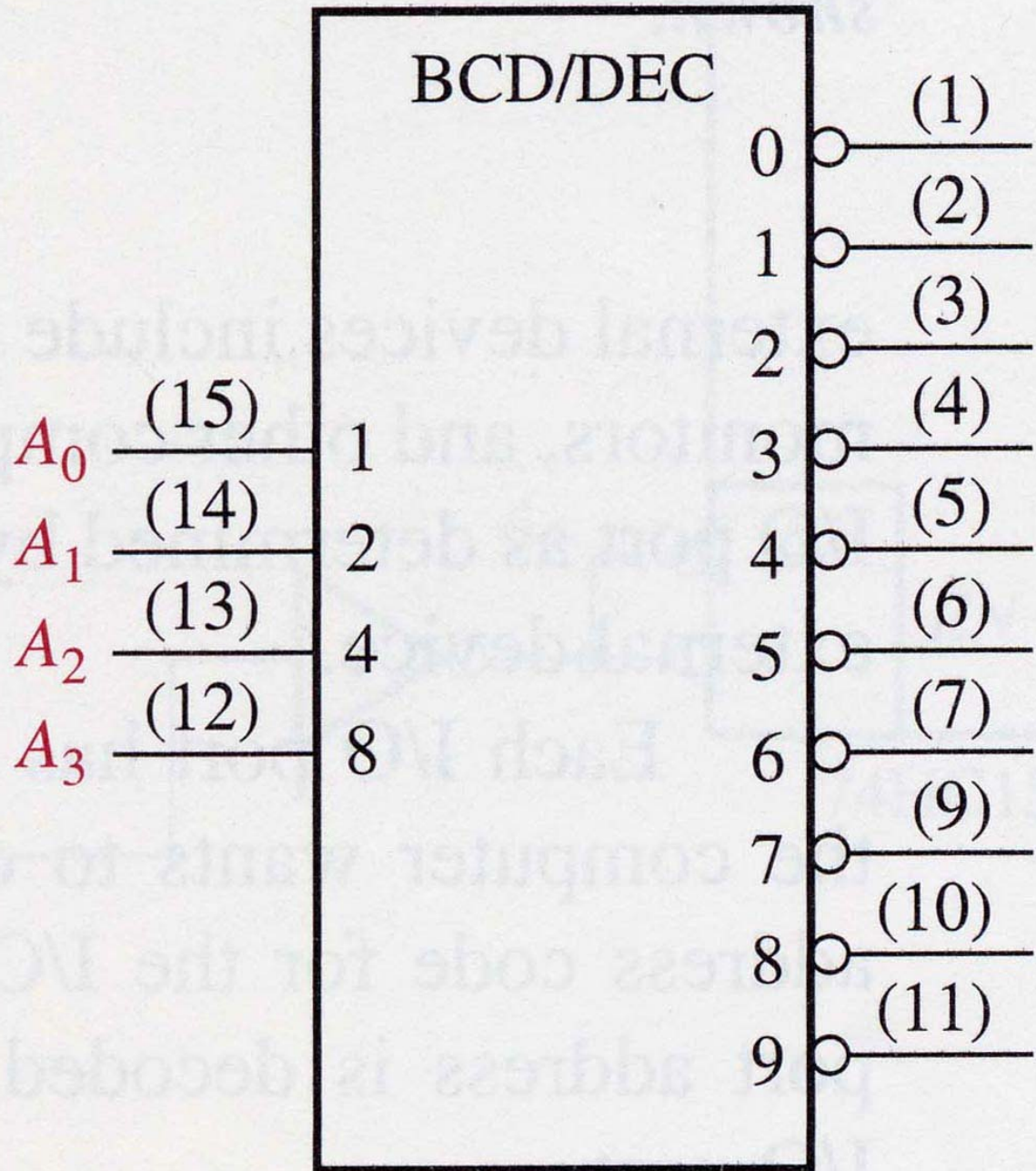
The BCD-to-Decimal Decoder

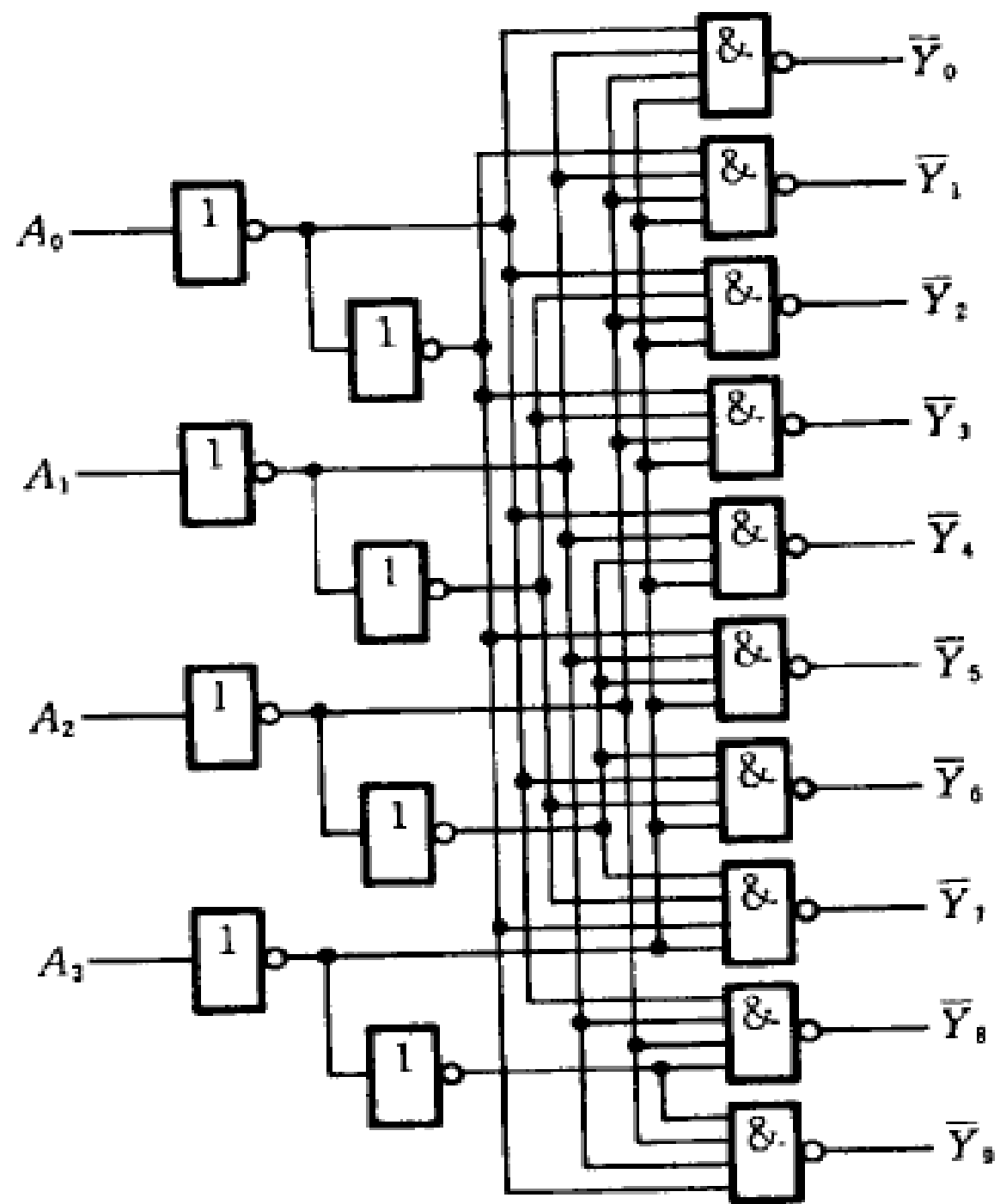
The BCD-to-Decimal Decoder converts each BCD code into one of ten possible decimal digit indications, usually called *4-line-to-10-line decoder* or *1-of-10 decoder*.

Function table of 1-of-10 Decoder

Decimal Digit	BCD Code				Decoding Function
	A_3	A_2	A_1	A_0	
0	0	0	0	0	$\bar{A}_3\bar{A}_2\bar{A}_1\bar{A}_0$
1	0	0	0	1	$\bar{A}_3\bar{A}_2\bar{A}_1A_0$
2	0	0	1	0	$\bar{A}_3\bar{A}_2A_1\bar{A}_0$
3	0	0	1	1	$\bar{A}_3\bar{A}_2A_1A_0$
4	0	1	0	0	$\bar{A}_3A_2\bar{A}_1\bar{A}_0$
5	0	1	0	1	$\bar{A}_3A_2\bar{A}_1A_0$
6	0	1	1	0	$\bar{A}_3A_2A_1\bar{A}_0$
7	0	1	1	1	$\bar{A}_3A_2A_1A_0$
8	1	0	0	0	$A_3\bar{A}_2\bar{A}_1\bar{A}_0$
9	1	0	0	1	$A_3\bar{A}_2\bar{A}_1A_0$

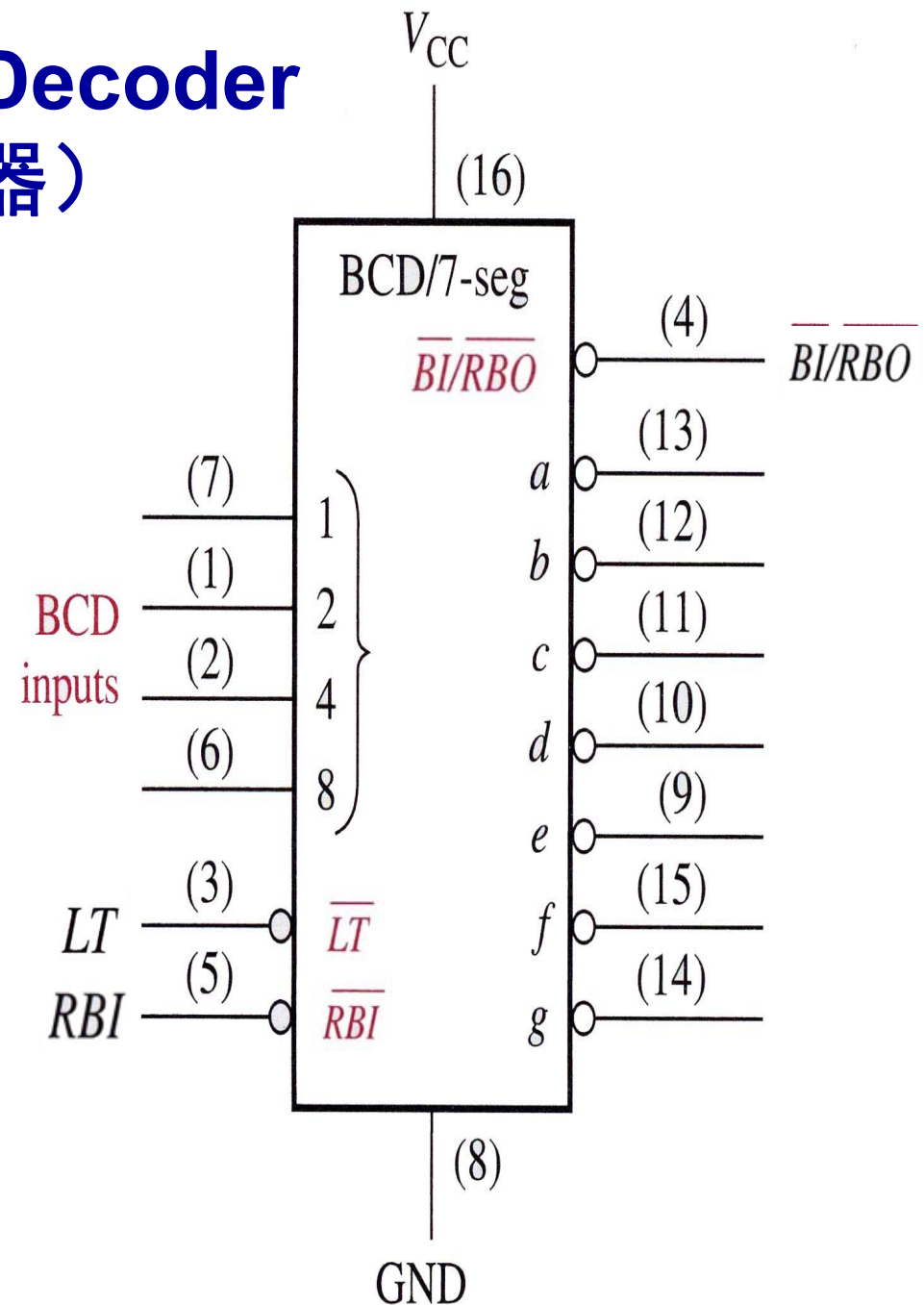
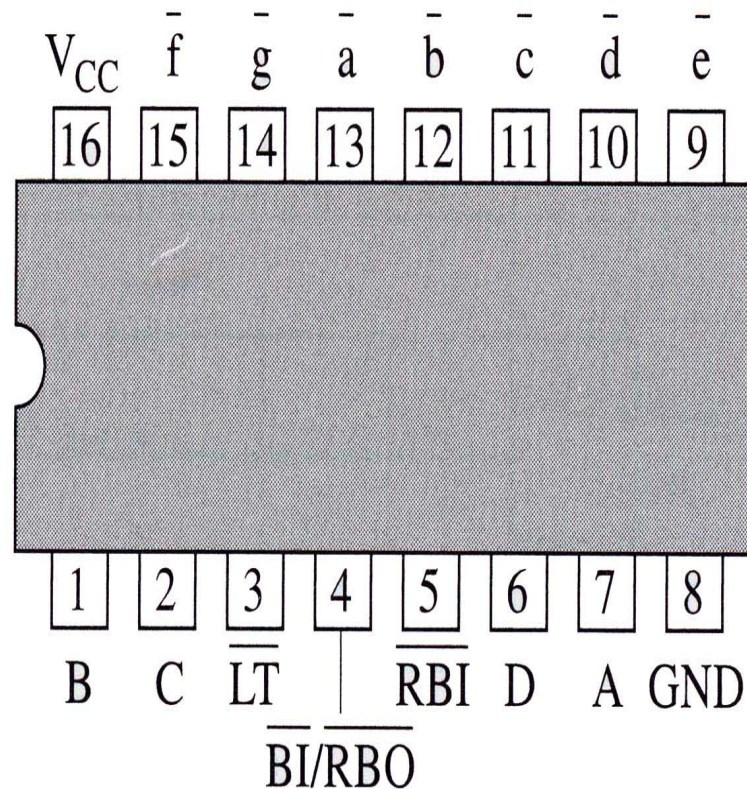
74HC42





BCD-to-7-Segment Decoder (BCD-7段解码器)

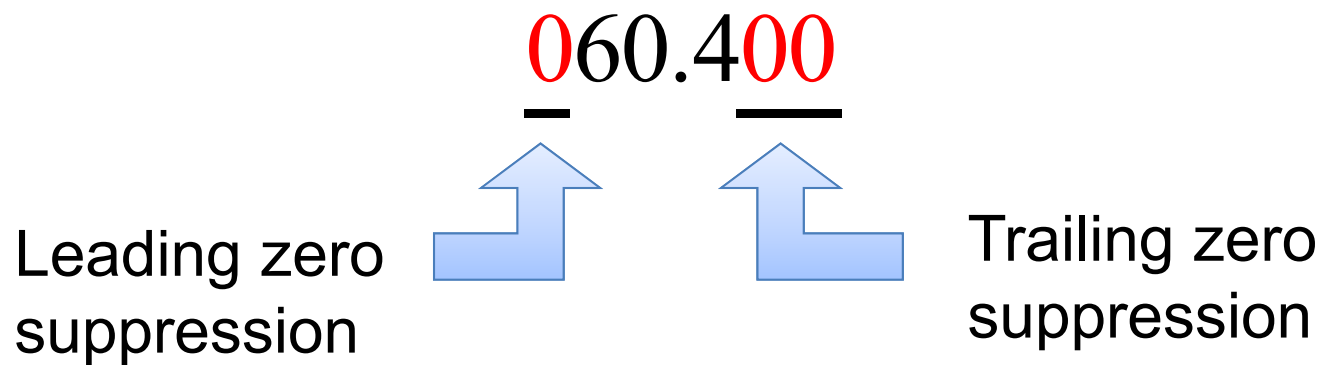
74LS47



Additional Pins:

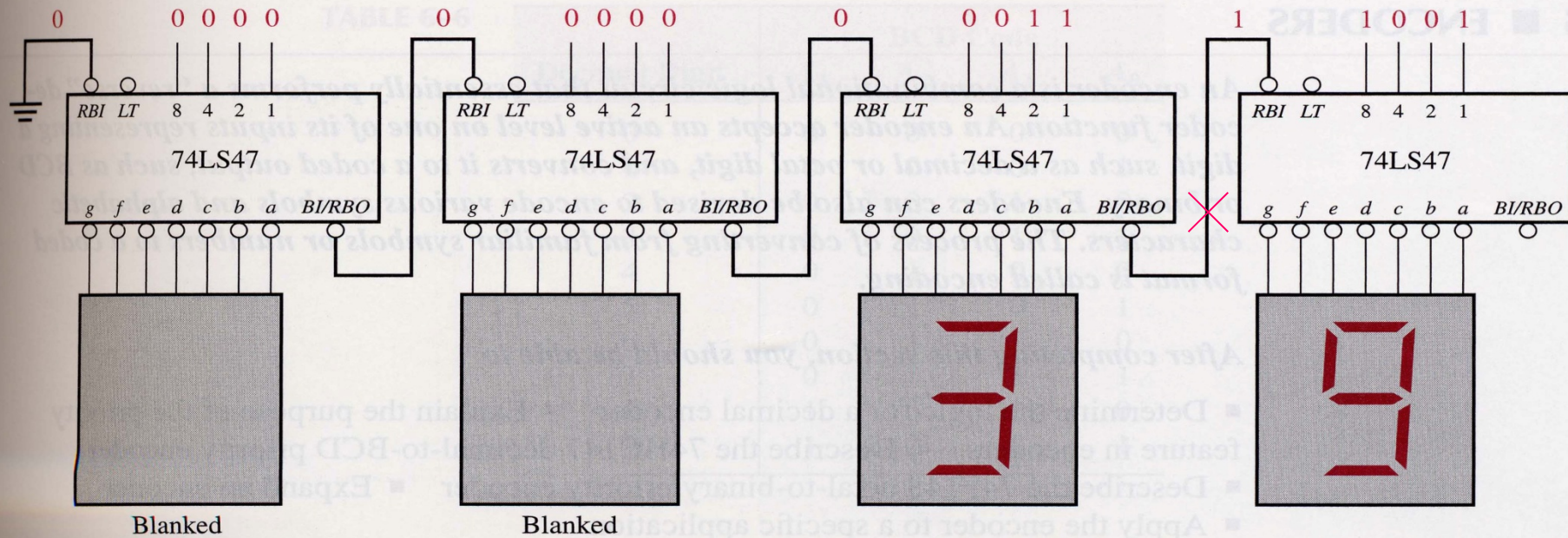
- *LT* : Lamp Test. When the input to lamp test is LOW, and *BI / RBO* is HIGH, all the 7 segments are turned on. Used to verify no segments are burned out.
- $\overline{RBI}, \overline{BI} / \overline{RBO}$ (Ripple Blanking Input , Blanking Input/Ripple Blanking Output): **zero suppression**. used for blank out unnecessary zeros.

- Zero suppression: used for multi-digit displays to blank out unnecessary zeros.
 - RBI : ripple blanking input;
 - RBO: ripple blanking output;
 - BI : blanking input, all segments output are HIGH(nonactive) when BI is LOW (overrides all other inputs)
- } zero suppression signals

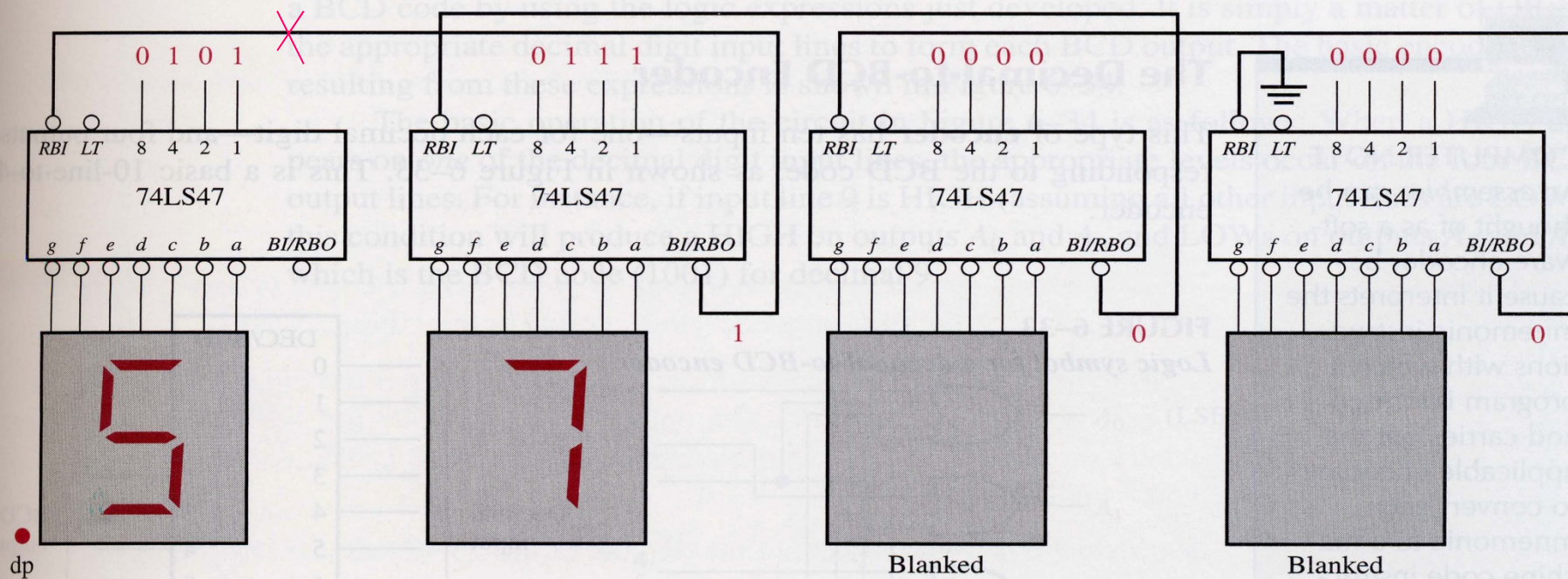


- When **RBI** is LOW and all BCD inputs are zeros (0000), then all segment outputs are HIGH (nonactive), the LEDs are turned off.
- When **RBO** is used as output, **RBO** is LOW when the above condition is satisfied.

$$RBO = \overline{\overline{A_3} \overline{A_2} \overline{A_1} \overline{A_0} LT \overline{RBI}}$$



(a) Illustration of leading zero suppression



(b) Illustration of trailing zero suppression

4 Encoders

编码器

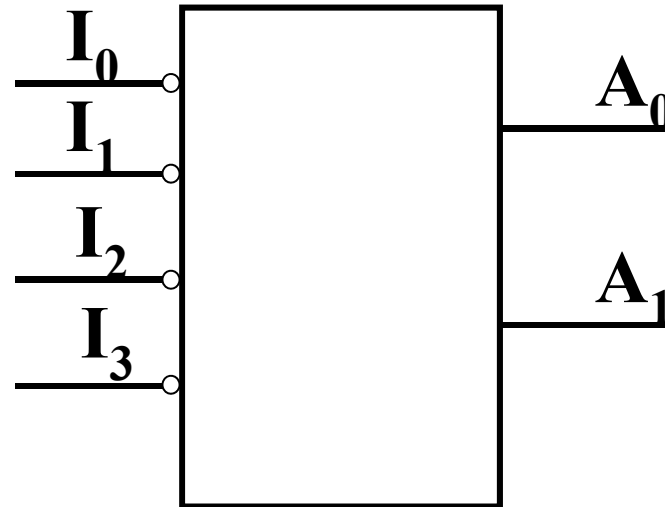
- 4-2 Encoders
- Decimal-to-BCD
- Priority Encoders
- Expansion

Principle of Encoders

- Encoder performs a “reverse” decoder function. i.e. converts a decimal digit to a binary code.

4-2 Encoder

I_0	I_1	I_2	I_3	A_1	A_0
0	1	1	1	0	0
1	0	1	1	0	1
1	1	0	1	1	0
1	1	1	0	1	1



$$\begin{cases} A_0 = I_0 \bar{I}_1 I_2 I_3 + I_0 I_1 I_2 \bar{I}_3 \\ A_1 = I_0 I_1 \bar{I}_2 I_3 + I_0 I_1 I_2 \bar{I}_3 \end{cases}$$

Decimal-to-BCD Encoder

- Convert decimal number to BCD coder.

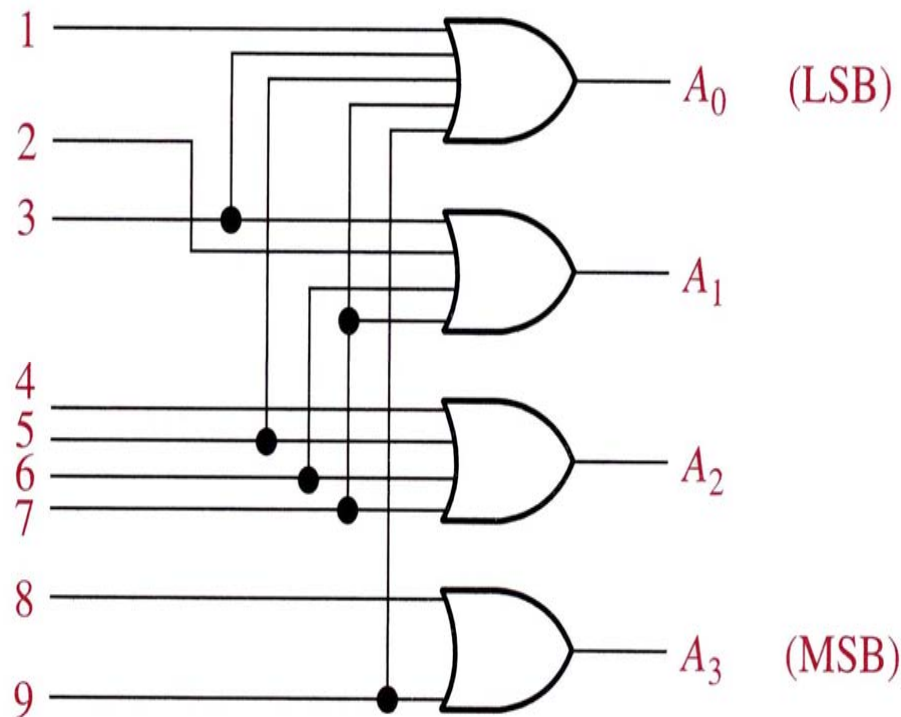
X_9 X_8 X_7 X_6 X_5 X_4 X_3 X_2 X_1 X_0	A_3 A_2 A_1 A_0
0 0 0 0 0 0 0 0 0 1	0 0 0 0
0 0 0 0 0 0 0 0 1 0	0 0 0 1
0 0 0 0 0 0 0 1 0 0	0 0 1 0
0 0 0 0 0 0 1 0 0 0	0 0 1 1
0 0 0 0 0 1 0 0 0 0	0 1 0 0
0 0 0 0 1 0 0 0 0 0	0 1 0 1
0 0 0 1 0 0 0 0 0 0	0 1 1 0
0 0 1 0 0 0 0 0 0 0	0 1 1 1
0 1 0 0 0 0 0 0 0 0	1 0 0 0
1 0 0 0 0 0 0 0 0 0	1 0 0 1

$$A_3 = X_8 + X_9$$

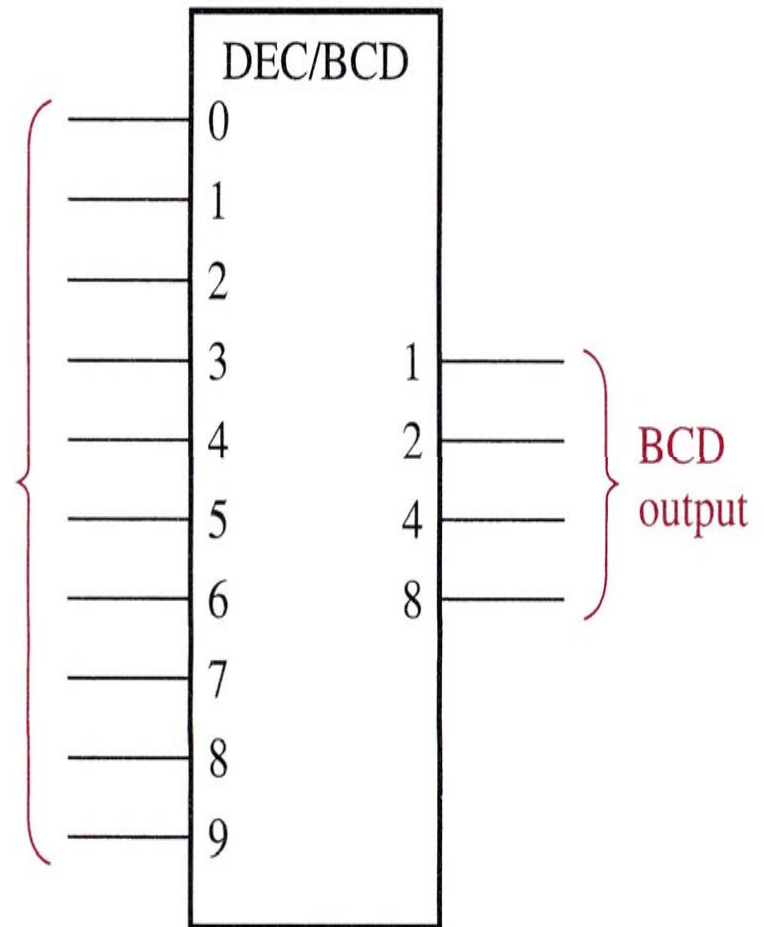
$$A_2 = X_4 + X_5 + X_6 + X_7$$

$$A_1 = X_2 + X_3 + X_6 + X_7$$

$$A_0 = X_1 + X_3 + X_5 + X_7 + X_9$$



Decimal
input



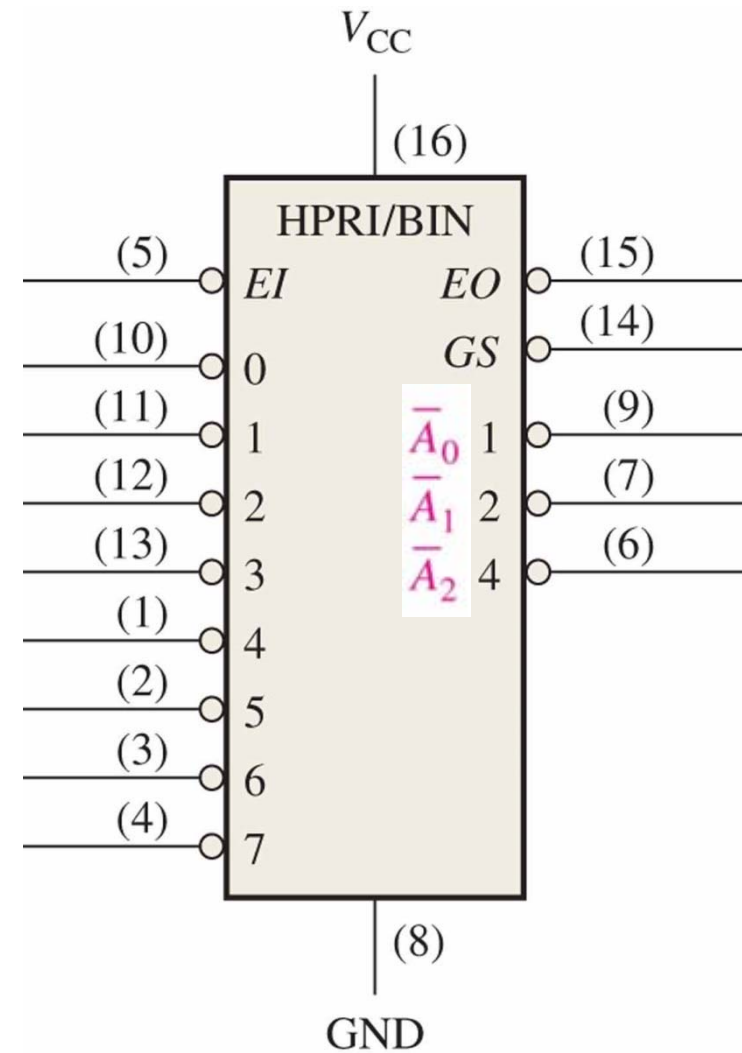
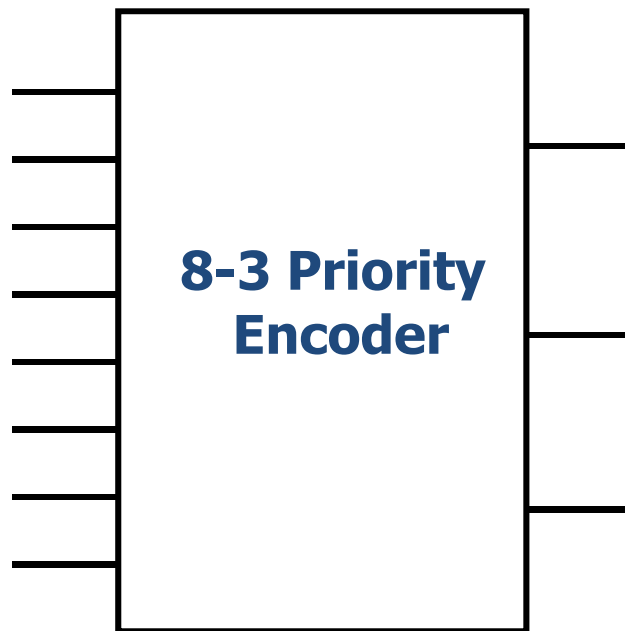
Note:

This kind of encoder works only when one input is active (HIGH) at a time, otherwise it will result in incorrect outputs.

Priority Encoder优先编码器

- The priority function means that the encoder will produce an output corresponding to the *highest-order* decimal digit input that is active and ignore any other lower-order active inputs.

- 74LS148 Oct-binary(8-3) priority encoder



- EI: enable input, active LOW.
- EO: enable output, active LOW when the EI is low and none of the inputs is active.
- GS: output, active LOW when EI is low and any of the inputs is active.

Function table (I)

<i>EI</i>	0	1	2	3	4	5	6	7	A ₀	A ₁	A ₂	<i>GS</i>	<i>EO</i>
0	×	×	×	×	×	×	×	0	0	0	0	0	1
0	×	×	×	×	×	×	0	1	1	0	0	0	1
0	×	×	×	×	×	0	1	1	0	1	0	0	1
0	×	×	×	×	0	1	1	1	1	1	0	0	1
0	×	×	×	0	1	1	1	1	1	0	1	0	1
0	×	×	0	1	1	1	1	1	1	0	1	0	1
0	×	0	1	1	1	1	1	1	1	0	1	1	0
0	0	1	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
1	×	×	×	×	×	×	×	×	1	1	1	1	1

Function table (II)

$\overline{EI}$	$\overline{0}$	$\overline{1}$	$\overline{2}$	$\overline{3}$	$\overline{4}$	$\overline{5}$	$\overline{6}$	$\overline{7}$	$\overline{A_0}$	$\overline{A_1}$	$\overline{A_2}$	$\overline{GS}$	$\overline{EO}$
1	×	×	×	×	×	×	×	1	1	1	1	1	0
1	×	×	×	×	×	×	1	0	0	1	1	1	0
1	×	×	×	×	×	1	0	0	1	0	1	1	0
1	×	×	×	×	1	0	0	0	0	0	1	1	0
1	×	×	×	1	0	0	0	0	1	1	0	1	0
1	×	×	1	0	0	0	0	0	0	1	0	1	0
1	×	1	0	0	0	0	0	0	1	0	0	1	0
1	1	0	0	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	0	0	0	0	0	1
0	×	×	×	×	×	×	×	×	0	0	0	0	0

$$\overline{A_0} = \overline{EI} \left(\overline{7} + 7\overline{6}\overline{5} + 76\overline{5}4\overline{3} + 765432\overline{1} = \overline{7} + \overline{6}\overline{5} + 64\overline{3} + 642\overline{1} \right)$$

$$\Rightarrow A_0 = \overline{\overline{EI} \left(\overline{7} + \overline{6}\overline{5} + 64\overline{3} + 642\overline{1} \right)} = EI + \overline{\overline{7} + \overline{6}\overline{5} + 64\overline{3} + 642\overline{1}}$$

$$\overline{A_1} = \overline{EI} \left(\overline{7} + 7\overline{6} + 76\overline{5}4\overline{3} + 765432\overline{2} = \overline{7} + \overline{6} + 54\overline{3} + 542\overline{2} \right)$$

$$\Rightarrow A_1 = \overline{\overline{EI} \left(\overline{7} + \overline{6} + 54\overline{3} + 542\overline{2} \right)} = EI + \overline{\overline{7} + \overline{6} + 54\overline{3} + 542\overline{2}}$$

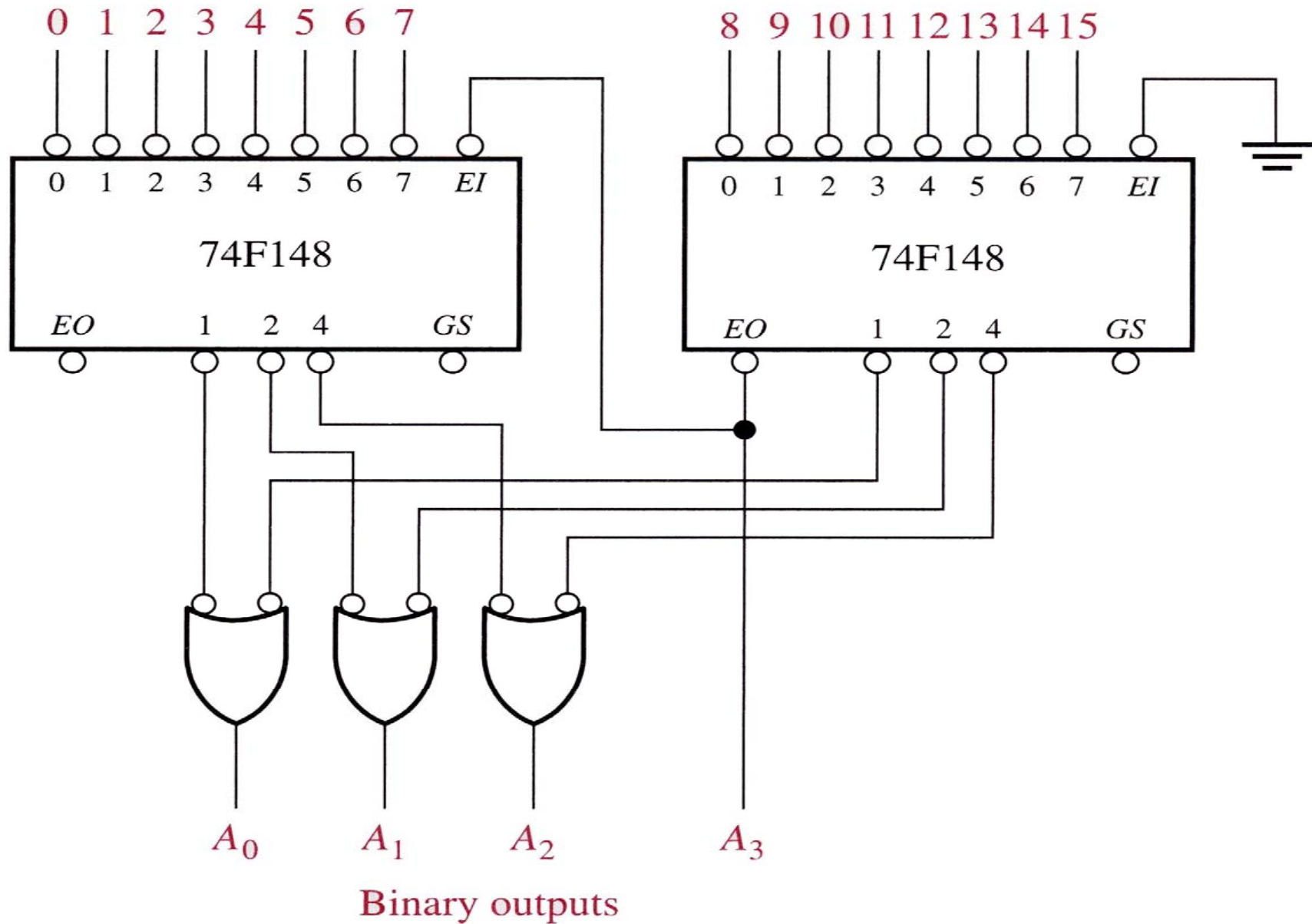
$$\overline{A_2} = \overline{EI} \left(\overline{7} + 7\overline{6} + 76\overline{5} + 765\overline{4} = \overline{7} + \overline{6} + \overline{5} + \overline{4} \right)$$

$$\Rightarrow A_2 = \overline{\overline{EI} \left(\overline{7} + \overline{6} + \overline{5} + \overline{4} \right)} = EI + \overline{\overline{7} + \overline{6} + \overline{5} + \overline{4}}$$

$$\overline{GS} = \overline{EI76543210} \Rightarrow GS = EI + 76543210$$

$$\overline{EO} = GS \bullet \overline{EI} \Rightarrow EO = \overline{GS \bullet \overline{EI}} = \overline{GS} + EI$$

Expansion: 16-4 Priority Encoder



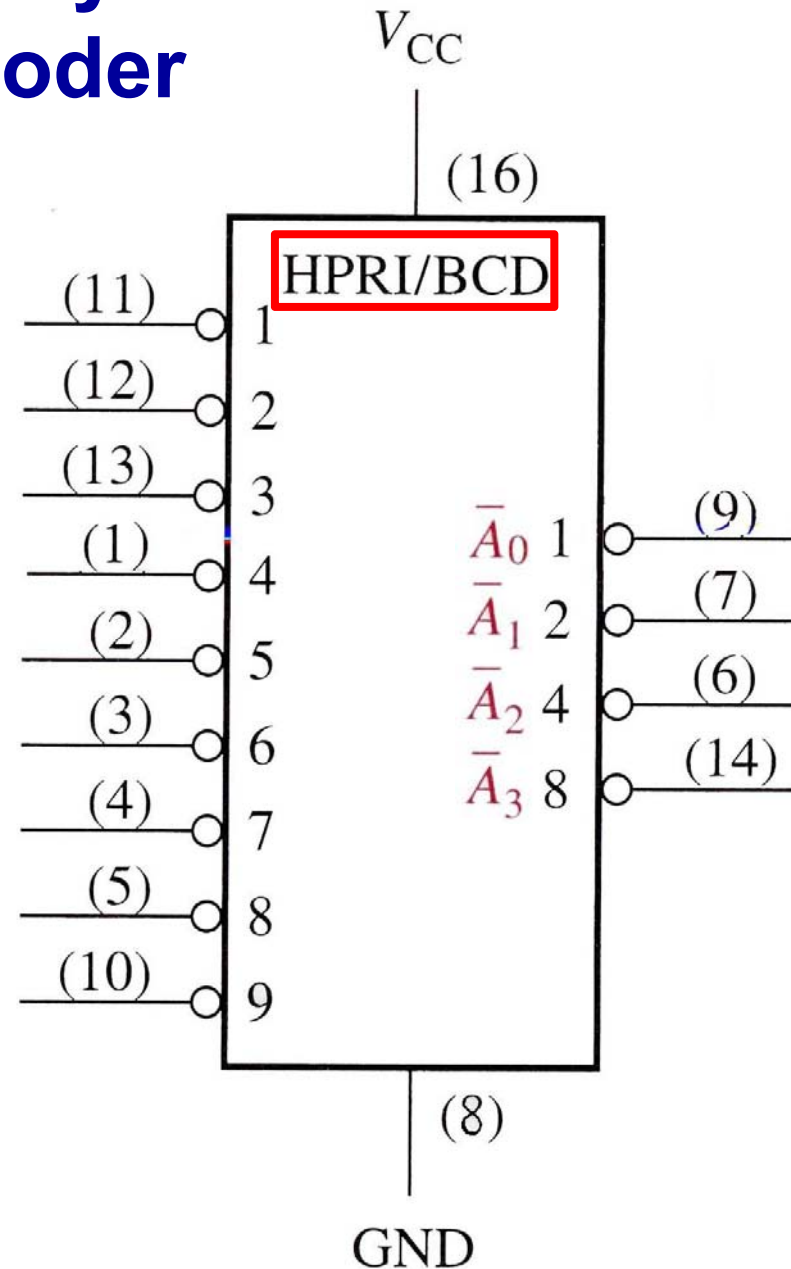
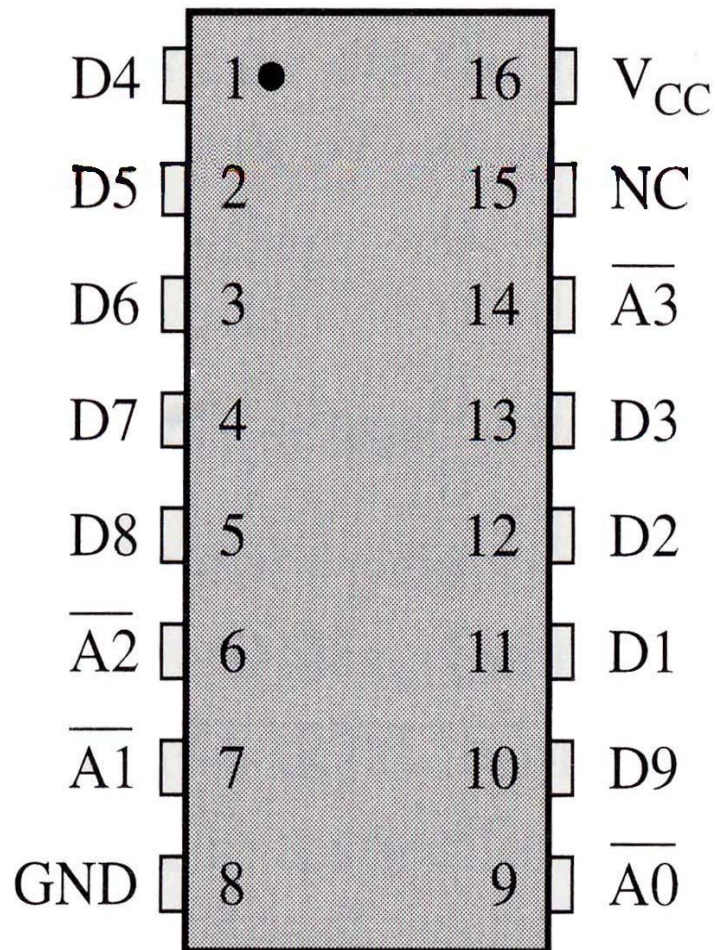
Decimal-to-BCD Priority Encoder (十进制BCD优先编码器)

74HC147: Active-LOW

(note: 74LS47 BCD to 7-segment decoder)

74LS147 (9-4) Priority Decimal-to-BCD encoder

(1-9, 0 for none)

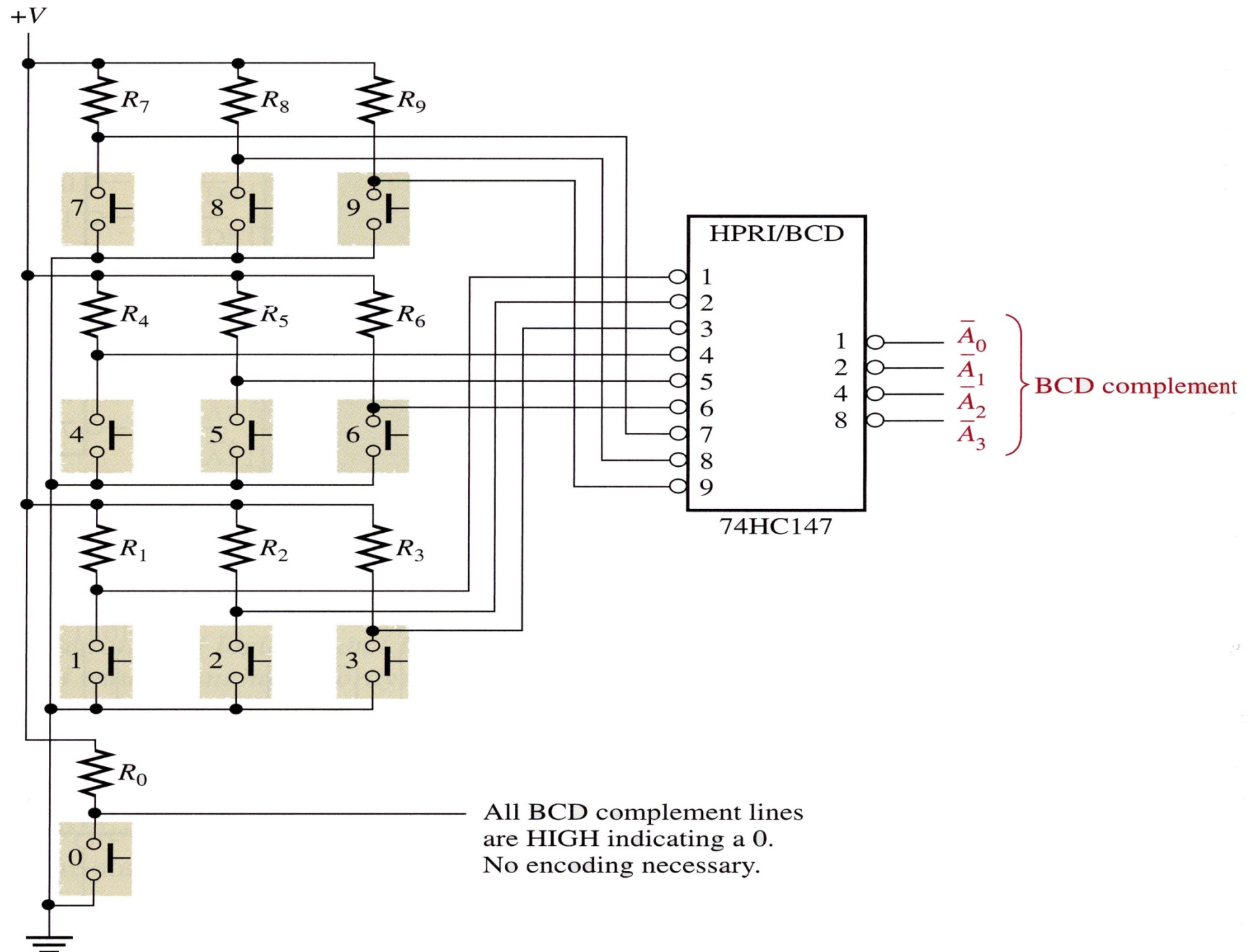


Exercise (homework) :

设计74LS147的实现电路，要求给出设计过程，包括真值表，逻辑表达式及化简

Application example

Simplified keyboard encoder.



5 Coder Converter (码字转换器)

- Convert one coder to another

BCD-Binary Conversion

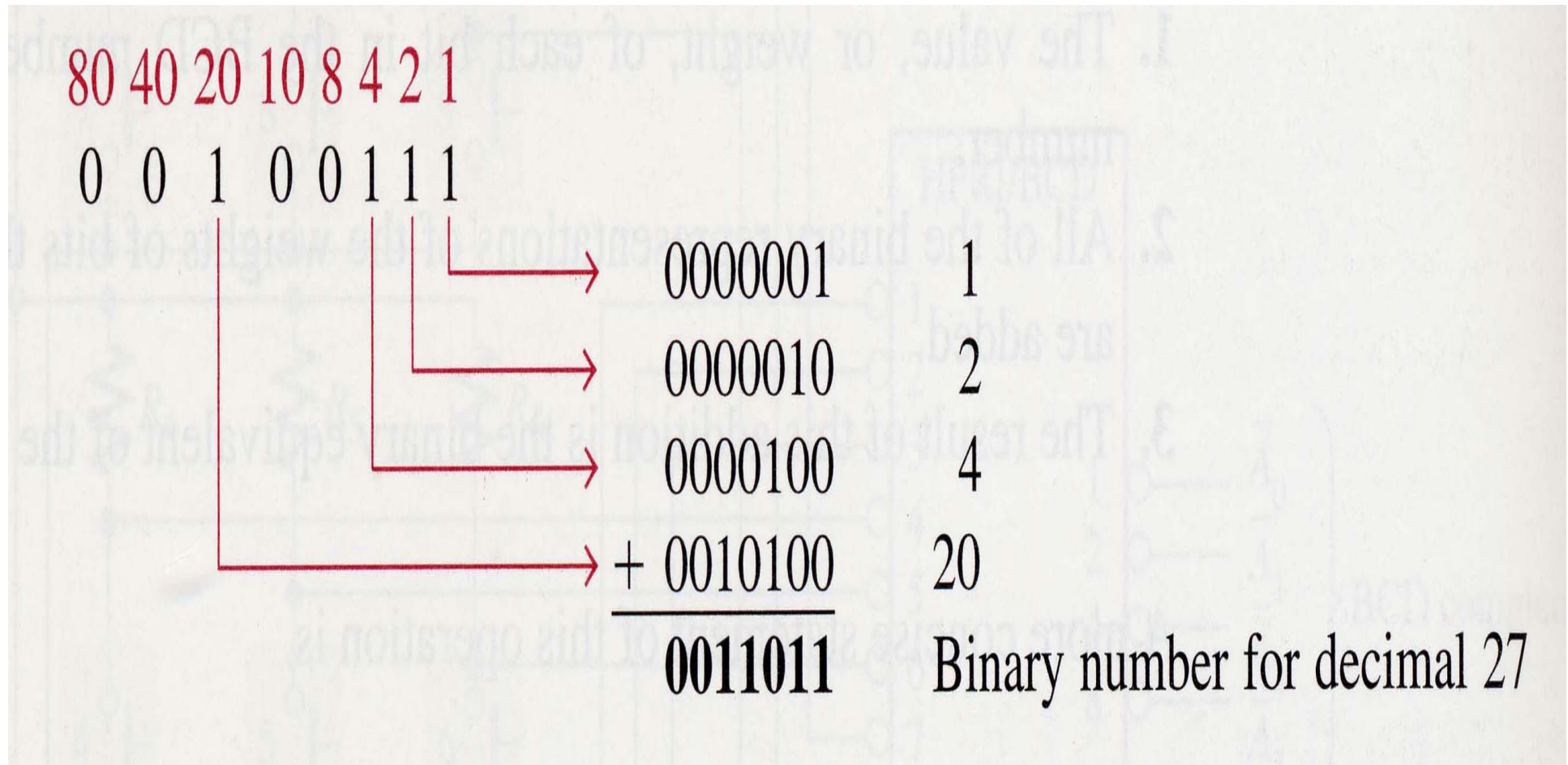
- Steps
 1. Determine the value or weight of each bit in the BCD number.
 2. Add all the weights that the corresponding bits are 1s.
 3. The sum is the equivalent binary number of BCD number.

Example: BCD00100111(27)→Binary number

1. The weights of each bit as follows:

BCD numbers	0	0	1	0	0	1	1	1
	↑	↑	↑	↑	↑	↑	↑	↑
weights	80	40	20	10	8	4	2	1
	↑	↑	↑	↑	↑	↑	↑	↑
Binary weights	1010000	0101000	0010100	0001010	0001000	0000100	0000010	0000001

2. Sum of the weights:



Sum **00011011**

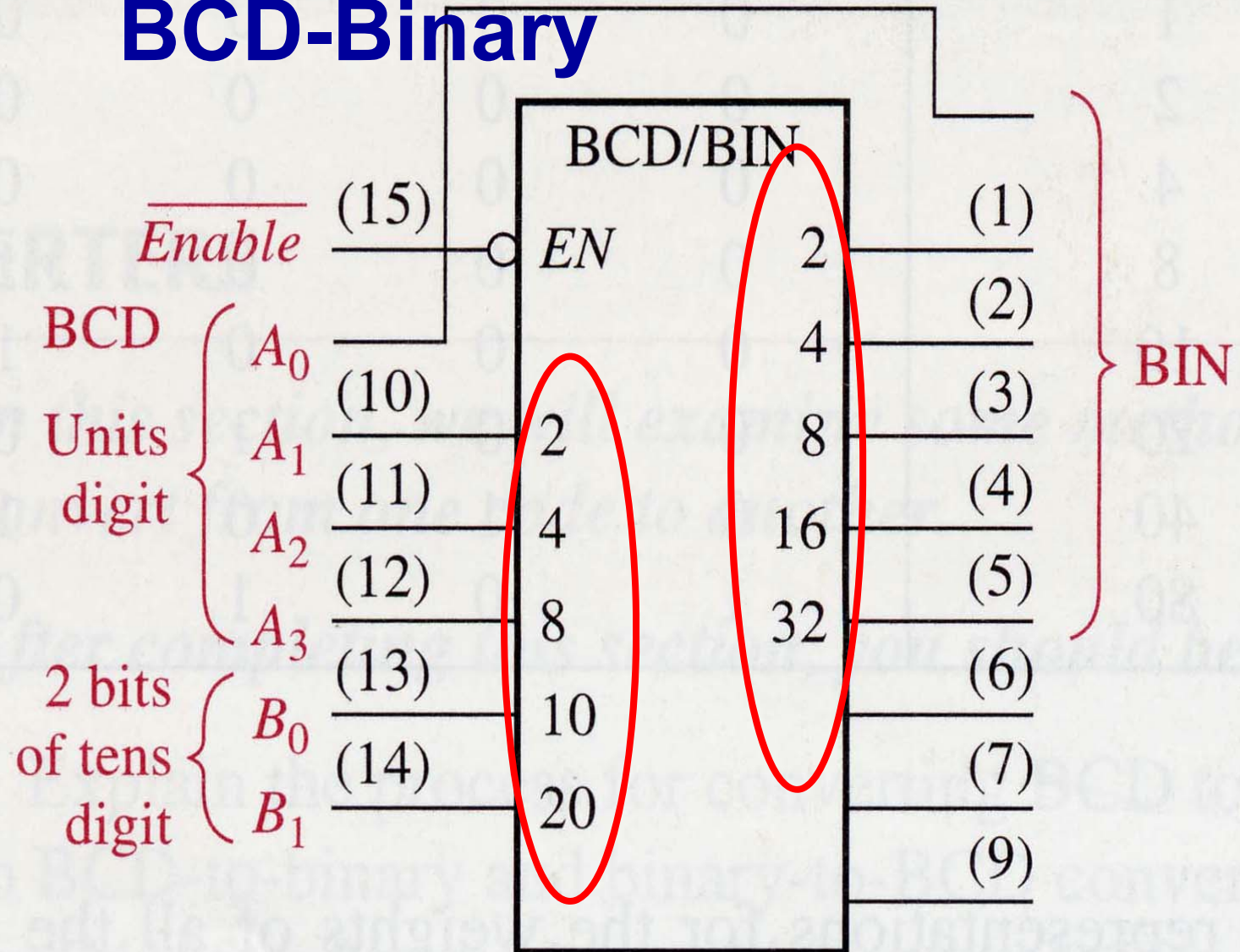
(99 < 127 = 2⁷)

BCD to Binary Conversion

- The binary number that corresponds to the BCD number is produced by adding the binary representations for the weights of all 1s in the BCD number:

$$\begin{aligned} & (10000111)_{\text{BCD}} \\ &= 1 \times 80 + 0 \times 40 + 0 \times 20 + 0 \times 10 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1 \\ &= 80 + 4 + 2 + 1 \\ &= 1010000 + 0000100 + 0000010 + 0000001 \\ &= (1010111)_2 \\ &= (87)_{10} \end{aligned}$$

BCD-Binary



(a) The 74184 6-bit BCD-to-binary converter

Binary-to-Gray / Gray-to-Binary

- 二进制码与格雷码转换

The Gray Code (格雷码)

- The Gray code is unweighted and is not an arithmetic code; that is, there are no specific weights assigned to the bit positions.
- The important feature of the Gray code is that it exhibits only a single bit change from one code number to the next.相邻码

TABLE 2-6
Four-bit Gray code.

Decimal	Binary	Gray Code	Decimal	Binary	Gray Code
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

Binary-to-Gray code Conversion

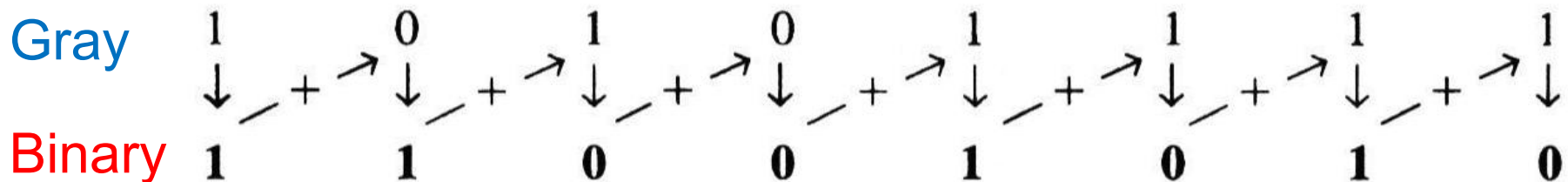
- The MSB in the Gray code is the same as the corresponding MSB in the binary number.
- Going from left to right, add each adjacent pair of binary code bits to get the next Gray code bit. Discard carries.

Binary	1	—	+	→	1	—	+	→	0	—	+	→	0	—	+	→	0	—	+	→	1	—	+	→	1	—	+	→	0
	↓				↓				↓				↓				↓			↓				↓				↓	
Gray	1				0				1				0				0			1				0				1	

Four-bit binary-to-Gray conversion logic

Gray-to-Binary Conversion

- The MSB in the binary code is the same as the corresponding MSB in the Gray code.
- Going from left to right, add each binary code bit generated to the Gray code bit in the next adjacent position. Discard carries.



Four-bit Gray-to-binary conversion logic

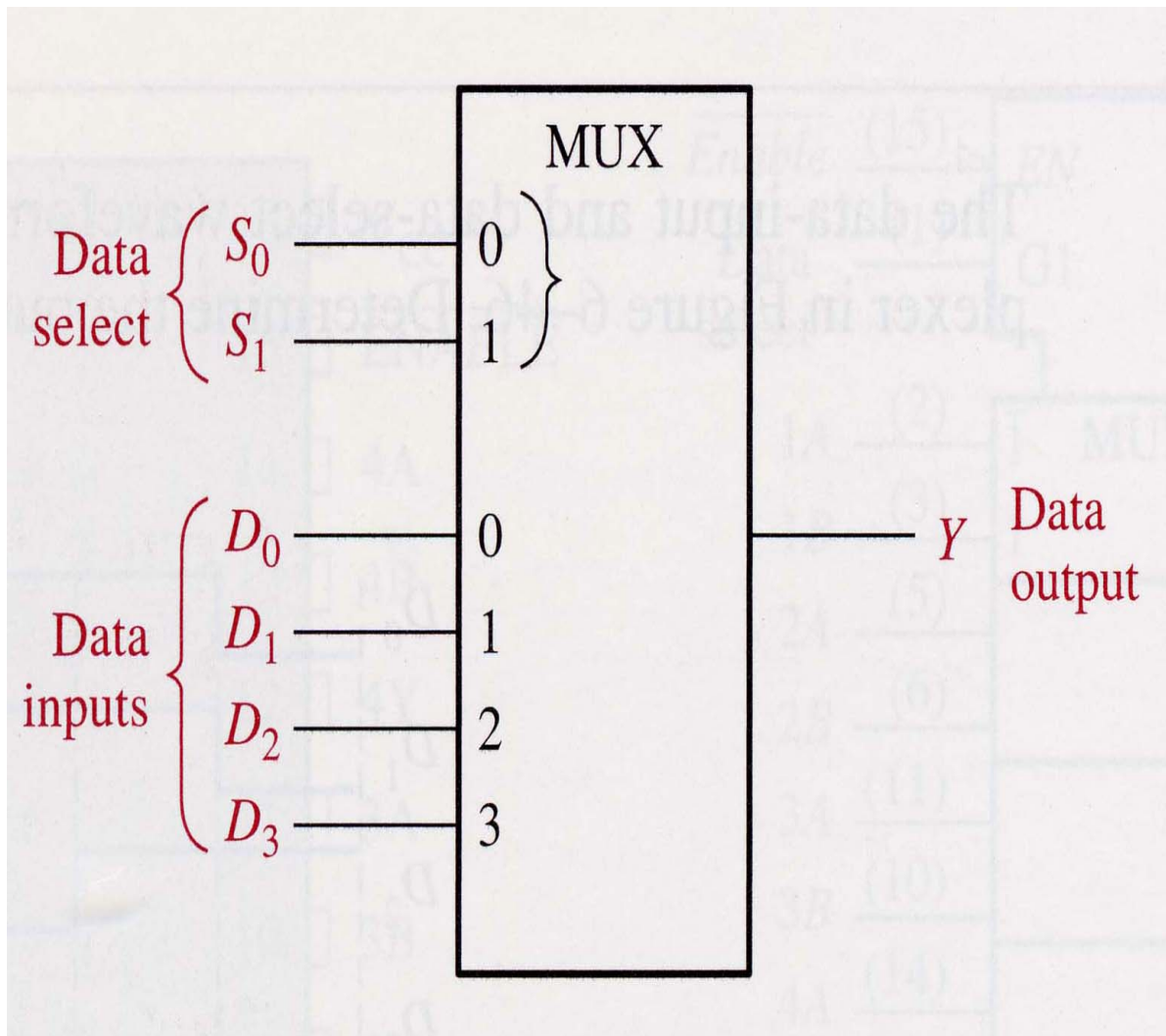
6 Data Selectors (Multiplexers)

数据选择器（多路转换器）

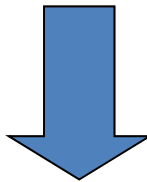
A multiplexer(MUX) is a device that allows digital information from several sources to be routed onto a single line for transmission.

Multiplexers are also known as data selectors.

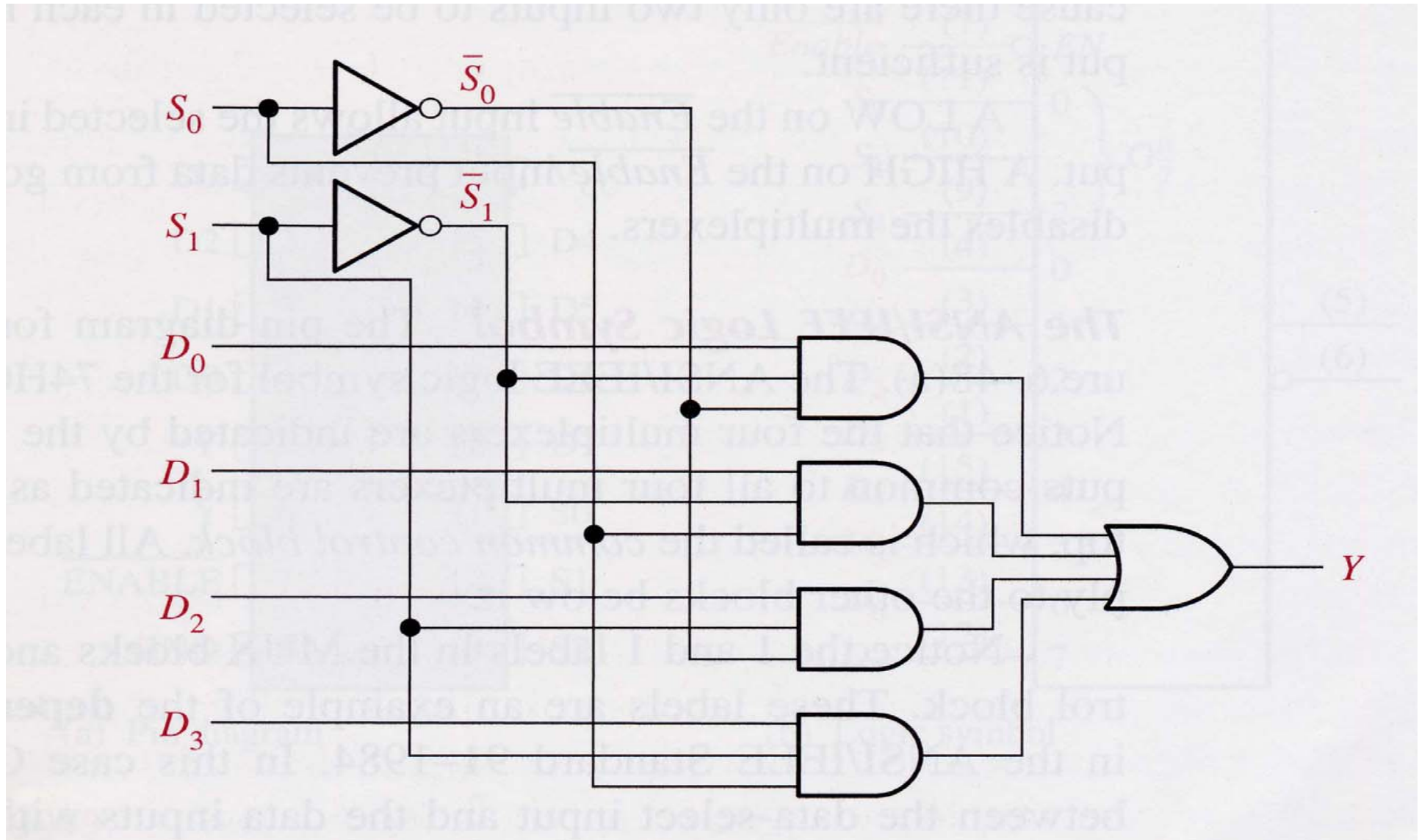
1-of-4 data selector

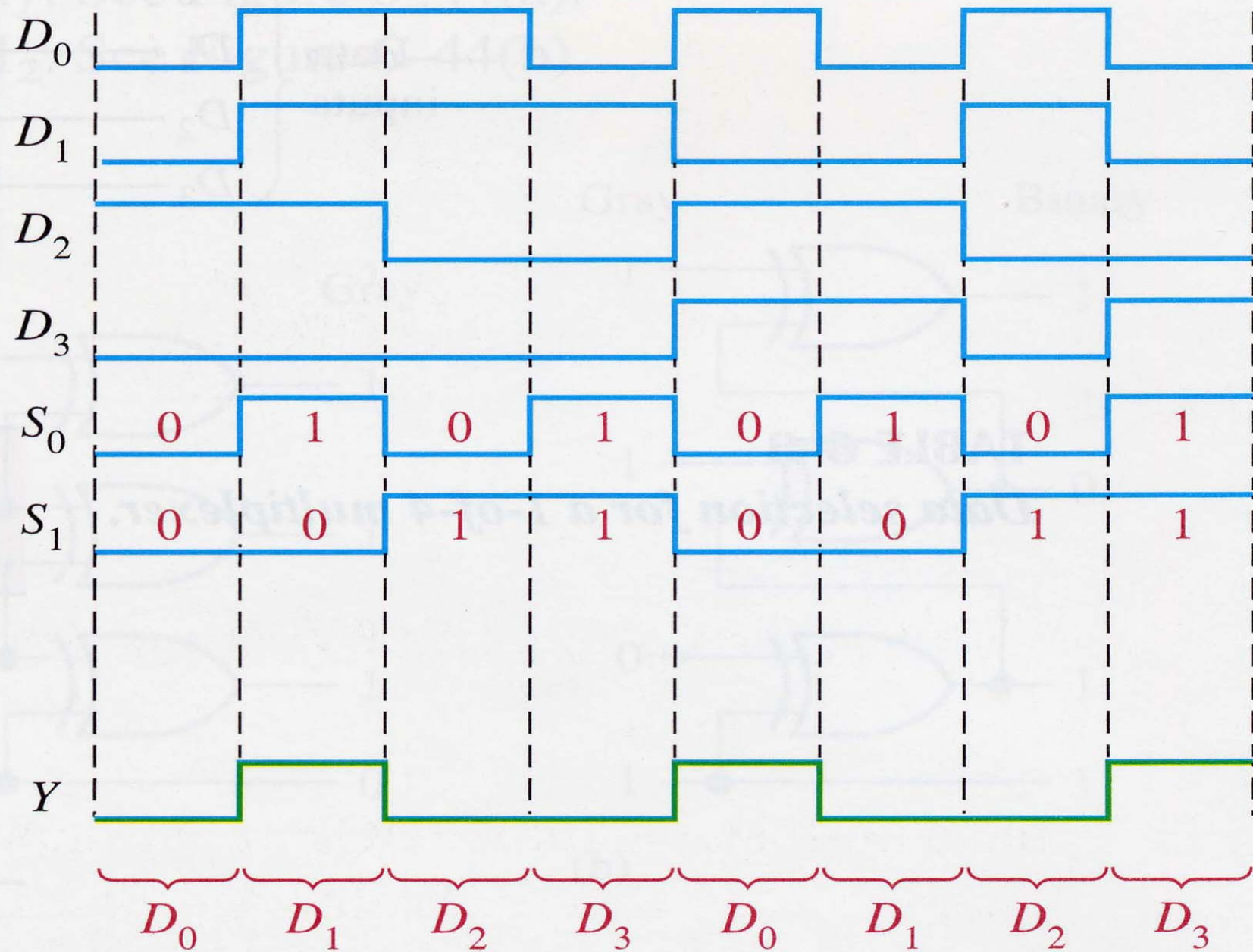


Data-select Inputs		Input Selected
S_1	S_0	
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

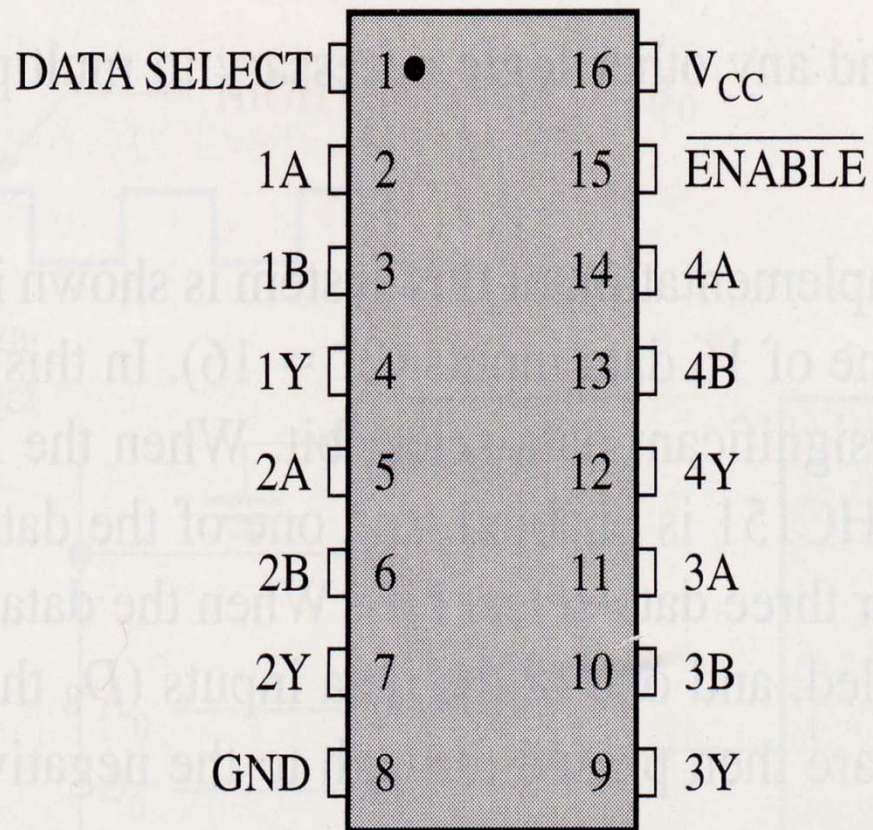


Logic Diagram for 4-bit data selector

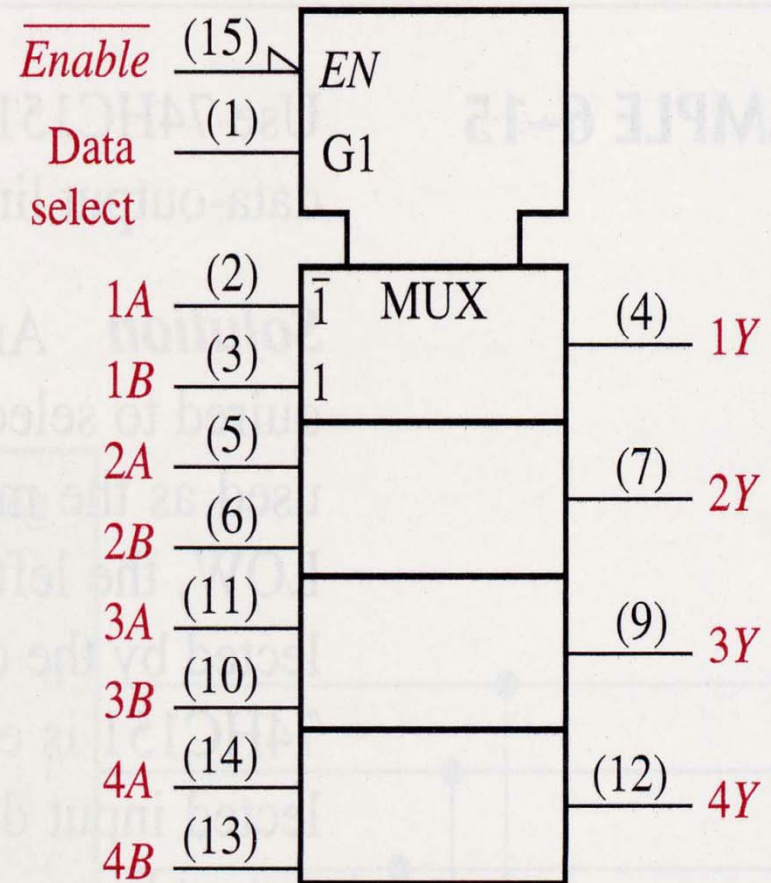




74HC157A

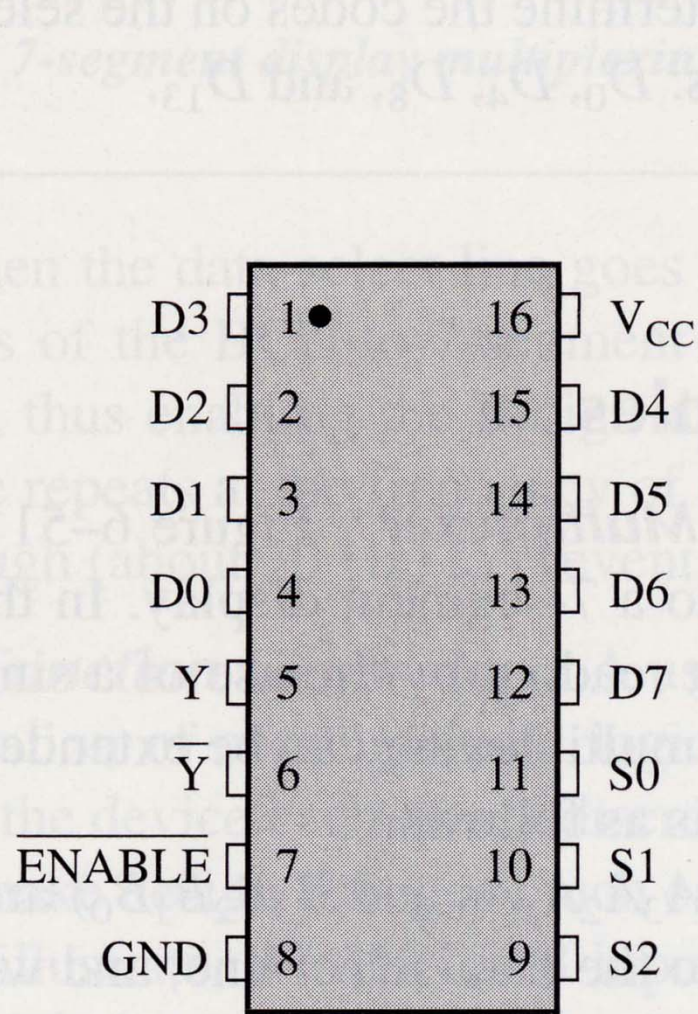


(a) Pin diagram

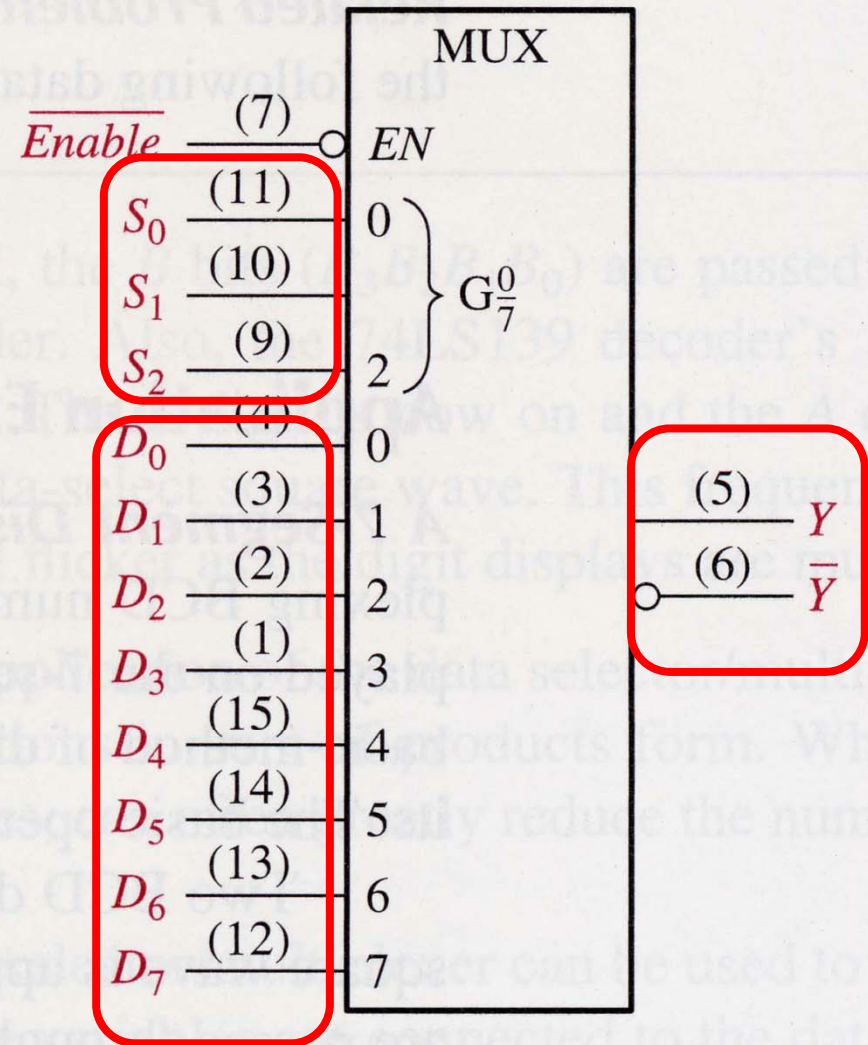


(b) Logic symbol

74HC151

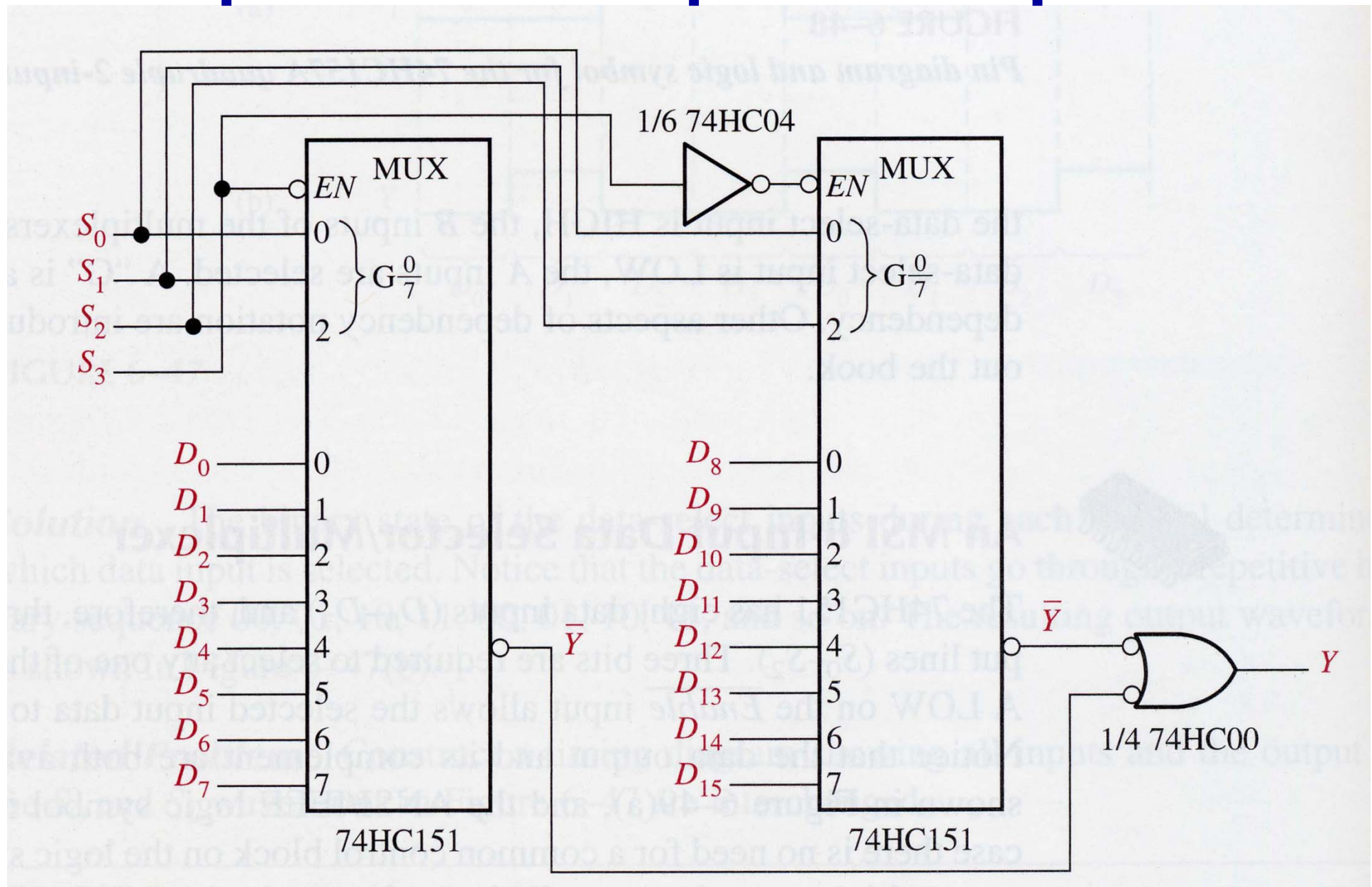


(a) Pin diagram



(b) Logic symbol

Expansion: 16-inputs multiplexer



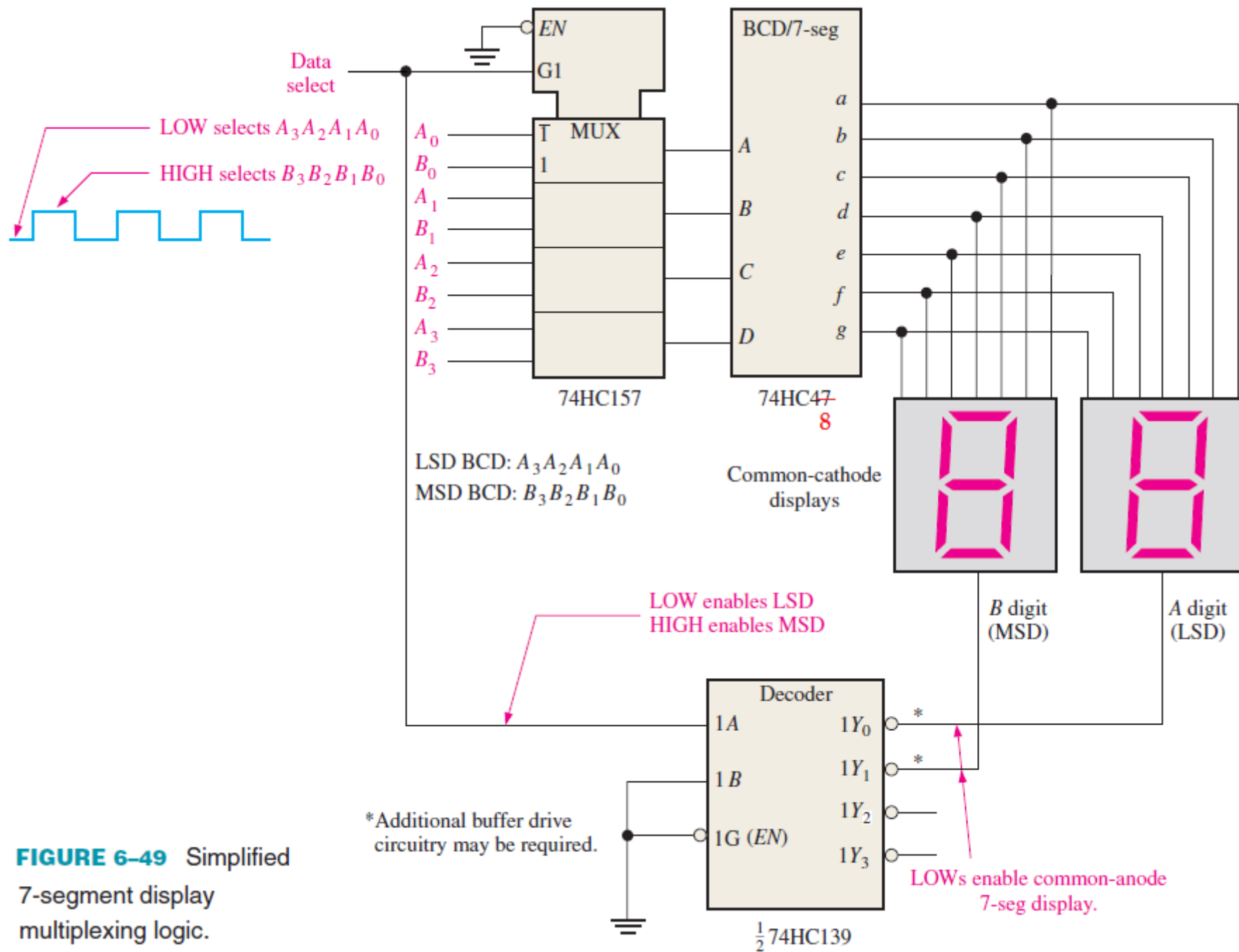
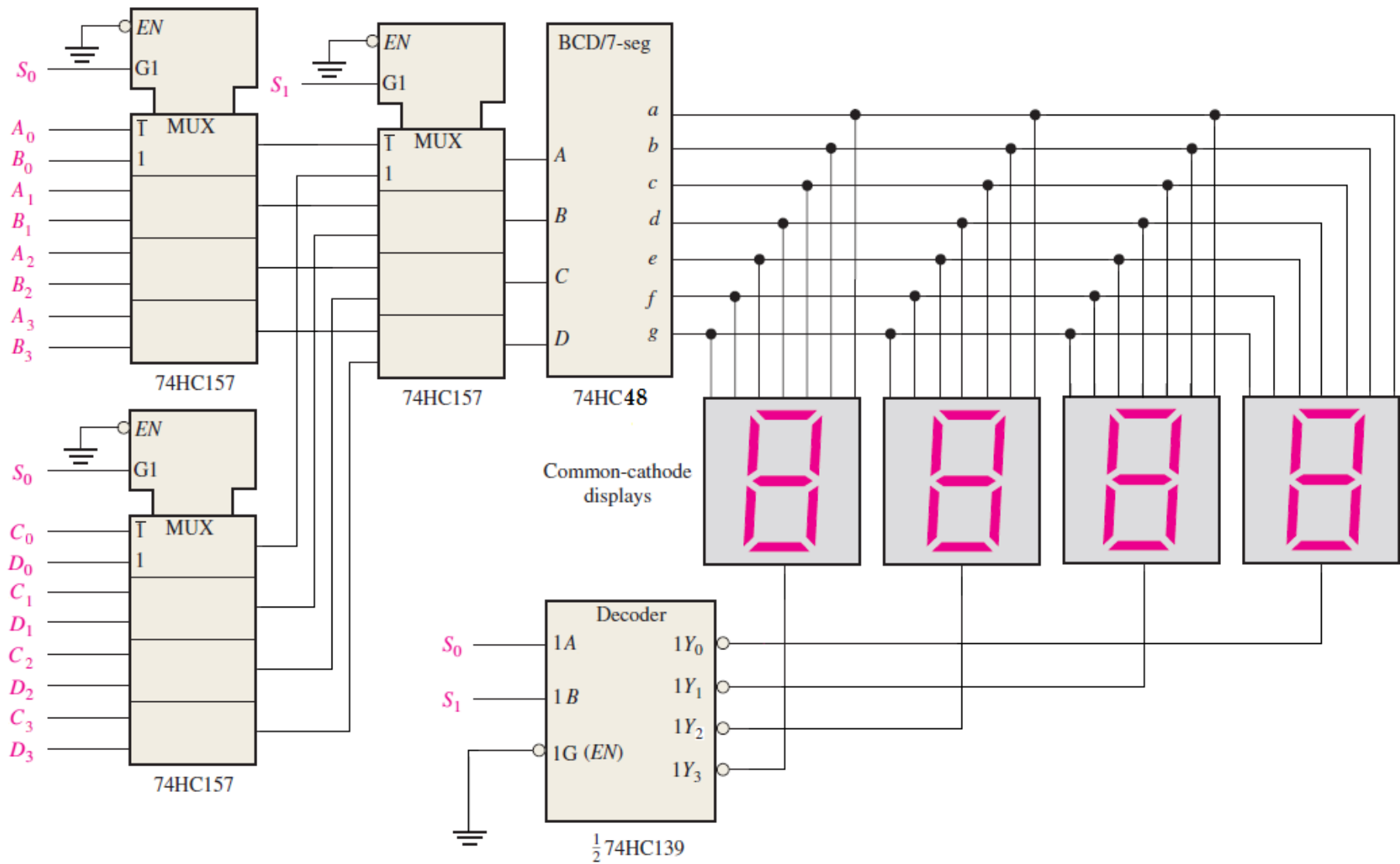


FIGURE 6-49 Simplified 7-segment display multiplexing logic.



Example : Implement the function table by using 74HC151.

Inputs			Output
A_2	A_1	A_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$$Y = \bar{A}_2 \bar{A}_1 A_0 + \bar{A}_2 A_1 A_0 + A_2 \bar{A}_1 A_0 + A_2 A_1 \bar{A}_0 = \sum (1,3,5,6)$$

Logic Function Generator by using data selectors

Remember: you can
also implement this
function by a 3-line-
input-to-8-line-output
decoder like 74LS138.

Expansion: Implement the function table by using 74HC151.

Decimal Digit	Inputs				Output Y
	A ₃	A ₂	A ₁	A ₀	
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	0
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	0
10	1	0	1	0	1
11	1	0	1	1	0
12	1	1	0	0	1
13	1	1	0	1	1
14	1	1	1	0	0
15	1	1	1	1	1

Decimal Digit	Inputs				Output Y
	A_3	A_2	A_1	A_0	
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	0
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	0
10	1	0	1	0	1
11	1	0	1	1	0
12	1	1	0	0	1
13	1	1	0	1	1
14	1	1	1	0	0
15	1	1	1	1	1

Implementation rules (I):

- Observe:

- (1) A 4-variable truth has sixteen combinations if input variables;
- (2) When an 8-bit data selector is used, each input is selected twice:

The first time when A_0 is 0, and the second time when A_0 is 1.

Implementation rules (II):

- Rules:

- (1) If $Y = 0$ for both cases, connect the data input selected by $A_3A_2A_1$ to $GND(0)$
- (2) If $Y = 1$ for both cases, connect the data input selected by $A_3A_2A_1$ to $V_{cc}(1)$
- (3) If Y is different at the two cases, and if $Y = A_0$, connect the corresponding input data to A_0
- (4) If Y is different at the two cases, and if $Y = \overline{A_0}$, connect the corresponding input data to $\overline{A_0}$

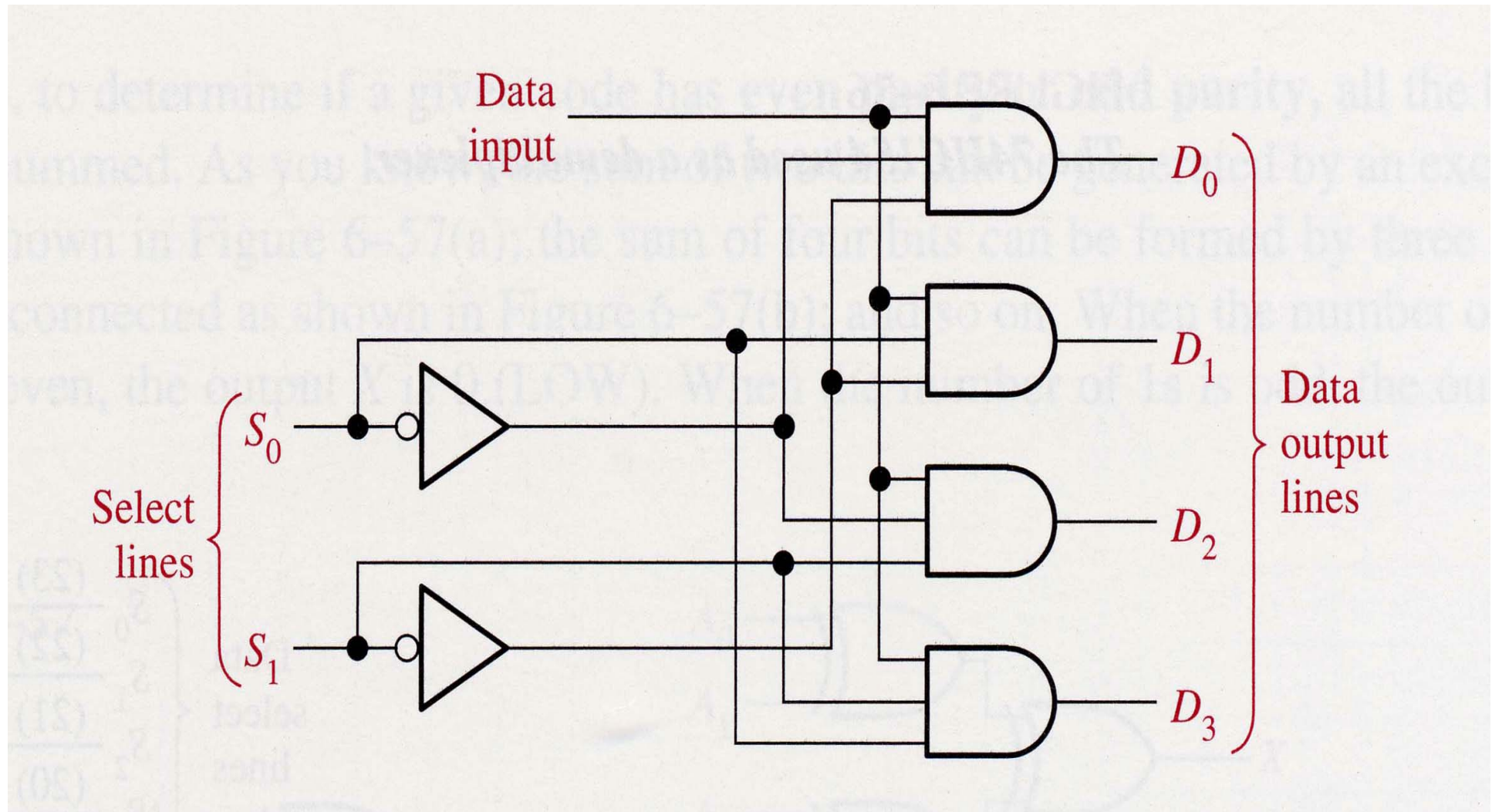
7 Demultiplexer (多路复用器)

A demultiplexer (DEMUX) is a device that reverses the multiplexing function.

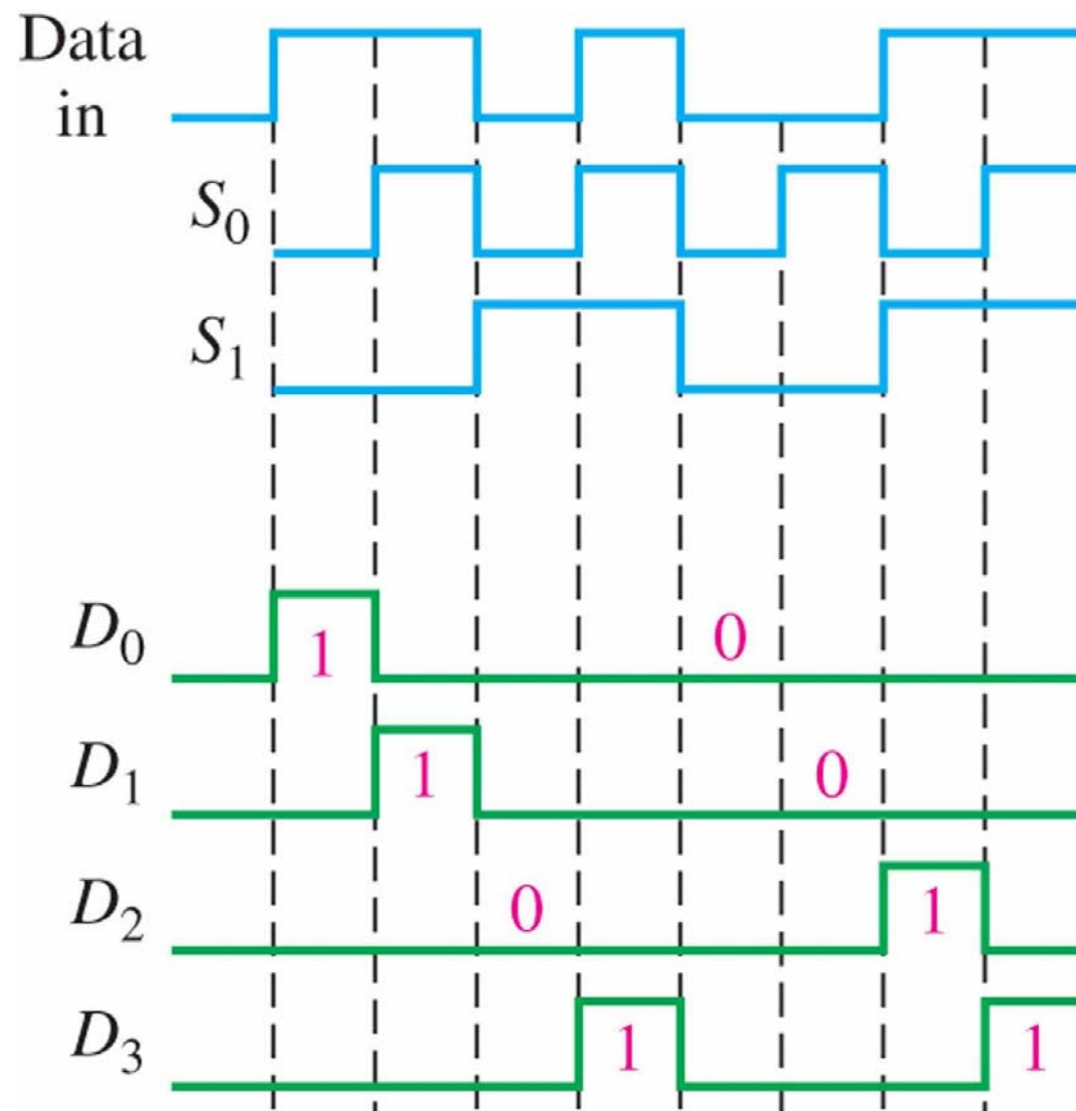
It takes digital information from one line and distributes it to a given number of output lines.

Demultiplexers are also known as data distributors.

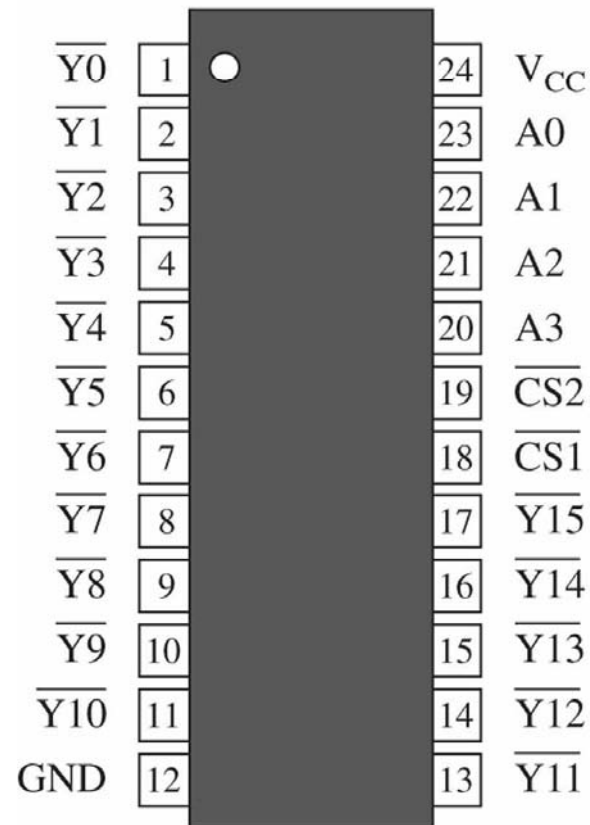
A 1-to-4 DEMUX



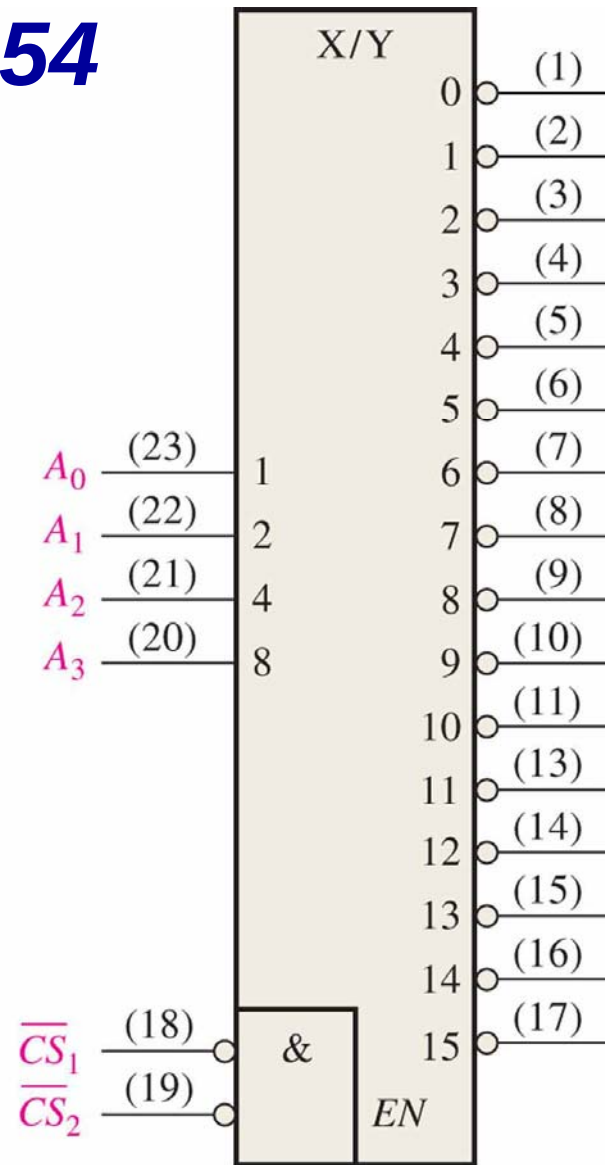
Waveform of the 1-to-4 DEMUX



4-16 Demultiplexer by using 1-of-16 decoder 74HC154



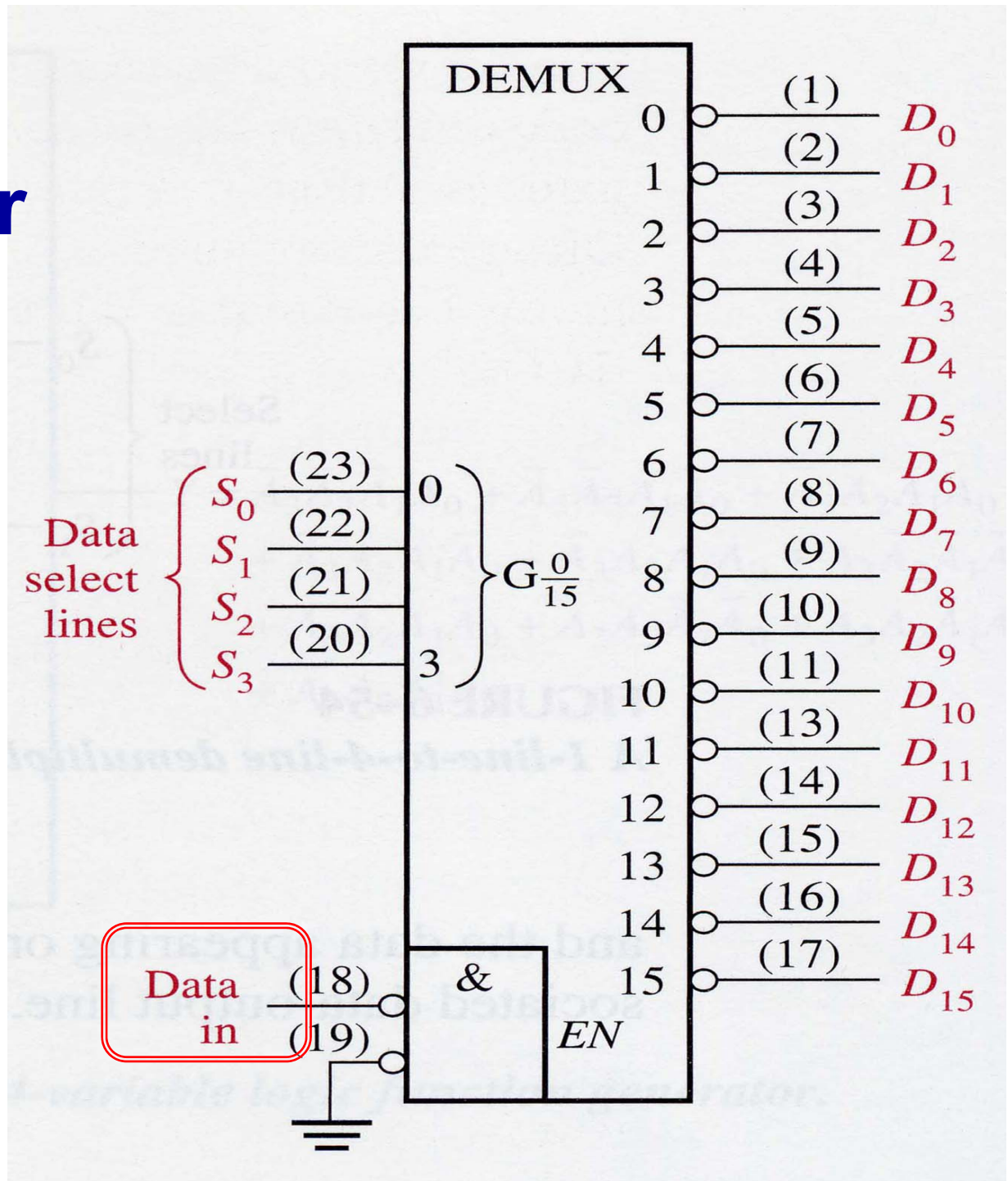
(a) Pin diagram



(b) Logic symbol

4-16 Demultiplexer

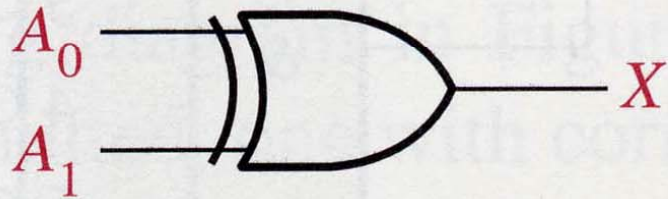
- 74HC154



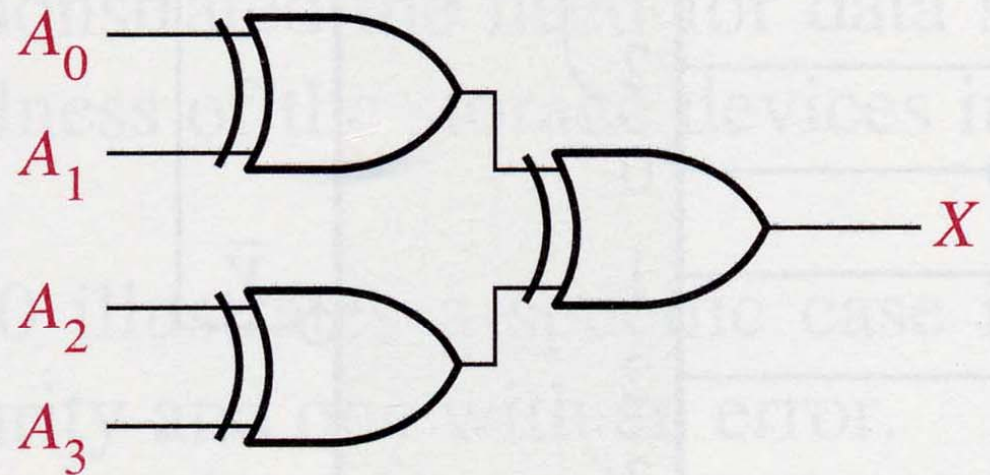
8 Parity Generators/checkers (奇偶产生/校验电路)

- To determine if a given code has *even parity* or *odd parity*.
- Basic Principle:
 - The sum of an *even number* of 1s is always **0**, and the sum of *odd number* of 1s is always **1**.

Sum of 2-bit by XOR



(a) Summing of two bits



(b) Summing of four bits

Data
input

(8)

A

(9)

B

(10)

C

(11)

D

(12)

E

(13)

F

(1)

G

(2)

H

(4)

I

(5)

Σ Even

(6)

Σ Odd

Function Table

Number of Inputs <i>A</i> Thru <i>I</i> That Are HIGH	Outputs	
	Σ Even	Σ Odd
0, 2, 4, 6, 8	H	L
1, 3, 5, 7, 9	L	H

Generator / Checker

- *Parity Generator:*

- For *even* parity generator, take **odd** output as the parity bit;
- For *odd* parity generator, take **even** output as the parity bit.

- *Parity Checker:*

the number of inputs should always be even/odd, when the output is different (odd/even), an error occurs.

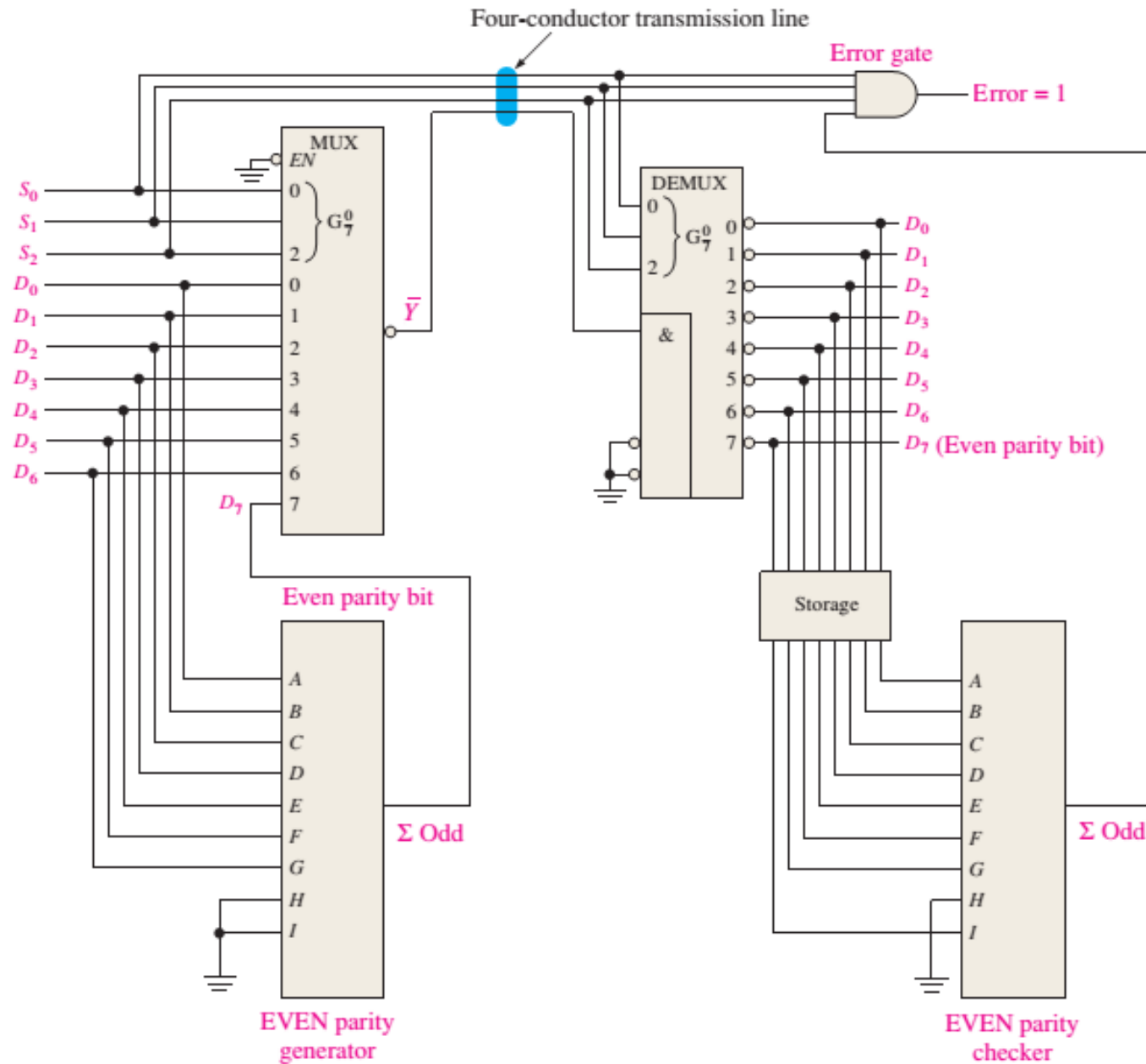


FIGURE 6-58 Simplified data transmission system with error detection.

Quiz

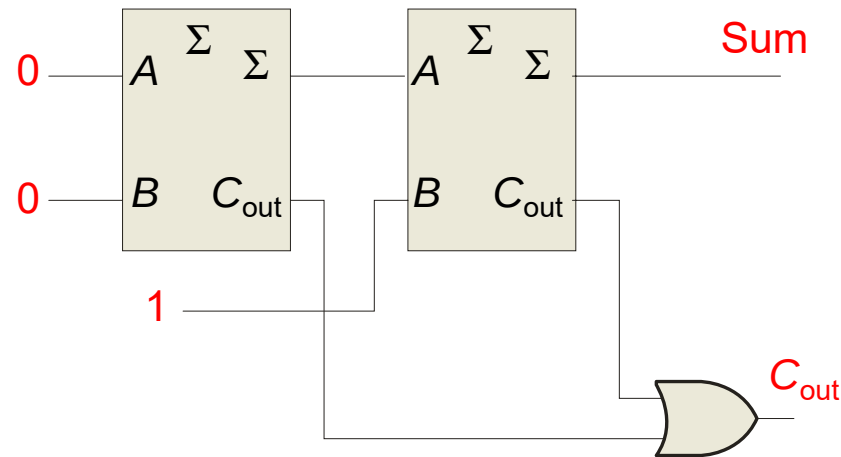
1. For the full-adder shown, assume the input bits are as shown with $A = 0$, $B = 0$, $C_{in} = 1$. The **Sum** and C_{out} will be

a. $\text{Sum} = 0$ $C_{out} = 0$

b. $\text{Sum} = 0$ $C_{out} = 1$

c. $\text{Sum} = 1$ $C_{out} = 0$

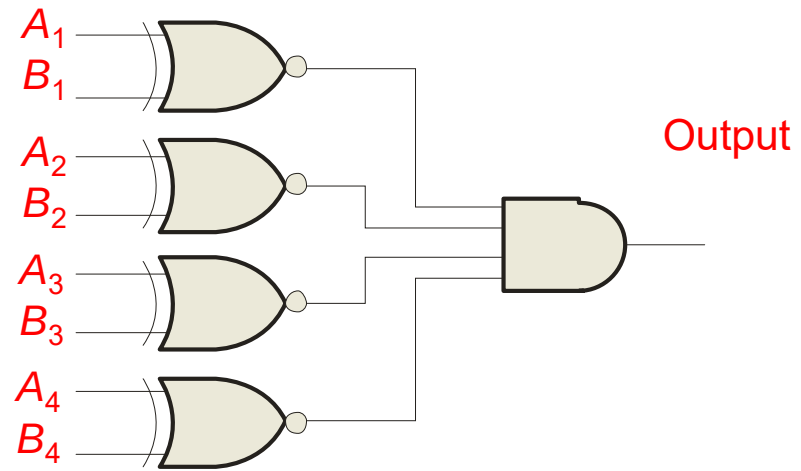
d. $\text{Sum} = 1$ $C_{out} = 1$



Quiz

2. The output will be LOW if

- a. $A < B$
- b. $A > B$
- c. both a and b are correct
- d. $A = B$



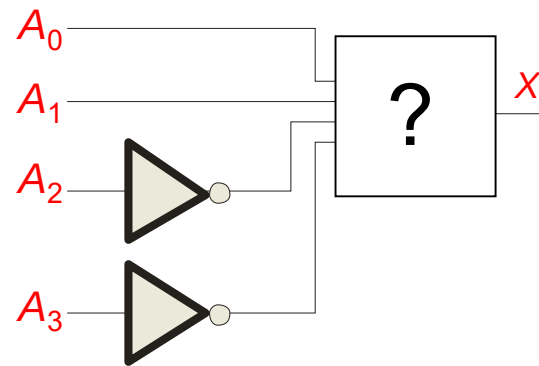
Quiz

3. If you expand two 4-bit comparators to accept two 8-bit numbers, the output of the least significant comparator is
- a. equal to the final output
 - b. connected to the cascading inputs of the most significant comparator
 - c. connected to the output of the most significant comparator
 - d. not used

Quiz

4. Assume you want to decode the binary number 0011 with an active-LOW decoder. The missing gate should be

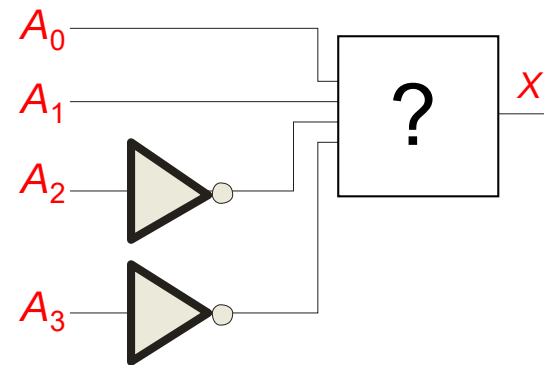
- a. an AND gate
- b. an OR gate
- c. a NAND gate
- d. a NOR gate



Quiz

5. Assume you want to decode the binary number 0011 with an active-HIGH decoder. The missing gate should be

- a. an AND gate
- b. an OR gate
- c. a NAND gate
- d. a NOR gate



Quiz

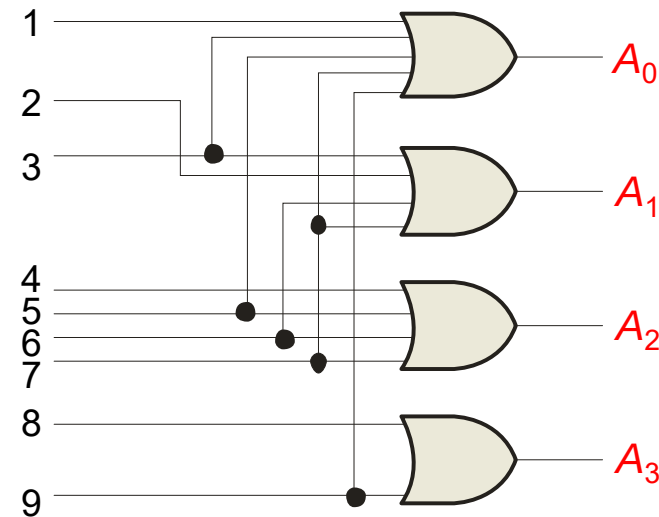
6. The 74138 is a 3-to-8 decoder. Together, two of these ICs can be used to form one 4-to-16 decoder. To do this, connect

- a. one decoder to the LSBs of the input; the other decoder to the MSBs of the input
- b. all chip select lines to ground
- c. all chip select lines to their active levels
- d. one chip select line on each decoder to the input MSB

Quiz

7. The decimal-to-binary encoder shown does not have a zero input. This is because

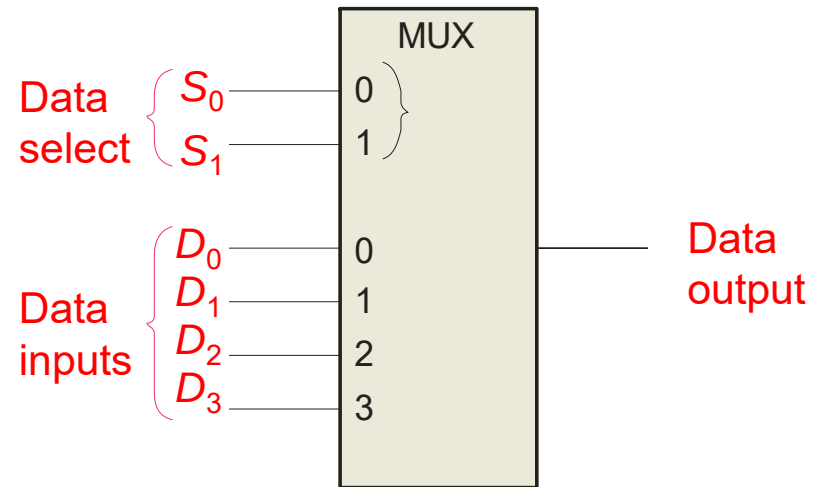
- a. when zero is the input, all lines should be LOW
- b. zero is not important
- c. zero will produce illegal logic levels
- d. another encoder is used for zero



Quiz

8. If the data select lines of the MUX are $S_1S_0 = 11$, the output will be

- a. LOW
- b. HIGH
- c. equal to D_0
- d. equal to D_3



Quiz

9. The 74138 decoder can also be used as
- a. an encoder
 - b. a DEMUX
 - c. a MUX
 - d. none of the above

Quiz

10. The 74LS280 can generate even or odd parity. It can also be used as

- a. an adder
- b. a parity tester
- c. a MUX
- d. an encoder

Homework

- Problems: 2, 4, 6 (注意, 图6-59中应为异或电路, 原图有误), 7, 10, 11(要求有求解过程), 12, 14, 19 (要求有求解过程), 20, 22, 24, 26, 28, 30, 31, 32, 33
- 用4片74HC154级联, 设计一个6bit (6 line – 64 line) 解码器
- 设计74LS147的实现电路, 要求给出设计过程, 包括真值表, 逻辑表达式及化简



Answers

1. c 6. d

2. c 7. a

3. b 8. d

4. c 9. b

5. a 10. b